BANGLADESH UNIVERSITY OF ENGINEERING AND TECHNOLOGY
DEPARTMENT OF ELECTRICAL AND ELECTRONIC ENGINEERING

EEE 312 (January 2026)

Digital Signal Processing I Laboratory

Project Report

Section: A2 Group: 05

Multi-Stage Underwater Image Restoration and Enhancement

Course Instructors:

Dr. S. M. Mahbubur Rahman, Professor
Abrar Assaeem Fuad, Lecturer

Signature of Instructor: _____

Academic Honesty Statement:

IMPORTANT! Please carefully read and sign the Academic Honesty Statement, below. Type the student ID and name, and put your signature. *You will not receive credit for this project experiment unless this statement is signed in the presence of your lab instructor.*

"In signing this statement, We hereby certify that the work on this project is our own and that we have not copied the work of any other students (past or present), and cited all relevant sources while completing this project. We understand that if we fail to honor this agreement, We will each receive a score of ZERO for this project and be subject to failure of this course."

Signature: _____ **Full Name: Rayan Akbar** **Student ID: 2206037**	**Signature:** _____ **Full Name: Md. Tasirul Umor** **Student ID: 2206042**
Signature: _____ **Full Name: Ahnaf Waes** **Student ID: 2206043**	**Signature:** _____ **Full Name: Abdullah Al Hadi** **Student ID: 2206049**

Table of Contents

1 Abstract 1

2 Introduction 1

3 Design 2

3.1 Problem Formulation (PO(b)) 2

3.1.1 Identification of Scope 2

3.1.2 Literature Review 3

3.1.3 Formulation of Problem 4

3.1.4 Analysis 5

3.2 Design Method (PO(a)) 6

3.2.1 Unified Notation and Symbol Glossary 6

3.2.2 Underwater Image Formation and the Five-Phase Strategy 11

3.2.3 Phase 1: Colour Compensation and White Balance (Ancuti et al., 2018) 12

3.2.4 Phase 2: Contrast-Limited Adaptive Histogram Equalization in CIELAB (Zuiderveld, 1994) 15

3.2.5 Phase 3: Multi-Scale Retinex with Colour Restoration (Jobson et al., 1997) 18

3.2.6 Phase 4: Underwater Dark Channel Prior with Safeguards (Drews et al., 2013) 20

3.2.7 Phase 5: Multi-Scale Laplacian-Pyramid Fusion (Ancuti et al., 2018) 24

3.2.8 Candidate Selection and Score-Weighted Blending 28

3.3 Simulation Model (System Architecture) 29

4 Implementation 32

4.1 Description 32

5 Design Analysis and Evaluation 35

5.1 Novelty 35

5.2 Design Considerations (PO(c)) 35

5.2.1 Considerations to public health and safety 35

5.2.2 Considerations to environment 36

5.2.3 Considerations to cultural and societal needs 36

5.3 Investigations (PO(d)) 36

5.3.1 Design of Experiment 36

5.3.2 Data Collection 37

5.3.3 Results and Analysis 38

5.3.4 Interpretation and Conclusions on Data 40

5.4 Limitations of Tools (PO(e)) 41

5.5 Impact Assessment (PO(f)) 44
5.5.1 Assessment of Societal and Cultural Issues 44
5.5.2 Assessment of Health and Safety Issues 44
5.5.3 Assessment of Legal Issues 44
5.6 Sustainability Evaluation (PO(g)) 44
5.7 Ethical Issues (PO(h)) 45
6 Reflection on Individual and Team work (PO(i)) 46
6.1 Individual Contribution of Each Member 46
6.2 Mode of Team Work 46
6.3 Diversity Statement of Team 47
6.4 Log Book of Project Implementation 47
7 Communication to External Stakeholders (PO(j)) 50
7.1 Executive Summary 50
7.2 User Manual 50
7.3 GitHub Link 51
8 Project Management and Cost Analysis (PO(k)) 51
8.1 Bill of Materials 51
8.2 Calculation of Per Unit Cost of Prototype 51
8.3 Calculation of Per Unit Cost of Mass-Produced Unit 51
8.4 Timeline of Project Implementation 52
9 Future Work (PO(l)) 53
10 References 54
11 Appendix B: Full Source Code of MATLAB Software 56

1 Abstract

Underwater photographs suffer from wavelength-dependent absorption, back-scattering and non-uniform artificial lighting, which together produce a blue-green colour cast, low contrast and a veiling haze. This project designs, implements and evaluates a multi-stage, MATLAB-based enhancement framework for single underwater images. From one input the framework generates five complementary candidate restorations: (1) red-channel compensation with Minkowski-norm white balance, (2) contrast-limited adaptive histogram equalization (CLAHE) of the CIELAB lightness channel, (3) multi-scale retinex with colour restoration (MSRCR), (4) a guarded underwater dark-channel-prior (UDCP) dehazer, and (5) a two-input multi-scale Laplacian-pyramid fusion. The operator inspects a numbered preview and may blend any subset of the candidates with user-assigned scores through a generalized fusion engine. The report derives each algorithm from the five source papers, adopts one non-overlapping mathematical notation shared by the equations and the source code, and traces the constants in the code (for example $p = 6$, $\omega = 0.5$, $t_0 = 0.15$, $\sigma_n = 15{,}80{,}250$, $\delta = 0.1$ and $\chi = 0.0035$) to its origin and purpose. A mechanism analysis and a controlled synthetic experiment show that CLAHE cannot correct non-uniform illumination: at the project's clip limit it leaves a spotlight in place while amplifying fine-scale contrast and noise by almost exactly its predictable gain ceiling, which is why the framework treats CLAHE as an optional candidate and not as a mandatory stage. The result is a transparent, reproducible and training-free tool for low-resource inspection and monitoring tasks.

2 Introduction

Light is the carrier of almost all information in underwater inspection, from coral-reef surveys and fisheries monitoring to hull, pier and pipeline inspection. Water, however, is a hostile optical medium. Absorption removes energy from the light beam in a strongly wavelength-dependent way: red light is attenuated within a few metres, green light survives to moderate depths and blue light travels furthest [1], [19]. Scattering by dissolved matter and suspended particles redirects light into the camera as a veil of back-scattered light and blurs the signal that reaches it. Artificial lamps and torches add a further difficulty, because they light the scene non-uniformly and create a bright central spot with a dark periphery. Many inland and coastal waters of Bangladesh carry high sediment loads, so even shallow scenes can show strong turbidity, a green-brown cast and very low contrast.

The practical consequence is that an unprocessed underwater image has (i) a colour cast, (ii) low global contrast, (iii) low local contrast caused by the veiling light, and (iv) spatially uneven brightness. Each symptom is addressed by a different family of algorithms. White-balancing methods repair the colour cast, histogram methods stretch the contrast, retinex methods remove slowly varying illumination, dehazing methods invert the scattering model, and fusion methods combine the strengths of several corrected versions. No single method is reliable for every water type, depth and light source. This observation drives the design philosophy of the project: generate several complementary candidates, then select or blend them under operator control, instead of committing to one monolithic algorithm.

The project therefore implements five enhancement phases and one reference (the unmodified input) in MATLAB. Phase 1 compensates the strongly attenuated red channel and white-balances the image with a Minkowski p-norm estimator [1], [8]. Phase 2 applies CLAHE [2] to the CIELAB lightness channel. Phase 3 is a multi-scale retinex with colour restoration (MSRCR) [3]. Phase 4 is an underwater dark-channel-prior dehazer [4], [6] equipped with two safeguards against its known failure

modes. Phase 5 is a two-input multi-scale Laplacian-pyramid fusion [1]. The same fusion machinery, generalised to K inputs and driven by user scores, forms the final blending stage. The related image-blurriness and light-absorption method of Peng and Cosman [5] is studied as the conceptual counterpart of the dark-channel family.

The principal objectives of the project are:

- to implement a complete, reproducible single-image underwater enhancement pipeline using the MATLAB Image Processing Toolbox;
- to provide a rigorous theoretical account of each referenced algorithm and to map each equation to the exact line, constant and configuration used in the source code;
- to establish one strict notation in which no symbol denotes two different quantities, and to document how the few identifiers that the source code reuses are resolved;
- to investigate, analytically and by a controlled experiment, the limitations of the chosen tools, with special attention to CLAHE on non-uniformly illuminated scenes; and
- to document the project management, cost, sustainability, societal and ethical aspects required by the course.

Underwater image enhancement is an ill-posed inverse problem: the three colour measurements at each pixel must explain four unknowns (three scene radiances and the transmission) in addition to three global background-light values, and the attenuation depends on wavelength, water type and distance [1], [4]. The problem requires depth in several areas of knowledge (optics, signal processing, colour science and numerical analysis), has conflicting requirements (more contrast versus fewer artefacts, more colour correction versus natural appearance), has no single accepted quality measure without a reference image, and has diverse stakeholders (divers, ecologists, inspectors and the public). Section 3.1.4 maps these attributes to the Washington Accord complex-problem profile and compares alternative solutions.

Chapter 3 formulates the problem, presents the design method with the unified notation and the full theory of the five papers, and shows the system architecture and source code. Chapter 4 describes the software implementation. Chapter 5 analyses and evaluates the design, including novelty, investigations, limitations, impact and sustainability. Chapter 6 reflects on individual and team work, Chapter 7 addresses communication to external stakeholders, Chapter 8 covers project management and cost, Chapter 9 outlines future work, and Chapter 10 lists the references.

3 Design

3.1 Problem Formulation (PO(b))

3.1.1 Identification of Scope

The scope of the project is a single-image, training-free, operator-assisted enhancement system for underwater RGB photographs. The boundaries are stated explicitly so that every design decision in the following sections can be traced to them.

In scope: (i) One still RGB image in, one enhanced RGB image out, at a working resolution whose longer side is at most $D_{\max} = 1024$ px. (ii) Classical algorithms drawn from the five referenced papers [1]–[5], implemented in MATLAB with the Image Processing Toolbox. (iii) A candidate-generation stage that produces six images (the input plus five enhancements), a numbered preview, and a selection or score-weighted blending stage. (iv) Documentation of every constant, its origin and its purpose.

Out of scope: Learning-based restoration, polarimetric or structured-light acquisition, stereo or depth sensors, video and temporal coherence, real-time embedded execution, and full-reference benchmarking (no ground-truth images of the scenes are available).

Assumptions:

(a) The input is an sRGB-encoded image with channel values in $[0,1]$ after conversion.

(b) A single, spatially constant background light A_c describes the water column in the field of view.

(c) The medium follows the Jaffe–McGlamery formation model [17], [18] in its simplified single-scattering form.

(d) The operator can judge the visual quality of the candidates, because no reference-free metric is embedded in the pipeline.

Table 1 lists the requirements derived from this scope. The "verification" column states where in the report each requirement is addressed.

Table *1*: *Requirements specification of the enhancement system*

ID	Requirement	Measure or target	Verification
R1	Valid output range	$0 \le J_c^*(x,y) \le 1$; same size as the working input	Clipping operator $\Pi\{\cdot\}$ at every stage (Section 3.2)
R2	Colour-cast reduction	Smaller spread of the channel means $\max_{c,c'}\lvert\bar{J}_c - \bar{J}_{c'}\rvert$ than the input	Phase 1 theory (3.2.3); data in 5.3.2
R3	Contrast gain without gross clipping	Higher local contrast; few pixels at 0 or 1	Phases 2–5 (3.2.4–3.2.7); data in 5.3.2
R4	Artefact containment	No halos, no strong colour shifts, bounded restoration gain	Guards in Phase 4; limits in 5.4
R5	Operator control	Numbered preview and score-weighted blending	`run_one.m`, `blend_score_weighted.m` (3.2.8)
R6	Reproducibility	No random numbers; identical output for identical input and MATLAB version	Code inspection (Chapter 4)
R7	Fault tolerance	A failing candidate must not abort the run	`try/catch` in `enhance_all.m` (4.1)
R8	Practical run time	Seconds per 1024-px image on a laptop CPU	Measurement protocol in 5.3.1

3.1.2 Literature Review

Underwater image formation: McGlamery [17] and Jaffe [18] modelled an underwater image as the sum of a direct component, a forward-scattered component and a back-scattered component. Under the usual simplification (forward scattering neglected) the image of a scene point at distance d is the attenuated radiance plus a veil of background light. Absorption by pure water is strongly selective: the absorption coefficient is about $0.009\ \mathrm{m}^{-1}$ near 450 nm, about $0.06\ \mathrm{m}^{-1}$ near 550 nm and about $0.34\ \mathrm{m}^{-1}$ near 650 nm [19], so red light is the first to disappear (Figure 1). Real water adds dissolved organic matter and particles, which increase both absorption and scattering.

Colour compensation and white balance (Ancuti et al., 2018) [1]: Ancuti et al. propose a single-image method that derives two inputs from a colour-compensated and white-balanced version of the

degraded image, defines three weight maps (Laplacian contrast, saliency and saturation) that promote edges and colour contrast, and blends the inputs with a multi-scale Laplacian-pyramid strategy to avoid halos. The red channel is first compensated with information from the better-preserved green channel, and the classical Gray-World family of illuminant estimators is then applied. This paper is the source of Phase 1, Phase 5 and the blending engine of the present project.

CLAHE (Zuiderveld, 1994) [2]: Adaptive histogram equalization [13] computes a grey-level mapping from the local histogram of each contextual region and therefore enhances local contrast, but it amplifies noise in homogeneous regions. Zuiderveld limits the slope of the mapping by clipping each histogram at a prescribed level, redistributes the clipped counts, and interpolates bilinearly between the mappings of neighbouring tiles to avoid visible tile borders. It is the source of Phase 2.

Multi-scale retinex with colour restoration (Jobson et al., 1997) [3]: Retinex theory [15] models the lightness sensation as a ratio between a pixel and its surround. Jobson et al. average the logarithmic centre/surround ratios of three Gaussian surrounds (15, 80 and 250 px) to combine dynamic-range compression with tonal rendition, and multiply the result by a logarithmic colour-restoration function to repair the desaturation (graying) that the retinex produces when the grey-world assumption fails. It is the source of Phase 3.

Underwater dark channel prior (Drews et al., 2013) [4]: He et al. [6] observed that, in haze-free outdoor images, at least one colour channel has very low intensity in most local patches (the dark channel), so the dark channel of a hazy image measures the veil and yields the transmission map. Drews et al. observed that red is almost always dark under water for a different reason (absorption), which would make the classical prior meaningless, and restrict the dark channel to the green and blue channels. It is the source of Phase 4.

Image blurriness and light absorption (Peng and Cosman, 2017) [5]: Peng and Cosman estimate scene depth from three cues: the maximum-intensity prior (the difference between the red maximum and the green/blue maxima in a patch), the red-channel attenuation, and image blurriness, since distant objects look blurrier. They also estimate the background light from the blurriest, smoothest regions instead of the brightest pixels. In contrast to UDCP, which discards red, they exploit the absorption of red as a depth cue. In this project the method is the conceptual reference for the limitations of Phase 4; it is not implemented in the uploaded code.

Supporting techniques: The guided filter [7], the Minkowski-norm family of colour-constancy estimators [8], [9], the Laplacian pyramid [10], exposure fusion [11] and frequency-tuned saliency [12] complete the toolset.

3.1.3 Formulation of Problem

The simplified underwater image formation model is

$$I_c(x,y) = J_c(x,y)\, t_c(x,y) + A_c\big(1 - t_c(x,y)\big), \qquad c \in \{R,G,B\} \tag{3.1}$$

$$t_c(x,y) = \exp\big(-\beta_c\, d(x,y)\big) \tag{3.2}$$

where I_c is the observed channel, J_c is the restored (scene-radiance) channel that the system tries to estimate, A_c is the global background light, t_c is the transmission, β_c is the wavelength-dependent attenuation coefficient and d is the distance between the camera and the scene point. The first term of (3.1) is the attenuated direct signal, the second is the veiling light. Because $\beta_R > \beta_G > \beta_B$ in clear water, $t_R < t_G < t_B$ at every distance, which is the physical origin of the blue-green cast.

Inverse problem: At every pixel there are three equations (one per channel) but four unknowns (J_R, J_G, J_B and d, hence t), plus the three global values A_c and the three coefficients β_c. The problem is under-determined and needs priors. The priors used by the five papers are: the red channel can be compensated from green (Ancuti), the channel means of a natural scene are equal (Gray-World), the illumination varies slowly while reflectance varies quickly (retinex), at least one of the green or blue channels is dark in a local patch (UDCP), and a good result has high local contrast, good saturation and visually salient regions (fusion weights).

Portfolio formulation: No prior is valid for every scene. The design problem is therefore posed as a portfolio problem. Let $\mathcal{F}_1, \ldots, \mathcal{F}_6$ denote the six candidate operators (identity, Phases 1–5) and $J^{(m)} = \mathcal{F}_m\{I\}$ the corresponding candidates, $\mathcal{J} = \{J^{(1)}, \ldots, J^{(6)}\}$. The operator selects a subset $\mathcal{K} \subseteq \{1, \ldots, 6\}$ with $K = |\mathcal{K}|$ and non-negative scores s_k, and the system returns

$$J^* = \mathcal{B}\left(\left\{J^{(m_k)}\right\}_{k=1}^{K}, \{s_k\}_{k=1}^{K}\right) \quad \text{with} \quad 0 \le J_c^*(x,y) \le 1 \tag{3.3}$$

where $\mathcal{B}$ is the identity for $K = 1$ and the score-weighted multi-scale fusion for $K > 1$. The engineering task is to design the operators $\mathcal{F}_m$ and $\mathcal{B}$ so that (a) each operator has a *bounded failure mode*, meaning a guard or a clipping rule that prevents catastrophic output, and (b) the set $\mathcal{J}$ is *diverse*, meaning that the operators correct different symptoms (colour, global contrast, illumination, haze, local contrast), so that for most water types at least one candidate or blend is acceptable.

3.1.4 Analysis

Complex-problem attributes: Table 2 maps the project to the complex engineering problem attributes and to the knowledge profile cited in the PO statements (K1–K8).

Table *2*: *Complex engineering problem attributes of the project*

Attribute	Evidence in this project	Knowledge profile
Depth of knowledge	First-principles optics, colour science (CIELAB), spatial filtering, multi-resolution analysis and numerical analysis are needed to derive and implement the five algorithms.	K2–K4, K8
Range of conflicting requirements	Contrast gain versus noise and halo artefacts; colour correction versus natural appearance; restoration gain versus noise amplification ($1/t_0$).	K3, K5
Depth of analysis	The inverse problem is under-determined, so priors must be selected, justified and their failure modes analysed (Sections 3.2 and 5.4).	K2, K4
Familiarity of issues	Non-uniform torch lighting, green or turbid water and unknown camera processing are infrequently met in textbook problems.	K4, K8
Extent of applicable codes	No standard prescribes underwater image quality; design relies on research literature and documented toolbox behaviour.	K6, K8
Stakeholders	Divers, ecologists, inspection engineers, students and the wider public with different quality criteria.	K7
Interdependence	Five coupled algorithms, a preview, a selection stage and a blending engine whose parameters interact.	K5

Alternative solutions: Table 3 compares the main alternatives considered. The comparison is qualitative and is based on the properties of each method class, not on measurements by the team.

Table *3*: *Alternative solution architectures*

Alternative	Strengths	Weaknesses for this project	Decision
A. One classical method (for example white balance only, CLAHE only, or UDCP only)	Simple, fast	Each corrects one symptom; each fails on some water types	Rejected as the sole solution
B. Physics-based restoration with depth cues (Peng and Cosman [5])	Principled, uses red absorption	Many interacting thresholds, more code, harder to guard	Studied as reference; not implemented
C. Learning-based enhancement	Strong results on benchmark data	Needs training data and usually a GPU; low transparency; not suitable for a theory-oriented laboratory report	Rejected
D. Classical multi-candidate generation, preview, operator selection and score-weighted multi-scale fusion	Transparent; each stage is explainable; failure of one candidate is contained; no training data	Needs a human decision; no automatic quality metric	Selected

Technical risks and mitigations:

(1) Over-amplification in dehazing is limited by the lower bound t_0 and by the overshoot guard of Phase 4.

(2) Colour shift on green water is limited by the green-dominance bypass of Phase 4.

(3) Division by zero and logarithm of zero are prevented by the guards $\epsilon_{\log}$, $A_{\min}$, ϵ_m and δ.

(4) Failure of one candidate is contained by the `try/catch` block of the registry.

(5) Pyramid artefacts at small image sizes are prevented by the automatic limit on the number of levels.

(6) Illumination-driven artefacts of CLAHE are treated analytically in Section 5.4.

3.2 Design Method (PO(a))

Section 3.2.1 fixes one unified notation. Section 3.2.2 recalls the physics that all five papers share. Sections 3.2.3 to 3.2.8 develop the theory of each phase and map every equation to the MATLAB source.

3.2.1 Unified Notation and Symbol Glossary

The notation obeys eight rules. **(N1)** One symbol has one meaning in the whole report, including the equations of every phase and the code-mapping tables. **(N2)** Where a source paper or the MATLAB documentation uses one letter for different quantities, the later quantity is renamed and the glossary records the rename: α is used only for the channel-compensation strengths α_R, α_B (the Colour-Restoration constants of Jobson et al. are written μ and ν, and the Laplacian-kernel shape parameter

of `fspecial` is written η); β_c is used only for the optical attenuation coefficient; the red channel is written only as the subscript R, so the retinex output is ρ_c; the green channel is written only as the subscript G, so a Gaussian pyramid is always $G_l\{\cdot\}$. **(N3)** Channel labels R, G, B appear only as subscripts. **(N4)** CIELAB coordinates always carry an asterisk (L^*, a^*, b^*), whereas the pyramid operators always carry a level subscript and brace arguments ($L_l\{\cdot\}$, $G_l\{\cdot\}$); a bare L never occurs. **(N5)** Exponentials are written $\exp(\cdot)$, so that e_c can denote the illuminant estimate. **(N6)** Bold lower-case letters are vectors, calligraphic letters are sets or operators. **(N7)** Dummy indices (c', k', j) are local to the equation in which they appear. **(N8)** Every symbol is listed with the MATLAB identifier that holds it, so that the report and the code can be cross-searched.

The MATLAB identifiers themselves are left unchanged, because the code is complete and tested. Table 11 lists the identifiers that the code uses for more than one purpose and the unique symbol assigned to each purpose. The six candidates and the images that flow through the pipeline use the following names: I_c is the input; J_c^{wb}, J_c^{clahe}, J_c^{msrcr}, J_c^{udcp} and J_c^{fus} are the outputs of Phases 1 to 5; $J_c^{(m)}$ is the m-th entry of the candidate registry; and J_c^* is the final result.

Table *4*: *Symbols for the image domain, geometry and outputs*

Symbol	Meaning	Value or range	MATLAB identifier
(x, y), $\mathbf{x}$	Pixel column and row; position vector	$x \in \{1, \ldots, N_{\mathrm{col}}\}$, $y \in \{1, \ldots, N_{\mathrm{row}}\}$	array indices
Ω, $\lvert\Omega\rvert$	Set of all pixels; pixel count $N_{\mathrm{row}}N_{\mathrm{col}}$	—	`numPixels` (`udcp_dehaze.m`)
N_{row}, N_{col}	Number of image rows and columns	e.g. 683, 1024	`H`; `W` or `Wd`
N_{side}	Shorter image side $\min\{N_{\mathrm{row}}, N_{\mathrm{col}}\}$	—	`minDim`
$D_{\max}$	Cap on the longer side of the working image	1024 px	`opts.max_dim`
$c \in \{R, G, B\}$	Colour-channel label (dummy: c')	—	loop variable `c`
$I_c(x, y)$	Input underwater image, channel c	[0,1]	`img`; `R`, `G`, `B` in `wb_underwater.m` and `udcp_dehaze.m`
$\bar{I}_c$	Spatial mean of I_c	[0,1]	`mR`, `mG`, `mB`
I_{gray}	Luma, $0.2989I_R + 0.5870I_G + 0.1140I_B$	[0,1]	`gray`, `rgb2gray`
$J_c(x, y)$	Restored or enhanced image, channel c (generic)	[0,1]	`out`
J^{wb}, J^{clahe}, J^{msrcr}, J^{udcp}, J^{fus}	Outputs of Phases 1–5	[0,1]	candidates `wb`, `clahe`, `msrcr`, `udcp`, `fusion`
$J^{(m)}$, $\mathcal{J}$	m-th registry candidate ($m = 1, \ldots, 6$); set of candidates	6 elements	`cands(m).image`, `cands`
J^*	Final chosen or blended image	[0,1]	`best`, `pick.image`
$\Pi\{\cdot\}$	Clip to [0,1]: $\min\{\max\{\cdot, 0\}, 1\}$	—	`min(max(x,0),1)`
ϵ_{m}	Machine epsilon	2.22×10^{-16}	`eps`

Table *5*: *Symbols of the optical model*

Symbol	Meaning	Unit	MATLAB identifier
$d(x, y)$	Distance between camera and scene point	m	not estimated
β_c	Attenuation coefficient of channel c	m^{-1}	not estimated
A_c	Global background (veiling) light	[0,1]	`A(c)` in `udcp_dehaze.m`
$t(x, y)$, $t_c(x, y)$	Transmission $\exp(-\beta_c d)$; t without subscript is the channel-independent map used by UDCP	(0,1]	`tRaw`, `tRefined`

Table 6: *Symbols of Phase 1 (colour compensation and white balance)*

Symbol	Meaning	Value	MATLAB identifier
α_R, α_B	Red and blue compensation strengths	1.0; 0.0	`cfg.alphaR`, `cfg.alphaB`
I_c^{cp}	Compensated channel ($I_G^{\mathrm{cp}} = I_G$)	[0,1]	`comp`
p	Minkowski exponent	6	`cfg.minkowski_p`
e_c	Minkowski illuminant estimate of channel c	(0,1]	`illum(c)` before normalisation
$\hat{e}_c$	Normalised estimate $e_c/(\max_{c'} e_{c'} + \epsilon_{\mathrm{m}})$	(0,1]	`illum` after normalisation
κ_c	Channel gain $1/(\hat{e}_c + \epsilon_{\mathrm{m}})$	≥ 1	`gains(c)`

Table 7: *Symbols of Phase 2 (CLAHE in CIELAB)*

Symbol	Meaning	Value	MATLAB identifier
L^*, a^*, b^*	CIELAB lightness and chromatic coordinates	$L^* \in [0,100]$	`L`, `a`, `b` in `clahe_contrast.m`
Λ, Λ^{eq}	Normalised lightness $L^*/100$; its equalized version	[0,1]	`L01`, `L01_eq`
$\mathbf{v}$, $\mathbf{v}_{\mathrm{w}}$	CIE tristimulus vector $[v_X, v_Y, v_Z]^{\mathrm{T}}$; white point	D65	inside `rgb2lab`
$\Phi(\cdot)$, $\Xi(\cdot)$	sRGB decoding; CIELAB cube-root compression	—	inside `rgb2lab`
$\mathbf{g} = [g_{\mathrm{r}}, g_{\mathrm{c}}]$	Tile grid (rows, columns)	[8,8]	`cfg.NumTiles`
u, $\mathcal{T}_u$, N_{T}	Tile index; tile region; pixels per tile	—	internal to `adapthisteq`
N_{bin}	Number of histogram bins	256	`NumBins` (default, not set)
$h_u[j]$, $\tilde{h}_u[j]$	Histogram and clipped histogram of tile u, bin j	counts	internal
χ	Normalised clip limit	0.0035	`cfg.ClipLimit`
$\bar{h}$, h_{clip}, h_{ex}	Mean bin count; absolute clip count; total excess	counts	internal
$\phi_u[j]$	Grey-level mapping of tile u (scaled CDF)	[0,1]	internal
ϑ_x, ϑ_y	Bilinear interpolation weights	[0,1]	internal
Range	Output-range option	`'original'`	`cfg.Range`

Table 8: *Symbols of Phase 3 (MSRCR)*

Symbol	Meaning	Value	MATLAB identifier
N_σ, n	Number of retinex scales; scale index	3; 1,2,3	`numScales`; `n`
σ_n	Standard deviation of the n-th Gaussian surround (px)	15, 80, 250	`cfg.sigmas`
λ_n	Weight of scale n ($\sum_n \lambda_n = 1$)	1/3 each	`cfg.weights`
Γ_σ, $*$	Normalised 2-D Gaussian kernel; convolution	$\sum \Gamma_\sigma = 1$	`imgaussfilt`
$\epsilon_{\log}$	Guard against ln0	10^{-6}	`cfg.epsVal`
I_c^ϵ	Guarded input $I_c + \epsilon_{\log}$	—	`imgSafe`
$\rho_{n,c}$, ρ_c	Single-scale and multi-scale retinex output	$\mathbb{R}$	`R` in `msrcr_retinex.m`
$E(x,y)$	Channel sum $\sum_c I_c^\epsilon$	—	`channelSum`
μ	CRF nonlinearity strength (Jobson's α)	125 core; 100 in pipeline	`cfg.alpha`
ν	CRF gain (Jobson's β)	46 core; 40 in pipeline	`cfg.beta`
C_c	Colour-restoration function	$\mathbb{R}$	`CRF`

Symbol	Meaning	Value	MATLAB identifier
ρ_c^{cr}	CRF-weighted retinex $C_c\rho_c$	$\mathbb{R}$	`msrcr`
q_{clip}	Percentile clipped at each end (%)	1	`cfg.clipPct`
ρ_{lo}, ρ_{hi}	Lower and upper stretch bounds	—	`lo`, `hi`
ϵ_{rng}	Guard for a degenerate stretch range	10^{-9}	literal in `rescaleToDisplayGlobal`
g_{J}, o_{J}	Jobson's canonical final gain and offset (not used in the code)	192; -30	—

Table *9*: *Symbols of Phase 4 (UDCP and its safeguards)*

Symbol	Meaning	Value	MATLAB identifier
s_{p}, $\mathcal{P}(\mathbf{x})$	Patch side; patch centred at $\mathbf{x}$	15 (core default 1)	`cfg.patchSize`
M, M^{dark}	$\min\{I_G, I_B\}$; its patch minimum	[0,1]	`minGB`, `darkChannel`
q_{top}, Ω_{top}, N_{top}	Fraction, set and number of dark-channel pixels used for A_c	0.001 (core default 0.1); about 699 px	`cfg.topPercent`, `topIdx`, `numTop`
$A_{\min}$	Floor for A_G and A_B	10^{-6}	literal in `udcp_dehaze.m`
Q, Q^{dark}	Background-normalised minimum; its patch minimum	≥ 0	`normMinGB`, `darkNorm`
ω	Haze-removal coefficient	0.5 (core default 0.1)	`cfg.omega`
t^{raw}, t	Raw and refined transmission	[0,1]	`tRaw`, `tRefined`
t_0	Lower bound of the transmission	0.15 (core default 0.10)	`cfg.t0`
s_{gf}, $\mathcal{N}_{\mathrm{gf}}$	Guided-filter window side $2s_{\mathrm{p}}+1$; window	31	`NeighborhoodSize`
ϵ_{gf}	Guided-filter smoothing	10^{-3}	`DegreeOfSmoothing`
$\mathrm{a}_{\mathbf{z}}$, $\mathrm{b}_{\mathbf{z}}$	Local linear coefficients of the guided filter at window centre $\mathbf{z}$	—	internal
J_c^{dh}	Dehazed image before the safeguard	[0,1]	`dehazed`
ι_{g}	Green-dominance ratio	1.25	literal in `cand_udcp.m`
$\mathrm{l}\tau_{\mathrm{os}}$	Overshoot tolerance	0.18	`cfg.overshoot_tol`
$\Delta_{\max}$	Largest channel-mean shift	[0,1]	`shift`
ψ	Safeguard blending weight	[0,1]	`w` in `cand_udcp.m`

Table *10*: *Symbols of Phase 5 (fusion) and of the selection and blending stage*

Symbol	Meaning	Value	MATLAB identifier
K, k, k'	Number of fusion inputs; input index; dummy index	2 (Phase 5); ≤ 6 (blend)	`N`, `i`
F_k, $F_{k,c}$	k-th fusion input image and its channel c	[0,1]	`candidateA`, `candidateB`, `imgs{i}`
γ	Gamma parameter of input 1	1.5	`cfg.gamma`
ζ, σ_{sh}	Unsharp-mask amount; Gaussian radius (px)	1.0; 2	`cfg.sharpAmount`, `cfg.sharpRadius`
η, $\mathbf{H}_\eta$	Laplacian-kernel shape parameter; 3×3 kernel	0.2	`laplacianAlpha`, `lapKernel` / `lap`
σ_{sal}	Gaussian std of the saliency weight (px)	$max\{4, \mathrm{round}(N_{\mathrm{side}}/16)\}$	`saliencySigma`, `sig`

Symbol	Meaning	Value	MATLAB identifier
U_k^{lap}, U_k^{sat}, U_k^{sal}	Laplacian-contrast, saturation and saliency weights	≥ 0	`WL`, `WSat`, `WS`
U_k	Aggregated (un-normalised) weight	≥ 0	`WA_agg`, `WB_agg`; `W{i}`
δ	Weight regularisation	0.1	`weightRegularization`; `delta`
W_k	Normalised weight map, $\sum_k W_k = 1$	(0,1)	`Wa`, `Wb`; `w{i}`
N_{lev}, N_{req}, N_{cap}	Levels used; levels requested; levels allowed by size	≤ 8	`numLevels`, `numLevelsRequested`, `maxLevels`
$\mathbf{r}_l$	Size (rows, columns) of pyramid level l	—	`sizes{l}`
$\text{rsz}_{\mathbf{r}}\{\cdot\}$	Bilinear resize to size $\mathbf{r}$	—	`imresize(.,sizes,'bilinear')`
$G_l\{\cdot\}$, $L_l\{\cdot\}$	Level l of the Gaussian and Laplacian pyramid	—	`G{i}{l}`, `L{i}{l}`
P_l, $\hat{P}_l$	Fused pyramid level; partially collapsed level	—	`Fused{l}`; `out`
m, $\mathcal{K}$	Registry index; operator-chosen subset of $\{1,\ldots,6\}$	—	`idx`
s_k	Operator score of the k-th chosen candidate	> 0	`wt`, `scores`
ξ, $\pi_{\min}$	Score exponent; floor of the mixing weights	1 (pipeline; core 1.5); 0.05	`weight_bias`, `weight_min`
π_k	Mixing weight of input k, $\sum_k \pi_k = 1$	(0,1)	`g`

Table *11*: *Identifiers that the source code uses for more than one purpose*

Identifier	File	Meaning in the code	Symbol used in Report
`A`	`udcp_dehaze.m`	Background light per channel	A_c
`A`, `B`	`cand_fusion.m`	First and second fusion inputs in the two-image call	F_1, F_2
`R`, `G`, `B`	`wb_underwater.m`, `udcp_dehaze.m`	Colour channels of the image	I_R, I_G, I_B
`R`	`msrcr_retinex.m`	Multi-scale retinex accumulator	ρ_c
`G{i}`	`blend_score_weighted.m`	Gaussian pyramid of the weights	$G_l\{W_k\}$
`L{i}`	`blend_score_weighted.m`	Laplacian pyramid of the inputs	$L_l\{F_k\}$
`L`, `a`, `b`	`clahe_contrast.m`	CIELAB channels	L^*, a^*, b^*
`W`, `Wd`	`ancuti_fusion.m`, `udcp_dehaze.m`; `blend_score_weighted.m`	Image width	N_{col}
`W{i}`	`blend_score_weighted.m`	Aggregated weight scaled by the score	$\pi_k U_k$
`w{i}`	`blend_score_weighted.m`	Normalised weight	W_k
`w`	`cand_udcp.m`	Safeguard blending weight	ψ
`alpha`	`msrcr_retinex.m`, `cand_msrcr.m`	CRF nonlinearity strength	μ
`alphaR`, `alphaB`	`wb_underwater.m`	Channel-compensation strengths	α_R, α_B
`laplacianAlpha`	`ancuti_fusion.m`	Laplacian-kernel shape parameter	η
`beta`	`msrcr_retinex.m`, `cand_msrcr.m`	CRF gain	ν

Identifier	File	Meaning in the code	Symbol used in Report
`sigmas`, `sig`, `saliencySigma`	`msrcr_retinex.m`; fusion files	MSR scales; saliency scale	σ_n; σ_{sal}
`g`	`blend_score_weighted.m`	Mixing weights from the scores	π_k
`s`	`blend_score_weighted.m`	Non-negative scores	s_k
`N`	`blend_score_weighted.m`	Number of images	K
`p`	`wb_underwater.m`; `simplePercentile`	Minkowski exponent; percentile value	p; ρ_{lo}, ρ_{hi}

Table *12*: *Symbols that occur only in the analysis, experiment, cost and comparison sections*

Symbol	Meaning	Where used
$\Theta_c(x,y)$, $\Upsilon_c(x,y)$	Illumination field and reflectance, $I_c = \Theta_c \Upsilon_c$	Retinex theory; Section 5.4
$\mathcal{F}_m$, $\mathcal{B}$	Operator of candidate m; blending operator	Problem formulation
$\mathcal{G}_{\max}$	CLAHE gain ceiling $h_{\text{clip}}/\bar{h}$	Sections 3.2.4, 5.3, 5.4
ϱ, σ_{sp}	Distance from the spot centre; spot width (px)	Synthetic experiment
$\langle\cdot\rangle_{\mathcal{Z}}$, pct_q, $\text{std}_{\mathcal{Z}}$	Mean and standard deviation over a region $\mathcal{Z}$; q-th percentile	Synthetic experiment
SC, $\text{TG}_{\mathcal{Z}}$, HO	Spot contrast; texture gain; halo overshoot	Synthetic experiment
N_{b}, $\sigma_{\text{b},i}$, d^{mip}, d^{r}, d^{b}, $\hat{d}$, ϖ_1, ϖ_2	Blurriness filters and scales; depth cues; fused depth; sigmoid weights	Peng and Cosman method
$\text{Energy}_{\text{img}}$, P_{avg}, t_{img}	Energy per image, average power, time per image	Sustainability
C_{unit}, C_{hw}, C_{lic}, C_{dev}, N_{inst}	Unit cost, hardware, licence and development cost; number of installations	Cost analysis

3.2.2 Underwater Image Formation and the Five-Phase Strategy

Two physical facts shape every design decision. First, the attenuation coefficient depends on wavelength. With the approximate pure-water values $\beta_R \approx 0.34$, $\beta_G \approx 0.057$ and $\beta_B \approx 0.009$ m^{-1} [19], the red component of a white target has fallen to about 10 % after 6.8 m of water, whereas the blue component has lost less than 10 % after 10 m (Figure 1). Turbid water has larger coefficients and also changes their ordering: in water rich in phytoplankton or sediment the blue and red channels are both absorbed and the green channel dominates [1]. Second, the veiling-light term $A_c(1 - t_c)$ grows with distance, so far objects lose contrast even when colour is corrected, and non-uniform artificial light adds a spatially varying multiplier on J_c that the model does not contain.

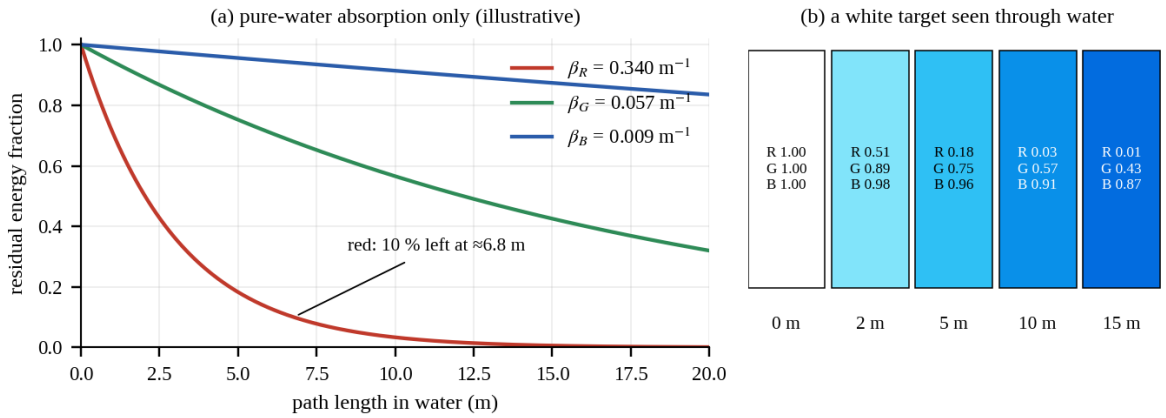


Figure *1*: *Illustrative wavelength-selective attenuation in pure water. (a) Residual energy of the red, green and blue channels versus path length; (b) appearance of a white target after 0–15 m. The coefficients β_c are approximate pure-water absorption values at 650, 550 and 450 nm after Pope and Fry [19]; real water attenuates more strongly.*

Each symptom has a different cause and therefore a different remedy. Table 13 summarizes the allocation of symptoms to phases and papers. Because the remedies partially overlap and partially conflict, the system keeps the outputs of all phases as separate candidates.

Table *13*: *Symptom, physical cause, remedy and source of each phase*

Symptom	Physical cause	Remedy (phase)	Source
Blue-green colour cast	$\beta_R > \beta_G > \beta_B$, hence $t_R < t_G < t_B$	Phase 1: red compensation and Minkowski white balance (candidate `wb`)	Ancuti et al. [1]
Poor global and local tonal contrast	Narrow histogram after attenuation	Phase 2: CLAHE on the CIELAB lightness channel (candidate `clahe`)	Zuiderveld [2]
Uneven illumination, lost colour constancy	Slowly varying light multiplies reflectance	Phase 3: multi-scale retinex with colour restoration (candidate `msrcr`)	Jobson et al. [3]
Veiling haze	Back-scatter term $A_c(1-t)$	Phase 4: underwater dark channel prior with safeguards (candidate `udcp`)	Drews et al. [4]; context from Peng and Cosman [5]
Residual softness, weak local contrast	Forward scattering, sensor	Phase 5: two-input multi-scale fusion (candidate `fusion`)	Ancuti et al. [1]

3.2.3 Phase 1: Colour Compensation and White Balance (Ancuti et al., 2018)

Purpose and source: The first phase removes the colour cast. Ancuti et al. [1] observe that the colours absorbed in deeper water are difficult to restore by white balancing alone. Rescaling a nearly empty red channel would mainly amplify noise, so they first add red information taken from the green channel and only then estimate the illuminant.

The four observations behind the compensation: Paraphrasing [1]: (i) the green channel is the best preserved of the three, because short and medium wavelengths survive longer than red; (ii) green carries opponent-colour information relative to red and is therefore a legitimate source from which to rebuild the lost red; (iii) the amount to add should be proportional to the difference between the mean green and the mean red values, because under the Gray-World assumption this difference measures the imbalance caused by attenuation; and (iv) the addition should be applied less to pixels whose red value is already significant, to prevent the red channel from saturating during the subsequent Gray-World step.

Compensation equations: The four observations translate directly into the compensated red channel

$$I_R^{\mathrm{cp}}(x,y) = I_R(x,y) + \alpha_R\big(\overline{I}_G - \overline{I}_R\big)\big(1 - I_R(x,y)\big)I_G(x,y) \tag{3.4}$$

and, for the blue channel (needed only in extremely turbid water or under heavy phytoplankton blooms [1]),

$$I_B^{\mathrm{cp}}(x,y) = I_B(x,y) + \alpha_B\big(\overline{I}_G - \overline{I}_B\big)\big(1 - I_B(x,y)\big)I_G(x,y) \tag{3.5}$$

while the green channel is left unchanged, $I_G^{\mathrm{cp}} = I_G$. The channel means are $\overline{I}_c = \frac{1}{|\Omega|}\sum_{(x,y)\in\Omega} I_c\,(x,y)$. The four factors of (3.4) implement the four observations in order: the strength α_R is a global trust in the idea; $\big(\overline{I}_G - \overline{I}_R\big)$ is observation (iii); $(1 - I_R)$ is observation (iv); and I_G is observations (i) and (ii), because red is only added where green is actually present.

Properties: If $\overline{I}_G \geq \overline{I}_R$ and $\alpha_R = 1$, then $I_R \leq I_R^{\mathrm{cp}} \leq 1$, because $\big(\overline{I}_G - \overline{I}_R\big)\,I_G \leq 1$ and therefore the increment is at most $1 - I_R$. If $\alpha_R = 0$ the equation reduces to the identity. For a reddish image with $\overline{I}_R > \overline{I}_G$ the correction changes sign and reduces red, and in rare cases the value may leave $[0,1]$, which is why we clipped the compensated image in our project (`comp = min(max(comp,0),1)`).

Worked example (illustrative numbers): Let $\overline{I}_R = 0.15$, $\overline{I}_G = 0.50$ and $\alpha_R = 1$. A pixel with $I_R = 0.05$, $I_G = 0.50$ becomes $I_R^{\mathrm{cp}} = 0.05 + 0.35 \times 0.95 \times 0.50 = 0.216$, a four-fold increase. A pixel with $I_R = 0.15$ becomes 0.299. A pixel that is already strongly red, $I_R = 0.90$, becomes $0.90 + 0.35 \times 0.10 \times 0.50 = 0.918$, almost unchanged. This behavior is plotted in Figure 2(a).

Parameter values in the code: The red strength is $\alpha_R = 1.0$ and the blue strength is $\alpha_B = 0.0$ (`cfg.alphaR`, `cfg.alphaB` in `wb_underwater.m`). Ancuti et al. report that $\alpha = 1$ is appropriate for a wide range of illumination conditions and cameras, which is the origin of $\alpha_R = 1$. Blue compensation is reserved by the authors for extreme turbidity; setting $\alpha_B = 0$ turns it off, so $I_B^{\mathrm{cp}} = I_B$ and the blue channel is never altered by the compensation step.

Illuminant estimation with the Minkowski p-norm: After compensation, the colour of the illuminant is estimated per channel. Buchsbaum's Gray-World hypothesis [9] states that the mean of a natural scene is achromatic, so the illuminant estimate is the channel mean. Finlayson and Trezzi [8] generalized this to the Minkowski p-norm,

$$e_c = \left(\frac{1}{|\Omega|}\sum_{(x,y)\in\Omega}\Big(I_c^{\mathrm{cp}}(x,y)\Big)^p\right)^{1/p} \tag{3.6}$$

which equals the mean for $p = 1$ (Gray-World) and tends to the maximum for $p \to \infty$ (Max-RGB). Larger p gives more weight to bright pixels. Finlayson and Trezzi found $p = 6$ to be the best-performing exponent in their tests, and Ancuti et al. refer to the same family of estimators [1]; we adopted $p = 6$ (`cfg.minkowski_p`). Figure 2(b) shows how the estimate rises from the mean towards the maximum as p grows.

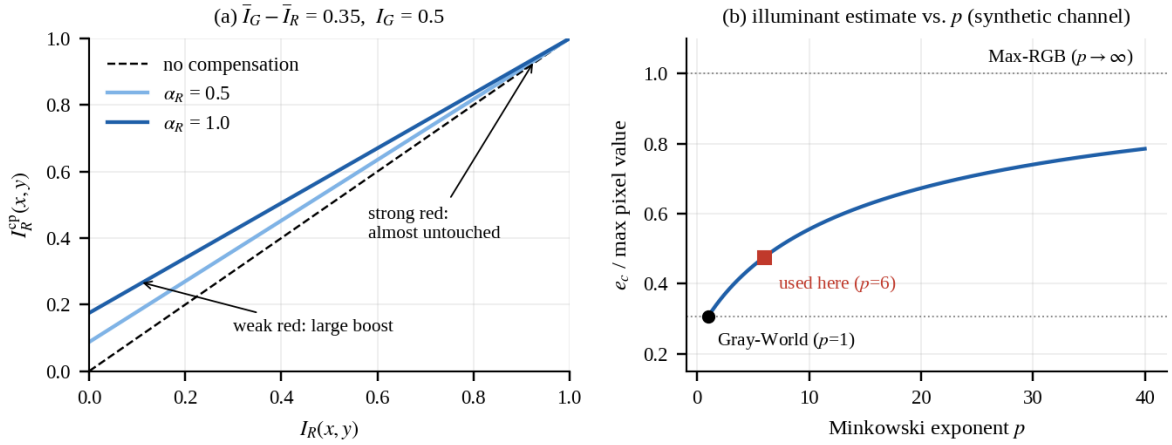


Figure 2: *Behavior of Phase 1. (a) Red compensation of (3.4) for $\bar{I}_G - \bar{I}_R = 0.35$ and $I_G = 0.5$: weak red values are raised strongly, strong red values are almost untouched. (b) Minkowski illuminant estimate versus exponent p for a synthetic right-skewed channel (illustrative): $p = 1$ is Gray-World, $p \to \infty$ is Max-RGB, $p = 6$ is used here.*

Normalization, gains and output: The estimates are normalized by the largest of the three, and the correction is a diagonal (von Kries) scaling:

$$\hat{e}_c = \frac{e_c}{\max_{c'} e_{c'} + \epsilon_{\mathrm{m}}}, \qquad \kappa_c = \frac{1}{\hat{e}_c + \epsilon_{\mathrm{m}}} \tag{3.7}$$

$$J_c^{\mathrm{wb}}(x,y) = \Pi\{\kappa_c \, I_c^{\mathrm{cp}}(x,y)\} \tag{3.8}$$

The guard ϵ_{m} only protects against an all-zero image. Because $\hat{e}_c \leq 1$, every gain satisfies $\kappa_c \geq 1$: the channel with the strongest illuminant keeps gain 1, and the other channels are *boosted* to its level.

Worked example (illustrative): if $(e_R, e_G, e_B) = (0.30, 0.55, 0.60)$ then $\hat{e} = (0.50, 0.917, 1.00)$ and $\kappa = (2.00, 1.09, 1.00)$. This differs from the normalisation to the mean grey used by a textbook Gray-World correction, which can also *reduce* channels. The consequence for the pipeline is that Phase 1 always brightens the image, so strongly lit areas may reach the clipping value 1 (Section 5.4 discusses the effect on CLAHE).

Mapping of the theory to `wb_underwater.m`: Table 14 gives the correspondence line by line.

Table *14*: *Mapping of Phase 1 equations to `wb_underwater.m`*

MATLAB statement	Equation	Remark
`img = min(max(im2double(img),0),1)`	$\Pi\{I\}$	Input forced to [0,1]; alpha plane dropped if present
`mR = mean(R(:)); mG = mean(G(:)); mB = mean(B(:))`	$\bar{I}_c$	Global channel means
`Rc = R + cfg.alphaR*(mG-mR)*(1-R).*G`	(3.4)	$\alpha_R = 1.0$
`Bc = B + cfg.alphaB*(mG-mB)*(1-B).*G`	(3.5)	$\alpha_B = 0.0$, hence `Bc = B`
`comp = min(max(cat(3,Rc,G,Bc),0),1)`	$\Pi\{I^{\mathrm{cp}}\}$	Clip before the norm
`illum(c) = mean(ch(:).^p)^(1/p)`	(3.6)	$p = 6$
`illum = illum/(max(illum)+eps)`	$\hat{e}_c$	Normalise by the maximum
`gains = 1./(illum+eps)`	κ_c	All gains ≥ 1
`out(:,:,c) = comp(:,:,c).*gains(c)`; clip	(3.8)	`cfg.clip = true`

Differences from the original paper:

(1) Ancuti et al. use the Gray-World estimator ($p = 1$) after the compensation, whereas we used $p = 6$.

(2) The paper corrects each channel by its estimated illuminant relative to the grey level, whereas we normalized by the maximum illuminant so that no channel is attenuated.

(3) The paper compensates only red unless the water is extreme; we exposed the blue term but sets it to zero. These are deliberate engineering choices we made here.

3.2.4 Phase 2: Contrast-Limited Adaptive Histogram Equalization in CIELAB (Zuiderveld, 1994)

Purpose: Phase 2 raises local tonal contrast without changing the colour balance produced by Phase 1. It combines the CLAHE algorithm of Zuiderveld [2] with a colour-space decision: the equalization is applied only to the lightness channel of CIELAB.

Histogram equalization and the gain identity: Let a tile contain N_{T} pixels and let $h[j]$ be the number of pixels in grey-level bin $j \in \{0, \dots, N_{\mathrm{bin}} - 1\}$. Classical equalization maps level j to the normalised cumulative histogram,

$$\phi[j] = \frac{1}{N_{\mathrm{T}}} \sum_{i=0}^{j} h\,[i] \tag{3.9}$$

The step of the mapping between neighboring levels is $\phi[j] - \phi[j-1] = h[j]/N_{\mathrm{T}}$, whereas the identity mapping has the step $1/N_{\mathrm{bin}}$. Hence the **local contrast gain** of the mapping at level j equals the histogram height relative to the mean height $N_{\mathrm{T}}/N_{\mathrm{bin}}$:

$$\frac{\phi[j] - \phi[j-1]}{1/N_{\mathrm{bin}}} = \frac{h[j]}{N_{\mathrm{T}}/N_{\mathrm{bin}}} \approx \frac{h[j]}{\bar{h}} \tag{3.10}$$

This identity explains both the strength and the weakness of equalization. Grey levels that many pixels share are stretched apart (gain above one), and rare levels are compressed. If a tile is nearly homogeneous, its histogram has a few tall bins, the gain of those bins approaches N_{bin}, and noise is amplified enormously. Adaptive histogram equalization (AHE) [13] applies (3.9) separately in every contextual region and inherits this weakness.

The CLAHE algorithm: Zuiderveld [2] bounds the gain by clipping the histogram. The steps, with the MATLAB `adapthisteq` conventions [21], [22], are as follows.

Step 1 (tiling): The image is divided into the tile grid $\mathbf{g} = [g_{\mathrm{r}}, g_{\mathrm{c}}] = [8,8]$ (`NumTiles`). For an image of $N_{\mathrm{row}} \times N_{\mathrm{col}}$ pixels each tile has about $\lceil N_{\mathrm{row}}/g_{\mathrm{r}} \rceil \times \lceil N_{\mathrm{col}}/g_{\mathrm{c}} \rceil$ pixels, i.e. N_{T} pixels.

Step 2 (histograms): A histogram $h_u[j]$ with $N_{\mathrm{bin}} = 256$ bins is computed for every tile u.

Step 3 (clip level): The normalized clip limit $\chi \in [0,1]$ is converted into an absolute count per bin:

$$\bar{h} = \left\lfloor \frac{N_{\mathrm{T}}}{N_{\mathrm{bin}}} \right\rfloor, \qquad h_{\mathrm{clip}} = \bar{h} + \mathrm{round}\left(\chi\left(N_{\mathrm{T}} - \bar{h}\right)\right) \tag{3.11}$$

For $\chi = 0$ the clip level equals the mean height, so all bins are equalized to a flat histogram and the mapping is the identity. For $\chi = 1$ nothing is clipped and the method is plain AHE.

Step 4 (clipping and redistribution): The excess $h_{\mathrm{ex}} = \sum_j \max\{h_u[j] - h_{\mathrm{clip}}, 0\}$ is removed and spread uniformly over the bins ($h_{\mathrm{ex}}/N_{\mathrm{bin}}$ per bin), repeating the step until no bin exceeds h_{clip}. The result is the clipped histogram $\tilde{h}_u[j]$ with $\sum_j \tilde{h}_u\,[j] = N_{\mathrm{T}}$.

Step 5 (mapping): $\phi_u[j] = \frac{1}{N_{\mathrm{T}}} \sum_{i \le j} \tilde{h}_u\,[i]$, scaled to the output range.

Step 6 (bilinear interpolation): A pixel is not mapped with the function of its own tile alone. Its grey level j is mapped by the four tiles whose centres surround it, and the four results are blended with the distance weights $\vartheta_x, \vartheta_y \in [0,1]$:

$$\Lambda^{\mathrm{eq}}(x,y) = (1-\vartheta_y)\left[(1-\vartheta_x)\,\phi_{u_{11}}[j] + \vartheta_x\,\phi_{u_{12}}[j]\right] + \vartheta_y\left[(1-\vartheta_x)\,\phi_{u_{21}}[j] + \vartheta_x\,\phi_{u_{22}}[j]\right] \tag{3.12}$$

Pixels within half a tile of the image border use two or one mapping. Interpolation removes visible tile seams but does not remove the *tone* differences between the mappings of neighboring tiles.

Gain ceiling: Combining (3.10) and (3.11), the contrast gain of every tile mapping cannot exceed

$$\mathcal{G}_{\max} = \frac{h_{\mathrm{clip}}}{\bar{h}} \approx 1 + (N_{\mathrm{bin}} - 1)\chi \tag{3.13}$$

The ceiling depends on χ and hardly on the tile size, because $h_{\mathrm{clip}} - \bar{h} \approx \chi\left(N_{\mathrm{T}} - \bar{h}\right)$ scales with N_{T} in the same way as $\bar{h}$. Table 15 evaluates it for the working resolutions of the pipeline; Figure 3 shows the mechanism.

Table *15*: *CLAHE geometry and gain ceiling for typical working sizes (8×8 tiles, 256 bins, (3.11))*

Image (rows×cols)	**Tile (rows×cols)**	N_{T}	$\bar{h}$	h_{clip} **for** $\chi =$ 0.0025 ($\mathcal{G}_{\max}$)	h_{clip} **for** $\chi =$ 0.0035 ($\mathcal{G}_{\max}$)	h_{clip} **for** $\chi =$ 0.01 ($\mathcal{G}_{\max}$)	**Remark**
683×1024	86×128	11 008	43	70 (1.63)	**81 (1.88)**	153 (3.56)	1024×683 photograph
768×1024	96×128	12 288	48	79 (1.65)	**91 (1.90)**	170 (3.54)	4:3 photograph
576×1024	72×128	9 216	36	59 (1.64)	**68 (1.89)**	128 (3.56)	16:9 frame
480×640	60×80	4 800	19	31 (1.63)	**36 (1.89)**	67 (3.53)	small image

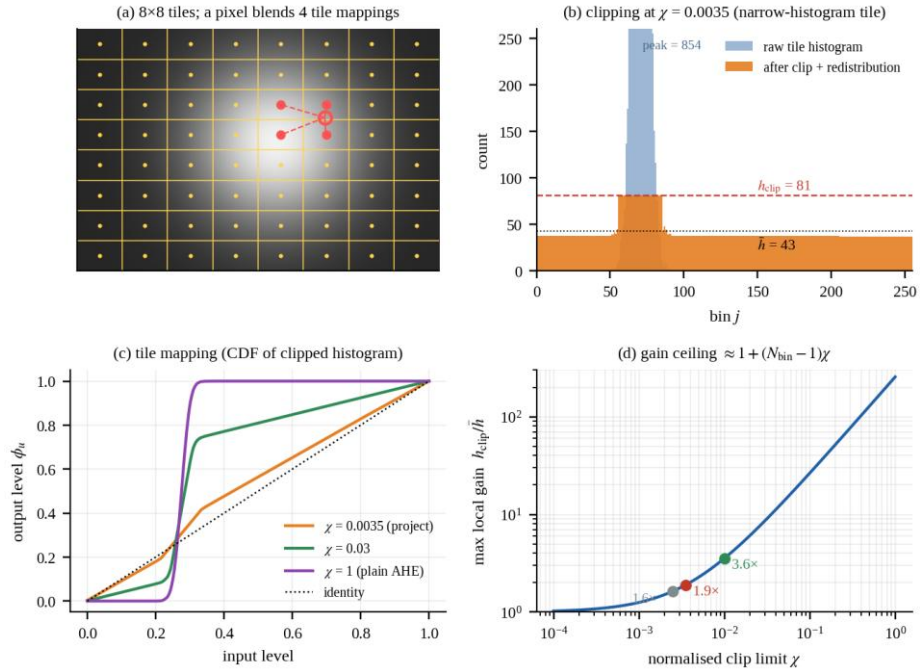


Figure *3*: *Mechanism of Phase 2. (a) The 8×8 tile grid; a pixel is mapped by the four surrounding tile mappings. (b) A narrow-histogram tile (illumination-dominated) clipped at* $h_{clip} = 81$ *for* $\chi = 0.0035$*; the excess is redistributed uniformly. (c) Resulting tile mappings: the project limit* $\chi = 0.0035$ *bends the mapping only mildly, whereas a weak limit approaches plain AHE. (d) The gain ceiling* $1 + (N_{bin} - 1)\chi$ *against* χ *(histogram and mapping are synthetic illustrations).*

Why CIELAB and why only L^*: The pipeline does not equalize the R, G and B channels separately. It converts the white-balanced image to CIELAB, equalizes the lightness, and converts back. The conversion consists of the following steps [23]. The sRGB values are decoded to linear intensities,

$$\Phi(u) = \begin{cases} \dfrac{u}{12.92}, & u \le 0.04045 \\ \left(\dfrac{u + 0.055}{1.055}\right)^{2.4}, & u > 0.04045 \end{cases} \tag{3.14}$$

then mapped to the CIE tristimulus vector with the sRGB (D65) primaries matrix,

$$\mathbf{v} = \begin{bmatrix} v_X \\ v_Y \\ v_Z \end{bmatrix} = \begin{bmatrix} 0.4124 & 0.3576 & 0.1805 \\ 0.2126 & 0.7152 & 0.0722 \\ 0.0193 & 0.1192 & 0.9505 \end{bmatrix} \begin{bmatrix} \Phi(J_R) \\ \Phi(J_G) \\ \Phi(J_B) \end{bmatrix} \tag{3.15}$$

and finally compressed with a cube root,

$$\Xi(\varsigma) = \begin{cases} \varsigma^{1/3}, & \varsigma > (6/29)^3 \\ \dfrac{\varsigma}{3(6/29)^2} + \dfrac{4}{29}, & \text{otherwise} \end{cases} \tag{3.16}$$

$$\begin{aligned} L^* &= 116\,\Xi\left(\frac{v_Y}{v_{\mathrm{w},Y}}\right) - 16, \\ a^* &= 500\left[\Xi\left(\frac{v_X}{v_{\mathrm{w},X}}\right) - \Xi\left(\frac{v_Y}{v_{\mathrm{w},Y}}\right)\right], \\ b^* &= 200\left[\Xi\left(\frac{v_Y}{v_{\mathrm{w},Y}}\right) - \Xi\left(\frac{v_Z}{v_{\mathrm{w},Z}}\right)\right] \end{aligned} \tag{3.17}$$

with the D65 white point $\mathbf{v}_{\mathrm{w}} = [0.9505,\ 1.0000,\ 1.0890]^{\mathsf{T}}$ (MATLAB `rgb2lab` defaults to the sRGB colour space and the D65 white point). There are five reasons for equalizing L^* alone.

- **Colour preservation:** Separate equalization of J_R, J_G, J_B would change the *ratios* between the channels at every pixel and therefore the hue. L^* is decoupled from the chromatic coordinates a^*, b^*, which are passed through unchanged, so the colour balance achieved in Phase 1 survives.
- **No forced re-colouring:** An equalized channel tends towards a flat histogram. In an underwater image the three channels have very different distributions, so channel-wise equalization would force them towards each other and would amount to an uncontrolled, over-strong white balance that also amplifies the noise of the weak red channel.
- **Perceptual uniformity:** L^* is approximately perceptually uniform, so equally populated histogram bins correspond to roughly equal lightness steps and the CDF mapping redistributes *visible* lightness.
- **One-dimensional problem:** CLAHE is a one-dimensional grey-level technique; L^* is the natural scalar for a colour image.
- **Separation of concerns:** Phase 1 owns colour, Phase 2 owns tone; the candidates can therefore be blended without double-correcting colour.

The recombination step is $L^{*\mathrm{eq}} = 100\,\Lambda^{\mathrm{eq}}$ followed by the inverse conversion, and the final clipping,

$$J^{\mathrm{clahe}} = \Pi\{\mathrm{lab2rgb}(100\,\Lambda^{\mathrm{eq}},\ a^*,\ b^*)\}, \qquad \Lambda = \frac{L^*}{100} \tag{3.18}$$

A limitation of this design is that a changed lightness together with the unchanged a^*, b^* can fall outside the sRGB gamut; `lab2rgb` then clips the values, which can alter saturation and hue in very bright or very dark pixels.

Parameter values in the code: `cand_clahe.m` first applies Phase 1 (`uw.wb_underwater`) and then calls `clahe_contrast.m` with `ClipLimit` $\chi = 0.0035$, `NumTiles` $= [8\ 8]$ and `Range` $=$ `'original'`. The core function `clahe_contrast.m` has its own default $\chi = 0.0025$, which the wrapper overrides; MATLAB's own default is 0.01. With $\chi = 0.0035$ the gain ceiling is about 1.9 (Table 15), so the pipeline intentionally uses a conservative limit. `Range='original'` restricts the output to the original range of Λ, so equalization redistributes tones but does not extend the dynamic range. `NumBins` (256) and `Distribution` (`'uniform'`) are MATLAB defaults.

Table *16*: *Mapping of Phase 2 equations to `cand_clahe.m` and `clahe_contrast.m`*

MATLAB statement	Equation	Remark
`img = uw.wb_underwater(img)`	(3.8)	CLAHE operates on the white-balanced image
`labImg = rgb2lab(img)`	(3.14)–(3.17)	sRGB, D65
`L01 = L/100`	Λ	Lightness normalised to [0,1]
`adapthisteq(L01,'ClipLimit',cfg.ClipLimit,'NumTiles',cfg.NumTiles,'Range',cfg.Range)`	(3.9)–(3.12)	$\chi = 0.0035$, $\mathbf{g} = [8,8]$
`L_eq = L01_eq*100`	$L^{*\mathrm{eq}}$	Back to the CIELAB scale
`lab2rgb(cat(3,L_eq,a,b))`; clip	(3.18)	a^*, b^* untouched

3.2.5 Phase 3: Multi-Scale Retinex with Colour Restoration (Jobson et al., 1997)

Purpose. Phase 3 removes slowly varying illumination and restores colour constancy. It is the candidate best suited to uneven lighting.

Illumination–reflectance model and single-scale retinex: Retinex theory [15] assumes that each channel is the product of an illumination field $\Theta_c(x, y)$, which varies slowly, and a reflectance $\Upsilon_c(x, y)$, which carries the scene structure: $I_c = \Theta_c \Upsilon_c$. Taking the logarithm turns the product into a sum, and a spatial low-pass estimate of the illumination can then be subtracted. With the guarded input $I_c^{\epsilon} = I_c + \epsilon_{\log}$ and a Gaussian surround Γ_σ, the single-scale retinex of scale σ_n is

$$\rho_{n,c}(x, y) = \ln I_c^{\epsilon}(x, y) - \ln\left[\Gamma_{\sigma_n} * I_c^{\epsilon}(x, y) + \epsilon_{\log}\right],$$
$$\Gamma_\sigma(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right) \tag{3.19}$$

with $\sum \Gamma_\sigma = 1$ after discretization and $*$ denoting convolution (the centre/surround retinex is analysed by the same authors in [14]). If Θ_c is nearly constant inside the support of Γ_σ, then $\Gamma_\sigma * (\Theta_c \Upsilon_c) \approx \Theta_c\,(\Gamma_\sigma * \Upsilon_c)$ and the illumination cancels:

$$\rho_{n,c} \approx \ln \Upsilon_c - \ln\left(\Gamma_{\sigma_n} * \Upsilon_c\right) \tag{3.20}$$

The output is the logarithm of the reflectance relative to its local average. The approximation fails if the illumination changes within the surround, for example at the edge of a torch beam whose width is comparable to σ_n; that case is the origin of retinex halos.

Multiple scales: A small surround (σ_n of a few tens of pixels) compresses dynamic range and enhances local detail but tends to grey out colours; a large surround gives better tonal rendition but

little compression. Jobson et al. [3] average three scales (a small, an intermediate and a large one) to obtain both. The multi-scale retinex is

$$\rho_c(x,y) = \sum_{n=1}^{N_\sigma} \lambda_n \, \rho_{n,c}(x,y), \qquad N_\sigma = 3, \quad \sigma_n \in \{15,80,250\}, \quad \lambda_n = \frac{1}{3} \tag{3.21}$$

The scales 15, 80 and 250 px and equal weights are the values suggested by Jobson et al. [3] and used in later work [16]. At the working width of 1024 px they correspond to 1.5 %, 7.8 % and 24 % of the image width. The Gaussian kernels of MATLAB `imgaussfilt` have the default size $2\lceil 2\sigma \rceil + 1$ (61, 321 and 1001 taps, i.e. the Gaussian is truncated at $\pm 2\sigma$) and use replicated borders (Figure 4a). Because the scales are expressed in pixels, they are tied to the working resolution $D_{\max} = 1024$; on much smaller images the largest surround would exceed the image.

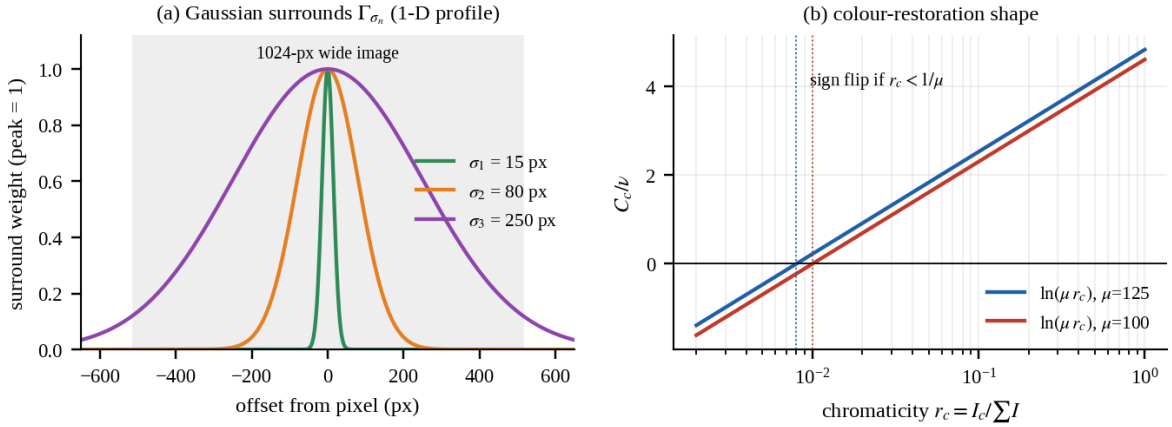


Figure *4*: *Phase 3 constants. (a) The three Gaussian surrounds Γ_{σ_n} with $\sigma_n = 15{,}80{,}250$ px compared with a 1024-px-wide image. (b) The colour-restoration shape $C_c/\nu = \ln(\mu\, r_c)$ against the chromaticity $r_c = I_c/\sum_{c'} I_{c'}$ for $\mu = 125$ (published) and $\mu = 100$ (pipeline); the curves differ only by the constant $\ln 1.25 = 0.22$, and the function becomes negative below $r_c = 1/\mu$.*

Colour restoration: Retinex output of a scene that violates the grey-world assumption (a large uniformly coloured area, as in blue water) is desaturated, because each channel is normalized by its own local mean. Jobson et al. therefore multiply the retinex by a colour-restoration function of the *chromaticity* $r_c = I_c/\sum_{c'} I_{c'}$:

$$C_c(x,y) = \nu[\ln(\mu\, I_c^{\epsilon}) - \ln E] = \nu \ln(\mu\, r_c), \qquad E(x,y) = \sum_{c' \in \{R,G,B\}} I_{c'}^{\epsilon}(x,y) \tag{3.22}$$

$$\rho_c^{\mathrm{cr}} = C_c \, \rho_c \tag{3.23}$$

Here μ controls the strength of the nonlinearity and ν is a gain. Jobson et al. published $\mu = 125$ and $\nu = 46$ together with a final linear gain $g_J = 192$ and offset $o_J = -30$ for display, $J = g_J \rho^{\mathrm{cr}} + o_J$ [3]. The function is positive whenever $r_c > 1/\mu$ and *negative* when a channel carries less than $1/\mu$ of the total (0.8 % for $\mu = 125$, 1 % for $\mu = 100$), in which case the retinex sign of that channel is inverted. This can occur for the red channel in deep, red-depleted scenes.

Display mapping used in the code: The fixed gain and offset are replaced by a data-driven global percentile stretch over all three channels jointly. With ρ_{lo} and ρ_{hi} the q_{clip}-th and $(100 - q_{\mathrm{clip}})$-th percentiles of all values of ρ^{cr}, and $q_{\mathrm{clip}} = 1$,

$$J_c^{\mathrm{msrcr}} = \Pi\left\{\frac{\rho_c^{\mathrm{cr}} - \rho_{\mathrm{lo}}}{\rho_{\mathrm{hi}} - \rho_{\mathrm{lo}}}\right\} \qquad (\text{constant } 0.5 \text{ if } \rho_{\mathrm{hi}} - \rho_{\mathrm{lo}} < \epsilon_{\mathrm{rng}}) \tag{3.24}$$

A single pair of bounds for the three channels preserves the inter-channel relationships created by the colour-restoration function; a per-channel stretch would impose an additional, uncontrolled colour balance.

A consequence of the display mapping: the gain ν cancels: Multiplying ρ^{cr} by any $\nu > 0$ multiplies ρ_{lo} and ρ_{hi} by the same factor, so (3.24) does not change. Therefore, the numerical value of ν (46 or 40) has *no influence* on the output image. Only μ matters, because $C_c = \nu(\ln\mu + \ln r_c)$ and $\ln\mu$ is an additive constant that sets how strongly the weighting depends on chromaticity. Table 17 quantifies the difference between the published value $\mu = 125$ and the value $\mu = 100$ that the pipeline passes.

Table *17*: *Colour-restoration weight $C_c/\nu = ln(\mu r_c)$ for the published and the pipeline value of μ*

Chromaticity r_c	$\mu = 125$	$\mu = 100$	**Ratio** $\ln(125r_c)/\ln(100r_c)$
0.333 (neutral grey)	3.73	3.51	1.06
0.100	2.53	2.30	1.10
0.020 (weak channel)	0.92	0.69	1.32
0.008	≈ 0 (zero at $1/125$)	-0.22	sign differs

Parameter provenance in the code: `msrcr_retinex.m` defines the published defaults $\sigma_n = [15\ 80\ 250]$, $\lambda_n = [1\ 1\ 1]/3$, $\mu = 125$ (`alpha`), $\nu = 46$ (`beta`), $\epsilon_{\log} = 10^{-6}$ and $q_{\mathrm{clip}} = 1$. The pipeline does not call this function directly; it calls the wrapper `cand_msrcr.m`, which sets `cfg.alpha = 100` and `cfg.beta = 40` before the call. Because `fillDefaults` only fills fields that are *missing* or empty, the values $\mu = 100$ and $\nu = 40$ take precedence, and the pipeline therefore runs with $\mu = 100$, $\nu = 40$. The published values $\mu = 125$, $\nu = 46$ remain the theoretical reference and the defaults of the core function. By the argument above, ν is immaterial to the output and $\mu = 100$ versus 125 changes the relative channel weighting by about 6–10 % for ordinary channels and about 30 % for a channel with 2 % chromaticity.

Table *18*: *Mapping of Phase 3 equations to `cand_msrcr.m` and `msrcr_retinex.m`*

MATLAB statement	**Equation**	**Remark**
`imgSafe = img + epsVal; logI = log(imgSafe)`	$\ln I_c^{\epsilon}$	$\epsilon_{\log} = 10^{-6}$
`Gn = imgaussfilt(imgSafe, cfg.sigmas(n))`	$\Gamma_{\sigma_n} * I_c^{\epsilon}$	Applied to each channel separately; replicate padding
`R = R + cfg.weights(n)*(logI - log(Gn+epsVal))`	(3.19), (3.21)	$\lambda_n = 1/3$
`channelSum = sum(imgSafe,3)`	E	Sum over c'
`CRF = cfg.beta*(log(cfg.alpha*imgSafe) - log(channelSum))`	(3.22)	μ, ν
`msrcr = CRF .* R`	(3.23)	Channel-wise product
`rescaleToDisplayGlobal(msrcr, cfg.clipPct)`	(3.24)	Global 1 %/99 % stretch

3.2.6 Phase 4: Underwater Dark Channel Prior with Safeguards (Drews et al., 2013)

Purpose: Phase 4 attacks the veiling-light term of (3.1) directly: it estimates the background light A_c and a transmission map t, and inverts the model. The method belongs to the dark-channel family [6], adapted to water by Drews et al. [4]. The conceptual alternative of Peng and Cosman [5] is discussed at the end of this section.

The dark channel prior and its transmission estimate; He et al. [6] observed that in most local patches of a haze-free outdoor image at least one colour channel contains pixels with very low intensity, so the *dark channel* of the scene radiance, $\min_{\mathbf{y}\in\mathcal{P}(\mathbf{x})}\min_c J_c(\mathbf{y})$, is close to zero. The estimate

of the transmission follows in four steps. Divide (3.1) by A_c and treat t as constant inside the patch $\mathcal{P}(\mathbf{x})$:

$$\frac{I_c(\mathbf{y})}{A_c} = t(\mathbf{x})\,\frac{J_c(\mathbf{y})}{A_c} + 1 - t(\mathbf{x}), \qquad \mathbf{y} \in \mathcal{P}(\mathbf{x}) \tag{3.25}$$

Take the minimum over the patch and over the channels. Because t and $1 - t$ do not depend on $\mathbf{y}$ and c,

$$\min_{\mathbf{y}\in\mathcal{P}(\mathbf{x})} \min_{c} \frac{I_c(\mathbf{y})}{A_c} = t(\mathbf{x}) \left(\min_{\mathbf{y}\in\mathcal{P}(\mathbf{x})} \min_{c} \frac{J_c(\mathbf{y})}{A_c} \right) + 1 - t(\mathbf{x}) \tag{3.26}$$

By the prior the first term on the right is approximately zero, hence $t(\mathbf{x}) \approx 1 - \min_{\mathbf{y}} \min_{c} I_c(\mathbf{y})/A_c$. He et al. keep a small amount of haze for depth perception by multiplying the minimum by a coefficient $\omega < 1$ ($\omega = 0.95$ in [6]), which yields the raw estimate $t^{\mathrm{raw}} = 1 - \omega \mathrm{minmin} I_c/A_c$.

Why the red channel is excluded: Underwater the dark channel prior is violated for a physical reason that has nothing to do with haze. Drews et al. [4] point out that, owing to the high absorption of red light, the red channel is nearly dark in most underwater scenes. If red were included in the minimum, the dark channel would almost always be the red channel, and it would therefore measure *absorption* and not the veiling light. Because $\beta_R \gg \beta_G, \beta_B$, the red transmission t_R is much smaller than t_G and t_B ((3.2)), so a single t derived from red would be far too small for green and blue, and the inversion ($1/t$ gain) would over-amplify them. The *underwater dark channel prior* (UDCP) therefore restricts the minimum to the two least-absorbed channels:

$$M(\mathbf{x}) = \min\{I_G(\mathbf{x}),\, I_B(\mathbf{x})\}, \qquad M^{\mathrm{dark}}(\mathbf{x}) = \min_{\mathbf{y}\in\mathcal{P}(\mathbf{x})} M(\mathbf{y}) \tag{3.27}$$

with a square patch $\mathcal{P}(\mathbf{x})$ of side $s_{\mathrm{p}} = 15$ (Drews et al. and He et al. both use 15×15 patches; in the code `ordfilt2(minGB,1,ones(P,P),'symmetric')` with `P = cfg.patchSize`, i.e. a minimum filter with symmetric borders).

Background-light estimation: The background light A_c is estimated from the pixels that are brightest in M^{dark}, because a pixel that is bright in the *dark* channel must be dominated by veiling light. Let Ω_{top} be the set of the $N_{\mathrm{top}} = \max\{1, \mathrm{round}(q_{\mathrm{top}}\,|\Omega|)\}$ pixels with the largest M^{dark}. Then

$$A_c = \frac{1}{N_{\mathrm{top}}} \sum_{\mathbf{x}\in\Omega_{\mathrm{top}}} I_c\,(\mathbf{x}), \qquad c \in \{R, G, B\} \tag{3.28}$$

With $q_{\mathrm{top}} = 0.001$ (0.1 %, the fraction proposed by He et al. and used by Drews et al.) and a 683×1024 image, $N_{\mathrm{top}} = 699$ pixels. The averaging form of (3.28) is the one that Peng and Cosman give for DCP-style background-light estimation [5]; He et al. instead choose, among these pixels, the single pixel with the highest input intensity. Averaging is more robust to one isolated bright pixel, but it still fails when the brightest region of the dark channel is a bright *object* and not water. Note that the identifier is called `topPercent` although it is used as a *fraction* (0.001 means 0.1 %; the core default 0.1 would mean 10 % of the pixels).

Raw transmission, with the coefficient ω: The channel values are normalized by the background light (with the floor $A_{\min} = 10^{-6}$ for A_G, A_B), the minimum is taken over G, B and over the patch, and the raw transmission is

$$Q(\mathbf{x}) = \min_{c\in\{G,B\}} \frac{I_c(\mathbf{x})}{\max\{A_c, A_{\min}\}},$$
$$t^{\text{raw}}(\mathbf{x}) = \Pi\{1 - \omega\, Q^{\text{dark}}(\mathbf{x})\}, \tag{3.29}$$
$$Q^{\text{dark}}(\mathbf{x}) = \min_{\mathbf{y}\in\mathcal{P}(\mathbf{x})} Q(\mathbf{y})$$

The paper of Drews et al. does not contain the coefficient ω; it is inherited from the DCP literature [6], where it is 0.95. We used $\omega = 0.5$, so only half of the estimated veil is removed. The consequence is quantified in Figure 5: because $Q^{\text{dark}} \leq 1$ for most pixels, $t^{\text{raw}} \geq 0.5$ and the restoration gain $1/t$ cannot exceed 2 until $Q^{\text{dark}} > 1.7$, whereas $\omega = 0.95$ would allow gains up to 10. A low ω is therefore a conservative, noise-limiting and colour-limiting choice, paid for with residual haze.

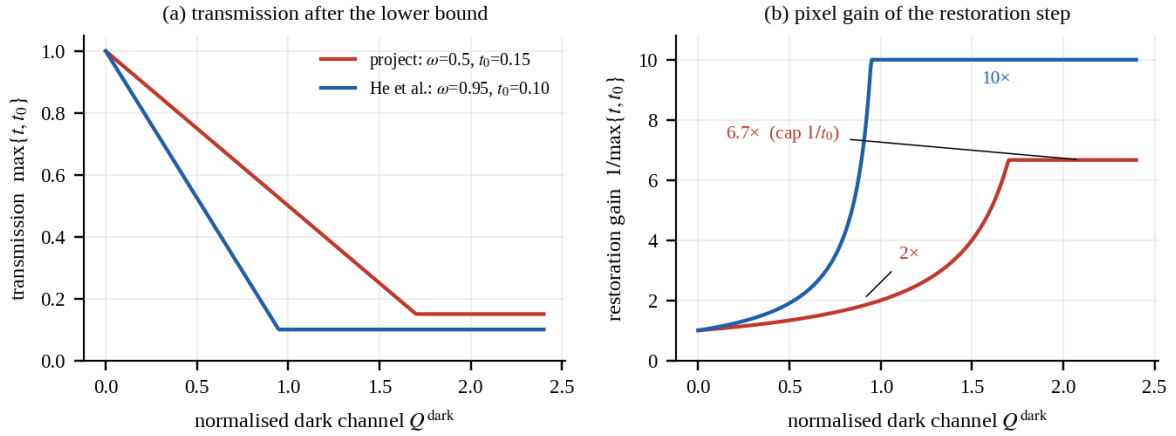


Figure 5: *Effect of ω and t_0 (Phase 4). (a) Transmission after the lower bound and (b) restoration gain $1/max\{t, t_0\}$ against the normalized dark channel Q^{dark}, for the project values $\omega = 0.5$, $t_0 = 0.15$ and for the values of He et al. ($\omega = 0.95$, $t_0 = 0.10$). With the project values the gain stays below 2 until $Q^{dark} > 1.7$ and is capped at $1/t_0 = 6.7$.*

Refinement by the guided filter: The patch minimum makes t^{raw} blocky. Drews et al. refine it with soft matting, which is slow; we used the guided filter of He et al. [7], which is a fast edge-preserving alternative. With the grey image I_{gray} as guidance, each window $\mathcal{N}_{\text{gf}}(\mathbf{z})$ of side $s_{\text{gf}} = 2s_{\text{p}} + 1 = 31$ gets a local linear model $t \approx \mathsf{a}_{\mathbf{z}} I_{\text{gray}} + \mathsf{b}_{\mathbf{z}}$ whose coefficients are

$$\mathsf{a}_{\mathbf{z}} = \frac{\frac{1}{|\mathcal{N}_{\text{gf}}|}\sum_{\mathbf{x}\in\mathcal{N}_{\text{gf}}(\mathbf{z})} I_{\text{gray}}(\mathbf{x})\, t^{\text{raw}}(\mathbf{x}) - \bar{I}_{\text{gray},\mathbf{z}}\, \bar{t}^{\text{raw}}_{\mathbf{z}}}{\text{var}_{\mathbf{z}}\{I_{\text{gray}}\} + \epsilon_{\text{gf}}}, \qquad \mathsf{b}_{\mathbf{z}} = \bar{t}^{\text{raw}}_{\mathbf{z}} - \mathsf{a}_{\mathbf{z}}\, \bar{I}_{\text{gray},\mathbf{z}} \tag{3.30}$$

where a bar with the subscript $\mathbf{z}$ denotes the mean over the window. The refined map is $t(\mathbf{x}) = \bar{\mathsf{a}}_{\mathbf{x}} I_{\text{gray}}(\mathbf{x}) + \bar{\mathsf{b}}_{\mathbf{x}}$, with the coefficients averaged over all windows that contain $\mathbf{x}$. The regularisation $\epsilon_{\text{gf}} = 10^{-3}$ (`'DegreeOfSmoothing'`) is a variance threshold: where the local variance of the guide is much larger than ϵ_{gf} (a standard deviation above $\sqrt{10^{-3}} \approx 0.032$, about 8 grey levels of 255) the filter follows the guide's edges ($\mathsf{a} \approx \text{cov}/\text{var}$), and where it is smaller the output is smoothed. MATLAB's default is 0.01 for a [0,1] guide, so the project filter is ten times more edge-sensitive; MATLAB's

default window is 5×5, whereas the project uses 31×31 so that the blocks of the 15×15 patch minimum are bridged. The refined map is bounded from below,

$$t = \min\{\max\{t_{\text{gf}},\, t_0\},\, 1\}, \qquad t_0 = 0.15 \tag{3.31}$$

Restoration: Solving (3.1) for J_c with a single, channel-independent t gives

$$J_c^{\text{dh}}(x,y) = \Pi\left\{\frac{I_c(x,y) - A_c}{\max\{t(x,y),\, t_0\}} + A_c\right\} \tag{3.32}$$

The lower bound t_0 limits the pixel gain to $1/t_0 = 6.7$ and avoids noise explosion in distant regions. The same t is applied to the red channel too, although physically $t_R < t$; red is therefore under-restored by Phase 4 and is better handled by Phase 1 or by blending.

Safeguards in `cand_udcp.m`: The dark-channel assumption fails in two situations that the wrapper detects cheaply, and each has a bounded response.

(S1) Green-dominance bypass: In green water the green channel is bright everywhere, so M^{dark} is large everywhere, the estimated transmission is small everywhere, and the dehazing produces a strong colour shift. If $\overline{I}_G > \iota_{\text{g}}\overline{I}_B$ and $\overline{I}_G > \iota_{\text{g}}\overline{I}_R$ with $\iota_{\text{g}} = 1.25$, the candidate returns the input unchanged, $J^{\text{udcp}} = I$.

(S2) Overshoot blend: After dehazing, the largest channel-mean shift $\Delta_{\max} = \max_c \left|\overline{J}_c^{\text{dh}} - \overline{I}_c\right|$ is computed. If it exceeds the tolerance $\tau_{\text{os}} = 0.18$, the result is blended back towards the input:

$$J^{\text{udcp}} = (1-\psi)J^{\text{dh}} + \psi I, \qquad \psi = \begin{cases} 0, & \Delta_{\max} \le \tau_{\text{os}} \\ \min\left\{1, \dfrac{\Delta_{\max} - \tau_{\text{os}}}{\tau_{\text{os}}}\right\}, & \Delta_{\max} > \tau_{\text{os}} \end{cases} \tag{3.33}$$

The response is continuous: no correction below a 0.18 mean shift, a linear return to the input between 0.18 and 0.36, and complete rejection above 0.36. The mean shift is a simple detector of a mis-estimated background light, which typically darkens the image or tints it.

Parameter provenance: The core function `udcp_dehaze.m` has the defaults $s_{\text{p}} = 1$, $\omega = 0.1$, $t_0 = 0.10$ and $q_{\text{top}} = 0.1$ (a one-pixel patch and very mild dehazing). The wrapper `cand_udcp.m` overrides all four and the pipeline operates with $s_{\text{p}} = 15$, $\omega = 0.5$, $t_0 = 0.15$, $q_{\text{top}} = 0.001$, $\tau_{\text{os}} = 0.18$. Because the wrapper always provides these fields, the core defaults are never used in the pipeline.

Table *19*: *Mapping of Phase 4 equations to `cand_udcp.m` and `udcp_dehaze.m`*

MATLAB statement	Equation	Remark
`if mG>mB*1.25 && mG>mR*1.25, out = img; return; end`	S1	$\iota_{\text{g}} = 1.25$
`minGB = min(G,B)`	M	Red excluded
`ordfilt2(minGB,1,ones(P,P),'symmetric')`	(3.27)	$s_{\text{p}} = 15$
`numTop = max(1,round(cfg.topPercent*numPixels))`	N_{top}	$q_{\text{top}} = 0.001$
`A(c) = mean(channel(topIdx))`	(3.28)	All three channels
`normMinGB = min(cat(3,G,B)./reshape(safe_A_GB,1,1,2),[],3)`	Q	Floor $A_{\min} = 10^{-6}$
`tRaw = 1 - cfg.omega*darkNorm`	(3.29)	$\omega = 0.5$
`imguidedfilter(tRaw, rgb2gray(img), 'NeighborhoodSize',[2P+1 2P+1], 'DegreeOfSmoothing',1e-3)`	(3.30)	31×31, $\epsilon_{\text{gf}} = 10^{-3}$
`tRefined = min(max(tRefined,cfg.t0),1)`	(3.31)	$t_0 = 0.15$

MATLAB statement	Equation	Remark
`out(:,:,c) = (Ic-A(c))./max(tRefined,cfg.t0) + A(c)`	(3.32)	Same t for all channels
`shift = max(abs(mOut-mIn)); w = min(1,(shift-tol)/tol)`	(3.33)	$\tau_{\mathrm{os}} = 0.18$

The Peng–Cosman perspective (image blurriness and light absorption) [5]: The uploaded code does not implement the method of Peng and Cosman, but its ideas explain the limits of Phase 4 and are part of the theory behind this project. Their image blurriness and light absorption (IBLA) method restores the image in four steps [5], [20]. (1) *Blurriness map.* Because distant objects look blurrier under water, a blurriness map is estimated as the mean absolute difference between the grey image and several Gaussian-blurred versions of it, $\frac{1}{N_{\mathrm{b}}}\sum_{i=1}^{N_{\mathrm{b}}}\left|I_{\mathrm{gray}} - \Gamma_{\sigma_{\mathrm{b},i}} * I_{\mathrm{gray}}\right|$ with increasing $\sigma_{\mathrm{b},i}$, followed by a maximum filter, morphological closing and guided-filter refinement. (2) *Background light* is chosen from candidates found in the blurriest pixels and in the smoothest, most uniform image region (quad-tree search), not from the brightest pixels, which makes it robust to bright objects. (3) *Depth* is estimated by fusing three cues with sigmoid weights ϖ_1, ϖ_2: the maximum-intensity prior d^{mip} (the difference between the largest red value and the largest green/blue value in a patch, since red decays fastest), the red-channel cue d^{r}, and the blurriness cue d^{b},

$$\hat{d}(\mathbf{x}) = \varpi_2\left[\varpi_1\, d^{\mathrm{mip}}(\mathbf{x}) + (1-\varpi_1)\, d^{\mathrm{r}}(\mathbf{x})\right] + (1-\varpi_2)\, d^{\mathrm{b}}(\mathbf{x}) \tag{3.34}$$

and (4) the red transmission $t_R = \exp(-\beta_R d)$ is converted to the green and blue transmissions through the ratio of the attenuation coefficients, $t_G = t_R^{\beta_G/\beta_R}$ and $t_B = t_R^{\beta_B/\beta_R}$, so that each channel is restored with its own transmission, $J_c = (I_c - A_c)/t_c + A_c$.

Table 20 contrasts the two ideas. The important conceptual difference is that UDCP *discards* the red channel because it is absorbed, whereas IBLA *uses* the absorption of red as a depth cue and models channel-dependent transmission. IBLA is not implemented here because it needs many interacting thresholds and several additional processing stages; the safeguards S1 and S2 of Phase 4 are a lightweight substitute that contains the worst failures of UDCP without a depth model.

Table *20*: *Phase 4 (UDCP) compared with the image blurriness and light absorption method*

Aspect	UDCP as implemented (Phase 4)	IBLA (Peng and Cosman)
Role of the red channel	Excluded from the dark channel	Used as a depth cue (absorption)
Transmission	One map t for all channels	Separate t_R, t_G, t_B linked through β_c
Background light	Mean of the brightest dark-channel pixels	From the blurriest and smoothest regions
Depth cues	Dark channel only	Maximum-intensity prior, red channel and blurriness
Known weakness	Bright objects and green water	More parameters, higher complexity
Failure containment	Bypass (S1) and overshoot blend (S2)	Sigmoid-weighted cue fusion

3.2.7 Phase 5: Multi-Scale Laplacian-Pyramid Fusion (Ancuti et al., 2018)

Purpose: Phase 5 combines two complementary versions of the white-balanced image so that each region is taken from the version in which it looks best. The same engine, generalized to K inputs, performs the final blending of Section 3.2.8.

Principle: A pixel-wise weighted average $\sum_k W_k F_k$ produces halos and seams wherever the weights switch abruptly between inputs. Multi-resolution fusion [10], [11] avoids this by decomposing each input into a Laplacian pyramid (band-pass levels), decomposing each weight map into a Gaussian pyramid (progressively blurred versions), and blending *each band with weights of the same scale*: the low-frequency content is mixed over wide, smooth transitions, whereas the fine detail is mixed over narrow ones. Ancuti et al. [1] build their underwater method on this principle.

Derived inputs: Both inputs come from the Phase 1 image J^{wb} (`cand_fusion.m` calls `uw.wb_underwater` first):

$$F_1 = \Pi\left\{\left(J^{\mathrm{wb}}\right)^{1/\gamma}\right\}, \quad \gamma = 1.5, \qquad F_{2,c} = \Pi\{J_c^{\mathrm{wb}} + \zeta\left(J_c^{\mathrm{wb}} - \Gamma_{\sigma_{\mathrm{sh}}} * J_c^{\mathrm{wb}}\right)\}, \quad \zeta = 1,\ \sigma_{\mathrm{sh}} = 2 \tag{3.35}$$

The first input is a gamma-corrected version, the second a sharpened (unsharp-masked) version, with the sharpening applied to each colour channel separately (`imsharpen(...,'Radius',2, 'Amount',1.0)`). Ancuti et al. [1] use the gamma step to correct the global contrast of the white-balanced image and a normalized unsharp mask, $S = (I + \mathcal{N}\{I - \Gamma * I\})/2$ with $\mathcal{N}$ a linear normalisation, for the sharpened input. We instead used the classical unsharp mask with amount $\zeta = 1$ followed by clipping. Two details deserve attention. (i) The exponent used in the code is $1/\gamma = 0.667 < 1$, which lifts shadows and mid-tones, whereas [1] motivates the gamma step by the observation that white-balanced underwater images tend to look too bright, a situation that is normally corrected with an exponent above one. Because the Phase 1 gains are never below one, the white-balanced image can already be bright, and the direction of the gamma step should therefore be checked on the image set. (ii) Sharpening adds noise energy; it is acceptable here only because the saliency and Laplacian weights decide, region by region, whether the sharpened version is used.

Weight maps: For each input the aggregated weight is the sum of three maps. Let $F_{k,\mathrm{gray}}$ be the luma of input k.

Laplacian contrast weight (edges and fine detail): The luma is filtered with the 3×3 Laplacian kernel of MATLAB `fspecial('laplacian',eta)`,

$$\mathbf{H}_\eta = \frac{1}{1+\eta}\begin{bmatrix} \eta & 1-\eta & \eta \\ 1-\eta & -4 & 1-\eta \\ \eta & 1-\eta & \eta \end{bmatrix}, \qquad \mathbf{H}_{0.2} \approx \begin{bmatrix} 0.167 & 0.667 & 0.167 \\ 0.667 & -3.333 & 0.667 \\ 0.167 & 0.667 & 0.167 \end{bmatrix} \tag{3.36}$$

and the absolute response is taken (replicated borders):

$$U_k^{\mathrm{lap}} = \left|\mathbf{H}_\eta \circledast F_{k,\mathrm{gray}}\right| \tag{3.37}$$

The shape parameter $\eta = 0.2$ is the MATLAB default. The coefficients sum to zero, so flat regions give zero response; $\eta = 0$ gives the classical 4-neighbour Laplacian and $\eta = 1$ a diagonal one. For inputs in $[0,1]$ the response lies in $[-3.33, 3.33]$.

Saturation weight (colourfulness and exposure): We used the standard deviation of the three colour values at each pixel,

$$U_k^{\mathrm{sat}} = \sqrt{\frac{1}{2}\sum_{c\in\{R,G,B\}}\left(F_{k,c} - \overline{F}_k\right)^2}, \qquad \overline{F}_k = \frac{1}{3}\sum_c F_{k,c} \tag{3.38}$$

where the factor $1/2$ results from MATLAB's `std(...,0,3)`, which normalizes by $N - 1 = 2$. The maximum value is $1/\sqrt{3} = 0.577$ for a pure primary colour. In [1], the saturation weight measures

the deviation of the three channels from the luminance with the factor $1/3$; this measure is an equivalent colourfulness indicator but not numerically identical.

Saliency weight (objects that lose prominence under water): Ancuti et al. use the frequency-tuned saliency of Achanta et al. [12], the distance between a slightly blurred pixel and the mean colour of the image. We used a luma centre–surround contrast with a large Gaussian,

$$U_k^{\text{sal}} = \left|F_{k,\text{gray}} - \Gamma_{\sigma_{\text{sal}}} * F_{k,\text{gray}}\right|, \qquad \sigma_{\text{sal}} = \max\left\{4,\ \text{round}\left(\frac{N_{\text{side}}}{16}\right)\right\} \tag{3.39}$$

So, the surround scales with the image: $\sigma_{\text{sal}} = 43$ px for 683×1024 and 48 px for 768×1024. The three weights are summed,

$$U_k = U_k^{\text{lap}} + U_k^{\text{sat}} + U_k^{\text{sal}} \tag{3.40}$$

Normalization and the regularization δ: The aggregated weights of the K inputs are normalised pixel by pixel,

$$W_k(x,y) = \frac{U_k(x,y) + \delta}{\sum_{k'=1}^{K} U_{k'}(x,y) + K\delta}, \qquad \delta = 0.1 \tag{3.41}$$

(for $K = 2$ the code writes `denom = WA_agg + WB_agg + 2*delta`). The term δ has four effects. (i) The weights sum to one exactly, $\sum_k W_k = 1$. (ii) Division by zero is impossible, because the denominator is at least $K\delta = 0.2$. (iii) Where all weights are small (flat, dark or featureless regions) the result tends to $1/K$, so the inputs are averaged and the arbitrary, noise-driven winner-take-all behaviour of a $0/0$ ratio is avoided. (iv) No input is ever excluded completely: $W_k \geq \delta/(\sum_{k'} U_{k'} + K\delta) > 0$, which suppresses the artefacts caused by hard weight transitions. The value $\delta = 0.1$ is the one used by Ancuti et al. [1]. Figure 6(a) shows how larger δ compresses the range of W_1.

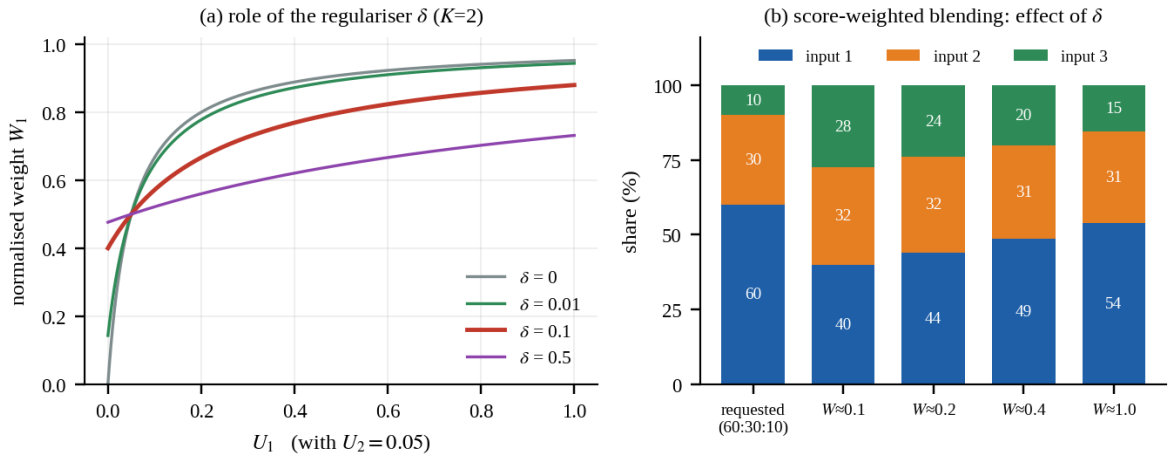


Figure 6: *Role of the constant δ. (a) Normalised weight W_1 of (3.41) versus U_1 for $K = 2$ and $U_2 = 0.05$ for several δ; $\delta = 0.1$ (bold) removes the $0/0$ singularity at $U_1 = 0$ and limits the swing of the weight. (b) Effect of δ on score-weighted blending: requested scores 60:30:10 and the effective shares for aggregated weights U of about 0.1, 0.2, 0.4 and 1.0 in all three inputs.*

Pyramids and level sizes: All resampling uses MATLAB `imresize(...,'bilinear')`, written $\text{rsz}_{\mathbf{r}}\{\cdot\}$ for the target size $\mathbf{r} = [\text{rows,columns}]$. The level sizes halve until a minimum of 4×4:

$$\mathbf{r}_1 = [N_{\text{row}}, N_{\text{col}}], \qquad \mathbf{r}_l = \max\left\{\left\lfloor\frac{\mathbf{r}_{l-1}}{2}\right\rfloor, [4,4]\right\}, \quad l = 2, \ldots, N_{\text{lev}} \tag{3.42}$$

The number of levels follows from the shorter image side N_{side}:

$$N_{\text{req}} = \min\left\{8, \max\left\{1, \left\lfloor \log_2 \frac{N_{\text{side}}}{16} \right\rfloor + 1\right\}\right\},$$
$$N_{\text{cap}} = \max\{1, \lfloor \log_2 N_{\text{side}} \rfloor - 1\}, \tag{3.43}$$
$$N_{\text{lev}} = \max\left\{1, \min\{N_{\text{req}}, N_{\text{cap}}\}\right\}$$

so that the coarsest level keeps a shorter side between 16 and 32 pixels (6 levels for 683×1024, 768×1024 and 576×1024, 5 levels for 480×640; Figure 7). Ancuti et al. choose the number of levels so that the coarsest image has a few tens of pixels (for example seven levels for a 600×800 image); our algorithm's rule gives six levels for that size. The Gaussian and Laplacian pyramids of an image or weight map X are

$$G_1\{X\} = X, \qquad G_l\{X\} = \text{rsz}_{\mathbf{r}_l}\{G_{l-1}\{X\}\} \tag{3.44}$$

$$L_l\{X\} = G_l\{X\} - \text{rsz}_{\mathbf{r}_l}\{G_{l+1}\{X\}\}\ (l < N_{\text{lev}}), \qquad L_{N_{\text{lev}}}\{X\} = G_{N_{\text{lev}}}\{X\} \tag{3.45}$$

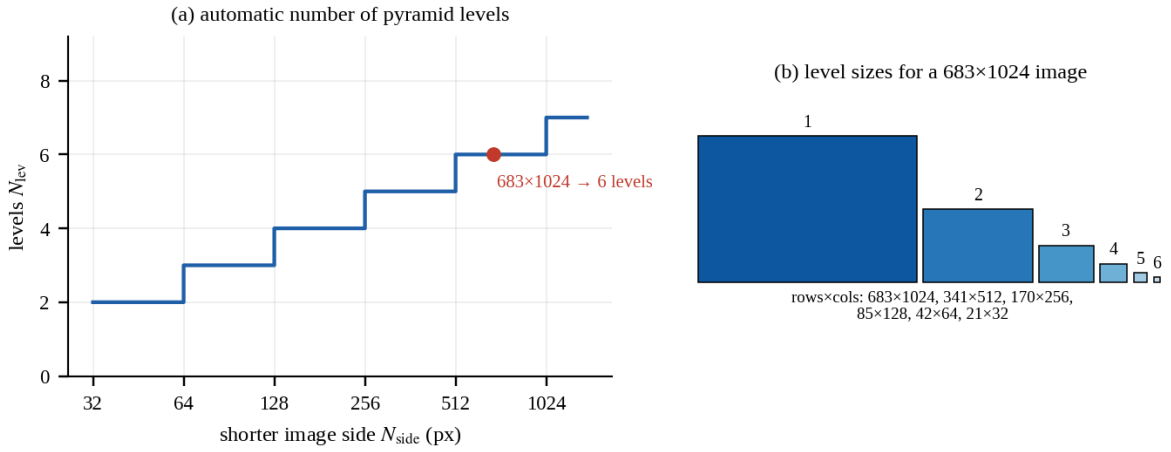


Figure 7: *Pyramid geometry. (a) Number of levels N_{lev} given by (3.43) against the shorter image side; the marker shows a 683×1024 image. (b) The six level sizes for that image from (3.42).*

Because `imresize` applies anti-aliasing when it reduces an image, each reduction is preceded by a low-pass operation, so the pyramid approximates the classical Gaussian pyramid of Burt and Adelson [10], whose reduction uses a 5-tap binomial filter; the equivalence is approximate and not exact.

Fusion and collapse: The fused pyramid combines the bands of the inputs with the Gaussian bands of the weights (the same weight at the three colour channels):

$$P_l(x,y) = \sum_{k=1}^{K} G_l\{W_k\}(x,y)\ L_l\{F_k\}(x,y), \qquad l = 1, \ldots, N_{\text{lev}} \tag{3.46}$$

and the output is rebuilt from the coarsest level upwards:

$$\hat{P}_{N_{\text{lev}}} = P_{N_{\text{lev}}}, \qquad \hat{P}_l = \text{rsz}_{\mathbf{r}_l}\{\hat{P}_{l+1}\} + P_l, \qquad J^{\text{fus}} = \Pi\{\hat{P}_1\} \tag{3.47}$$

Perfect reconstruction: Collapsing the Laplacian pyramid of an image returns the image exactly, because the same resampling operator is used in (3.45) and (3.47). By induction, if $P_l = L_l\{X\}$ then $\hat{P}_{N_{\text{lev}}} = G_{N_{\text{lev}}}\{X\}$ and $\hat{P}_l = \text{rsz}\{G_{l+1}\{X\}\} + G_l\{X\} - \text{rsz}\{G_{l+1}\{X\}\} = G_l\{X\}$, hence $\hat{P}_1 = X$. With

normalized weights, the fusion of K identical inputs therefore returns that input, which is a useful sanity check of the implementation.

Table *21*: *Mapping of Phase 5 equations to `cand_fusion.m` and `ancuti_fusion.m`*

MATLAB statement	Equation	Remark
`input1 = base .^ (1/cfg.gamma)`; clip	(3.35)	$\gamma = 1.5$, exponent 0.667
`imsharpen(base(:,:,c),'Radius',2,'Amount',1.0)` per channel	(3.35)	$\sigma_{\mathrm{sh}} = 2, \zeta = 1$
`lapKernel = fspecial('laplacian',cfg.laplacianAlpha)`	(3.36)	$\eta = 0.2$
`WL = abs(imfilter(gray,lapKernel,'replicate','same'))`	(3.37)	Luma from `rgb2gray`
`WSat = std(candidate,0,3)`	(3.38)	$N - 1$ normalisation
`WS = abs(gray - imgaussfilt(gray,cfg.saliencySigma))`	(3.39)	$\sigma_{\mathrm{sal}} = \max\{4, \mathrm{round}(N_{\mathrm{side}}/16)\}$
`W = WL + WSat + WS`	(3.40)	Aggregated weight
`Wa = (WA_agg+delta)./(WA_agg+WB_agg+2*delta)`	(3.41)	$\delta = 0.1, K = 2$
`computeLevelSizes(H,W,cfg.numLevels)`	(3.42), (3.43)	Automatic levels
`genGaussianPyramid`, `genLaplacianPyramid`	(3.44), (3.45)	`imresize(...,'bilinear')`
`Fused{l} = Ga3.*La{l} + Gb3.*Lb{l}`	(3.46)	`repmat` over the colour channels
`collapsePyramid(Fused,sizes)`	(3.47)	Up-sample and add

Differences from the original paper:

(1) Saturation: standard deviation of the channels instead of the deviation from the luminance.

(2) Saliency: luma centre–surround instead of the Lab frequency-tuned saliency.

(3) Input 2: classical unsharp masking instead of normalized unsharp masking.

(4) Input 1: exponent $1/\gamma$ with $\gamma = 1.5$, whose direction should be verified.

(5) Pyramids: bilinear resampling instead of a 5-tap binomial filter.

(6) Number of levels: derived from the image size by (3.43).

(7) White balance in front of the fusion: Minkowski $p = 6$ instead of Gray-World.

3.2.8 Candidate Selection and Score-Weighted Blending

Selection interface (`run_one.m`): After the six candidates have been written to `candidates/*.png` and shown as a numbered preview, the operator types a selection string. The parser accepts a single index (`4`), a list with scores (`4:60,5:30,6:10` or `(4,60),(5,30)`), or a bare list of indices (equal scores). Indices outside 1 ... 6 are dropped, duplicates are removed, and non-positive scores are replaced by 1. A single index selects that candidate unchanged, $J^{*} = J^{(m)}$. Several indices produce $K = |\mathcal{K}|$ images $F_k = J^{(m_k)}$ with scores s_k (normalised to sum 1 in `run_one.m`) that are passed to `blend_score_weighted.m` with the option `weight_bias = 1`, i.e. $\xi = 1$. Typing `quit` aborts.

Scores to mixing weights: The blending function first converts the scores into mixing weights in four steps:

$$\bar{s}_k = \begin{cases} \dfrac{\max\{s_k, 0\}}{\sum_{k'} \max\{s_{k'}, 0\}}, & \sum_{k'} \max\{s_{k'}, 0\} \geq \epsilon_m \\ \dfrac{1}{K}, & \text{otherwise} \end{cases}$$
$$\pi'_k = \bar{s}_k^{\xi} \qquad (3.48)$$
$$\pi''_k = \max\{\pi'_k, \pi_{\min}\}$$
$$\pi_k = \frac{\pi''_k}{\sum_{k'} \pi''_{k'}}$$

with the exponent ξ (`weight_bias`; the function default is 1.5, `run_one.m` passes 1) and the floor $\pi_{\min} = 0.05$ (`weight_min`) that guarantees every selected candidate a minimum share. Table 22 gives the mixing weights for the example 60:30:10.

Table *22*: *Mixing weights π_k for the example scores 60:30:10*

Setting	π_1	π_2	π_3
$\xi = 1$ (`run_one.m`)	0.600	0.300	0.100
$\xi = 1.5$ (function default)	0.684	0.242	0.074

Weighted fusion: The aggregated weights are those of Phase 5, (3.40), scaled by the mixing weight before the regularized normalization:

$$W_k(x, y) = \frac{\pi_k \, U_k(x, y) + \delta}{\sum_{k'=1}^{K} (\pi_{k'} \, U_{k'}(x, y) + \delta)}, \qquad \delta = 0.10 \qquad (3.49)$$

and the inputs are fused with (3.46) and (3.47) for K inputs. All images are first clamped to $[0,1]$ and resized bilinearly to the size of the first image if the sizes differ; for $K = 1$ the function returns the input. The effect is that an input contributes where *both* the operator trusts it (π_k large) *and* it is locally strong (U_k large).

An important property of (3.49): Since δ is added *after* the scaling by π_k, the regularisation dilutes the operator's scores. For equal aggregated weights U in all inputs, the effective share of input k is $(\pi_k U + \delta)/(U + K\delta)$. With $\pi = (0.6, 0.3, 0.1)$ and $U = 0.2$ the shares are 44 %, 32 %, 24 %; for $U = 0.1$ they are 40, 32, 28 %; for $U = 1.0$ they are 54, 31, 15 % (Figure 6b). The scores typed by the operator are therefore *preferences* that tilt the fusion, not exact mixing ratios, and the tilt grows with the strength of the local features.

3.3 Simulation Model (System Architecture)

The project is a software system, so its "simulation model" is the MATLAB program itself, executed on photographs. Figure 8 shows the architecture. A photograph enters `run_one.m`, which converts it to double precision, removes any alpha plane, clamps it to $[0,1]$ and shrinks it, if necessary, so that its longer side does not exceed $D_{\max} = 1024$ px. The registry in `enhance_all.m` then produces six candidates independently from the same pre-processed image (Phase 1 is also used *inside* Phases 2 and 5). The candidates are written to disk, assembled into a numbered preview, and presented to the operator, who selects one candidate or a weighted subset. A single index returns that candidate; several

indices are fused by `blend_score_weighted.m`. The result and a montage of all candidates with the chosen result are written next to the candidates.

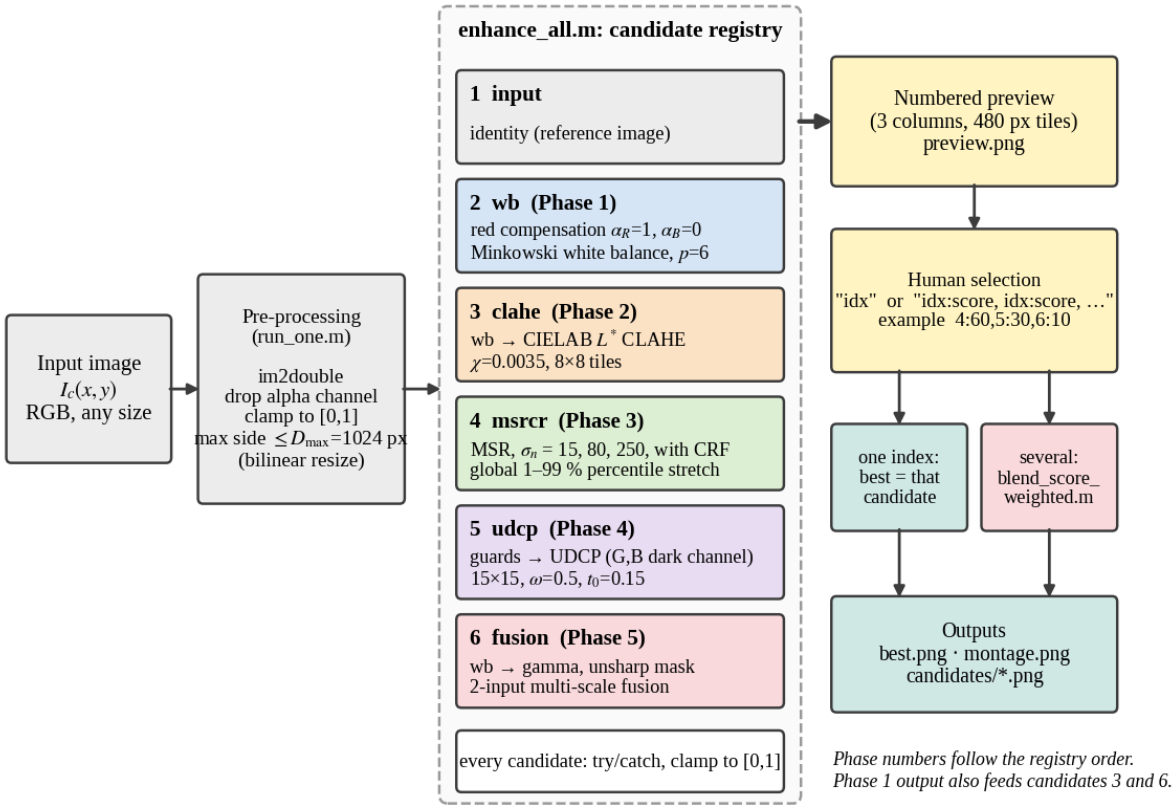


Figure *8*: *System architecture of the enhancement framework. The registry of `enhance_all.m` generates the six candidates; the operator inspects a numbered preview and selects one candidate or a score-weighted blend.*

Module map. Table 24 lists the thirteen source files, their role and their dependencies. The call syntax of the code (`uw.wb_underwater`, `uw.cand_clahe`, `uw.blend_score_weighted`, …) shows that the pipeline functions are organised in a MATLAB package named `uw`, whereas the four core algorithms (`clahe_contrast`, `msrcr_retinex`, `udcp_dehaze`, `ancuti_fusion`) are called without prefix and therefore sit in the `methods` folder that `run_one.m` adds to the path.

Table *23*: *Module map of the MATLAB source files*

File	Level	Role	Calls / uses
`run_one.m`	Driver	Reads and pre-processes the image, runs the registry, saves candidates and preview, parses the operator's choice, blends, saves `best.png` and `montage.png`	`uw.enhance_all`, `uw.blend_score_weighted`, `uw.make_montage`
`enhance_all.m`	Registry	Builds the candidate list `input, wb, clahe, msrcr, udcp, fusion`; each in `try/catch`; supports `opts.exclude`	`uw.wb_underwater`, `uw.cand_*`
`cand_clahe.m`	Wrapper	Phase 2: white balance, then CLAHE with $\chi =$ 0.0035, [8 8] tiles	`uw.wb_underwater`, `clahe_contrast`
`cand_msrcr.m`	Wrapper	Phase 3: sets σ_n, $\mu = 100$, $\nu = 40$, $q_{\text{clip}} = 1$	`msrcr_retinex`
`cand_udcp.m`	Wrapper	Phase 4: constants, guard S1, overshoot blend S2	`udcp_dehaze`
`cand_fusion.m`	Wrapper	Phase 5: white balance, gamma and sharpened inputs, fusion (single- or two-image call)	`uw.wb_underwater`, `ancuti_fusion`
`wb_underwater.m`	Core (Phase 1)	Red/blue compensation and Minkowski white balance	—
`clahe_contrast.m`	Core (Phase 2)	CIELAB conversion and CLAHE of L^*	`rgb2lab`, `adapthisteq`, `lab2rgb`
`msrcr_retinex.m`	Core (Phase 3)	Multi-scale retinex, colour restoration, percentile stretch	`imgaussfilt`

File	Level	Role	Calls / uses
udcp_dehaze.m	Core (Phase 4)	Dark channel, background light, transmission, guided filter, restoration	ordfilt2, imguidedfilter
ancuti_fusion.m	Core (Phase 5)	Weights, pyramids, fusion and collapse for two inputs	fspecial, imfilter, imgaussfilt, imresize
blend_score_weighted.m	Blending	Score-weighted K-input version of the same fusion	imresize, imfilter, imgaussfilt
make_montage.m	Output	Montage of all candidates and the chosen image with caption bars	insertText (optional)

Output structure:

For an input file <stem>.<ext>, the driver writes the following tree under opts.out_dir (default ../results): <stem>/candidates/{input,wb,clahe,msrcr,udcp,fusion}.png, <stem>/preview.png, <stem>/best.png and <stem>/montage.png.

4 Implementation

4.1 Description

Software environment

The implementation is written in MATLAB and uses functions of the Image Processing Toolbox: `im2double`, `imresize`, `imfilter`, `imgaussfilt`, `imguidedfilter`, `adapthisteq`, `rgb2lab`, `lab2rgb`, `rgb2gray`, `fspecial`, `imsharpen` and `ordfilt2`. The caption bars of the preview use `insertText` (Computer Vision Toolbox) and the preview is opened with `winopen` (Windows only); both calls sit inside `try/catch`, so on a system without them the captions or the automatic opening are simply omitted. No random numbers are used, so a run is deterministic for a given input, parameter set and MATLAB release

Execution flow

The program flow of `run_one.m` is as follows.

1. Set default options (`out_dir`, `max_dim` $= 1024$, `exclude`, `open_preview`, `verbose`, `preset`) and add the parent folders and the `methods` folder to the path.
2. Read the image, convert it with `im2double`, drop a fourth (alpha) plane, and reduce it with bilinear resizing if its longer side exceeds $D_{\max}$.
3. Call `enhance_all` to create the candidate structure array `cands(m).name` and `cands(m).image`; write each candidate to `candidates/<name>.png`.
4. Build the numbered preview (3 columns, 480-px tiles, 28-px caption bars) and save it as `preview.png`; open it if `open_preview` is true.
5. Print the list of candidates and read the choice: either the `preset` string (for batch use) or typed input. An empty string or `quit` aborts and returns an empty pick.
6. Parse the choice into indices and scores, filter invalid entries, and either select the single candidate or normalise the scores and call `blend_score_weighted` with $\xi = 1$.
7. Save `best.png` and a montage of all candidates plus the chosen image (`montage.png`), print the decision and return the structure `pick` with the name, the image and the file paths.

The registry in `enhance_all.m` is a cell array of function handles. The list of names is reduced by `setdiff(...,'stable')` when the caller excludes candidates, every handle is executed in `try/catch`, the result is converted with `im2double`, a fourth plane is dropped and the image is clamped to $[0,1]$. A failing candidate raises the warning `uw:enhance_all:cand_failed` and is skipped, so that the other candidates remain available.

Implementation notes per phase

- **Phase 1** works on the whole image with vectorized arithmetic. The Minkowski mean uses `mean(ch(:).^p)^(1/p)` for each channel; the sixth power of values in $[0,1]$ is numerically harmless in double precision.
- **Phase 2** converts to CIELAB with `rgb2lab`, divides the lightness by 100 so that `adapthisteq` receives a double image in $[0,1]$, equalizes, multiplies by 100 and converts back. The input must be a double array in $[0,1]$ (`validateattributes`).
- **Phase 3** filters the three channels together, because `imgaussfilt` treats the third dimension as independent planes; the colour-restoration function uses implicit expansion between the 3-D

array and the 2-D channel sum. The percentile is computed by sorting all $3|\Omega|$ values (`simplePercentile`).

- **Phase 4** extracts the dark channel with an order-statistic filter of rank 1 on the $P \times P$ neighbourhood, selects the brightest dark-channel pixels by sorting, and reconstructs each channel with the same refined transmission. The wrapper measures the channel means before and after.
- **Phase 5** builds the pyramids as cell arrays of matrices (one cell per level). The weights are replicated over the three colour planes with `repmat`.

Numerical considerations

All arithmetic is in double precision; only the saved PNG files are quantized to 8 bits (a step of $1/255$). Guards prevent invalid operations: ϵ_{m} for divisions in Phase 1, $\epsilon_{\log}$ for logarithms, $A_{\min}$ for divisions by the background light, t_0 for the restoration gain, δ for the weight normalisation and the floor $\pi_{\min}$ for the mixing weights. The clamp $\Pi\{\cdot\}$ at the end of every stage keeps rounding effects from propagating.

Complexity

Table 25 lists the dominant operations. The estimates assume direct separable convolution and are intended to show *which* phase dominates, not to predict absolute time; MATLAB may use faster algorithms internally.

Table *24*: *Dominant operations per phase for $|\Omega|$ pixels*

Phase	Dominant operation	Order of cost
1	Channel means, sixth power, gain	$O(\|\Omega\|)$
2	CIELAB conversion, 64 tile histograms, bilinear interpolation	$O(\|\Omega\|) + O(g_{\mathrm{r}} g_{\mathrm{c}} N_{\mathrm{bin}})$
3	Three Gaussian filters with 61, 321 and 1001 taps per pass, on three channels; sort of $3\|\Omega\|$ values	$O(\|\Omega\| \sum_n (2\lceil 2\sigma_n \rceil + 1)) + O(\|\Omega\| \log \|\Omega\|)$; the $\sigma_3 = 250$ filter dominates
4	Two 15×15 rank filters, sort of $\|\Omega\|$ values, guided filter (box filters)	$O\left(\|\Omega\| s_{\mathrm{p}}^2\right) + O(\|\Omega\| \log \|\Omega\|)$
5	3×3 filter, Gaussian of σ_{sal}, pyramids of K inputs and weights	$O(K\|\Omega\|)$ with a small constant

For a 683×1024 image ($|\Omega| = 699\,392$), the three retinex filters need about $2 \times (61 + 321 + 1001) \approx 2800$ multiply-adds per pixel and channel if they are evaluated by direct separable convolution, i.e. a few 10^9 operations per image, which suggests that Phase 3 is the slowest candidate. Measured run times are to be recorded with `tic/toc`.

Verification by invariants

Several properties follow from the equations and give implementation checks that need no reference images:

- **Partition of unity:** `max(abs(Wa(:)+Wb(:)-1))` after `[~,Wa,Wb] = ancuti_fusion(A,B)` must be of the order of 10^{-16} ((3.41)).
- **Perfect reconstruction:** `max(abs(ancuti_fusion(A,A)-A),[],'all')` must be zero within rounding, because the fusion of identical inputs returns the input ((3.47)).
- **Non-negative boost:** the gains of Phase 1 satisfy `gains >= 1` and the output of every phase lies in $[0,1]$.
- **Guard behavior:** an input with $\overline{I}_G > 1.25\,\overline{I}_B$ and $\overline{I}_G > 1.25\,\overline{I}_R$ must be returned unchanged by `cand_udcp`.

- **Single choice:** typing a single index must return exactly that candidate, and `blend_score_weighted` with one image must return it unchanged.

Operating procedure

A complete description for users is given in Section 7.2. In short, the user calls `pick = uw.run_one('photo.jpg')` (or with an `opts` structure), inspects `preview.png`, and types, for example, `4:60,5:30,6:10`.

5 Design Analysis and Evaluation

5.1 Novelty

The individual algorithms are published methods [1]–[4], so the novelty of the project lies in the engineering of the system and in the analysis, not in a new image-enhancement principle. Five contributions are claimed, each stated with the limits of the claim.

1. **A candidate-and-select architecture with bounded failure.** Five heterogeneous algorithms (colour correction, histogram equalization, retinex, dehazing, fusion) are wrapped behind one registry with a common interface (double image in $[0,1]$ in, double image in $[0,1]$ out), a clamp after every stage and a `try/catch` per candidate. A failure or a bad result of one candidate cannot prevent the operator from using the others.
2. **A K-input, score-weighted extension of the fusion of Ancuti et al.** The two-input fusion of [1] is generalized to an arbitrary number of whole candidates and driven by operator scores through the normalization of (3.49). This allows, for example, 60 % retinex, 30 % dehazing and 10 % fusion to be combined while still selecting the locally best detail. The analysis of Section 3.2.8 shows that the regularization dilutes the scores, which is a property that users of the tool should know.
3. **Safeguards for the underwater dark channel prior.** The green-dominance bypass (S1) and the continuous overshoot blend (S2) of (3.33) detect the two most common failures of UDCP from simple channel statistics and respond in a bounded, continuous manner. They are not part of [4] or [6].
4. **Separation of colour and tone.** White balance is performed in RGB (Phase 1), equalization in the lightness channel only (Phase 2), and both are available as independent candidates, so colour is never corrected twice.
5. **Transparency and analysis.** Every constant is traced to its origin, the code is mapped to the equations, the identifier collisions are resolved by a unified notation, and several non-obvious properties are derived: the gain ν is irrelevant after the global stretch, the coefficient ω (not t_0) limits the restoration gain, the regularisation δ dilutes the operator's scores, and CLAHE leaves illumination in place while amplifying fine-scale contrast by its gain ceiling.

The system is not claimed to outperform the state of the art on a benchmark; no benchmark comparison was performed.

5.2 Design Considerations (PO(c))

5.2.1 Considerations to public health and safety

The software has no direct hazard, but its output may inform decisions. Enhancement can *create or hide* structure: sharpening and CLAHE amplify noise into apparent texture, dehazing can remove real turbidity information, and retinex can introduce halos. Images that support inspection of piers, hulls, dams or diver safety must therefore be judged on the original, with the enhanced image as an aid only. The design mitigations are: the unprocessed input is always candidate 1 in the preview; the montage shows every candidate next to the chosen result; and the output names record the processing path (`blend:msrcr(60%)+udcp(30%)+…`). The tool is offline and is not intended for real-time navigation or diver communication.

5.2.2 Considerations to environment

The tool reduces the need to repeat dives or surveys because marginal images become usable, and it needs no additional hardware: only a laptop and MATLAB. It does not interact with the aquatic environment. Collecting the images, however, should avoid disturbance of marine life (flash and torch use near sensitive species, contact with coral).

5.2.3 Considerations to cultural and societal needs

Many water bodies in Bangladesh (rivers, estuaries, ponds and the near-shore Bay of Bengal) are turbid, so methods that assume clear blue water perform poorly; the safeguards for green and turbid scenes address this. The system is training-free, runs on an ordinary laptop and does not require a GPU or a data connection, which suits laboratories and small organizations with limited resources. A MATLAB licence is a cost barrier outside universities; Section 9 proposes a port to an open-source environment. Photographs of people (divers, fishers, local communities) require consent and respect for local norms.

5.3 Investigations (PO(d))

5.3.1 Design of Experiment

Three investigations were designed. They address, respectively, what the constants *do* (E1), whether CLAHE can be trusted on non-uniformly lit scenes (E2), and how the candidates behave on real photographs (E3).

E1: Analytical sensitivity of the constants. For each constant, the derivation of Section 3.2 gives a closed-form statement of its effect. No images are needed. Table 26 summarizes the findings.

Table *25*: *Analytical findings on the influence of the constants (E1)*

Constant	Finding	Basis
ν	No influence on the output: it cancels in the global percentile stretch	(3.24)
μ	$125 \rightarrow 100$ changes the channel weighting by 6–10 % for channels with 10–33 % chromaticity and by 32 % for a channel with 2 % chromaticity	Table 17
ω	Limits the restoration gain: with $\omega = 0.5$ the gain stays below 2 until $Q^{\mathrm{dark}} > 1.7$	Figure 5
t_0	Caps the gain at $1/t_0 = 6.7$; active only for $Q^{\mathrm{dark}} > 1.7$ when $\omega = 0.5$	(3.31)
χ	Gain ceiling $1 + (N_{\mathrm{bin}} - 1)\chi$: 1.63 (0.0025), 1.89 (0.0035), 3.55 (0.01)	(3.13)
δ	Dilutes scores: a requested 60 % share becomes 40–54 %	(3.49)
p	Moves the illuminant estimate from the mean ($p = 1$) towards the maximum	(3.6)
ϵ_{gf}	Edge threshold at a guide standard deviation of 0.032 (8/255)	(3.30)
N_{lev}	6 levels for 683×1024 and 768×1024, 5 for 480×640	(3.43)

E2: Controlled experiment on non-uniform illumination. The question is whether CLAHE improves, preserves or amplifies a spotlight effect. A synthetic design was chosen because it provides *known* illumination and reflectance, which no photograph can. Four scenes were generated: two illumination fields (a smooth Gaussian spot and a torch-like spot with a nearly flat top, a steep skirt and a back-scatter glow) combined with two reflectance fields (nearly flat, which represents open water, sand or hull plating, and textured, with objects). Each scene was processed by the identity (input), by CLAHE on the CIELAB lightness with $\chi \in \{0.0035, 0.01, 0.03\}$, 8×8 tiles and the output

limited to the original range, and by MSR with $\sigma_n = 15{,}80{,}250$. Three metrics were defined. Because CLAHE and MSR both change the overall tone scale, every metric is normalised by the output's own tonal range, $\mathrm{pct}_{99}(J) - \mathrm{pct}_1(J)$, so that it is invariant to any affine tone change $J \to aJ + b$. With ϱ the distance from the spot centre, σ_{sp} the spot width, $\langle\cdot\rangle_{\mathcal{Z}}$ the mean over a region $\mathcal{Z}$ and pct_q the q-th percentile of all pixel values:

$$\begin{aligned} \mathrm{SC}(J) &= \frac{\langle J\rangle_{\varrho<0.5\sigma_{\mathrm{sp}}} - \langle J\rangle_{\varrho>2.2\sigma_{\mathrm{sp}}}}{\mathrm{pct}_{99}(J) - \mathrm{pct}_1(J)}, \qquad \mathrm{TG}_{\mathcal{Z}}(J) \\ &= \frac{\mathrm{std}_{\mathcal{Z}}\{J - \Gamma_2 * J\}/\big(\mathrm{pct}_{99}(J) - \mathrm{pct}_1(J)\big)}{\mathrm{std}_{\mathcal{Z}}\{I - \Gamma_2 * I\}/\big(\mathrm{pct}_{99}(I) - \mathrm{pct}_1(I)\big)} \end{aligned} \tag{5.1}$$

The spot contrast, SC is the fraction of the displayed tonal range that separates the spot core from the far field (the input has SC close to 1; an ideal illumination correction would make it much smaller). The texture gain, $\mathrm{TG}_{\mathcal{Z}}$ compares the fine-scale contrast (noise and texture, isolated with a $\sigma = 2$ px high-pass) inside a region $\mathcal{Z} \in$ {core,far field} with that of the input, again in units of the tonal range; a value of 1 means unchanged and the theory of Section 3.2.4 predicts $\mathrm{TG} \approx \mathcal{G}_{\max}$ for CLAHE in homogeneous regions. For the flat scenes, whose input profile is monotone, the halo overshoot, $\mathrm{HO}(J) = \max_{\varrho}\big[\bar{q}(\varrho) - \min_{\varrho' \le \varrho}\bar{q}(\varrho')\big]$ measures any ring that a method adds, where $\bar{q}(\varrho)$ is the radial profile of $\Gamma_6 * J$ in units of the output range (0 for a monotone profile).

5.3.2 Data Collection

Synthetic data (E2). Table 27 lists the parameters, which are sufficient to regenerate the scenes. All random numbers use a fixed seed. CLAHE was computed with the scikit-image implementation [24], with the tile size, the clip count h_{clip} of (3.11) (converted to the normalised form that scikit-image expects) and 256 bins, so that the clipping rule is that of MATLAB `adapthisteq`; the output was then mapped linearly onto the original lightness range to imitate `Range='original'`. MSR followed (3.19)–(3.21) with Gaussians truncated at 2σ and the 1 %/99 % global stretch of (3.24). The numerical outputs are close to, but not identical with, those of MATLAB.

Table *26*: *Parameters of the synthetic scenes of E2*

Item	Value
Image size, format	683×1024 pixels, grey values in [0,1], processed as a neutral RGB image through CIELAB
Spot centre	(560, 320) px
Smooth spot	$\Theta = 0.20 + 0.85\exp\big(-\varrho^2/(2 \cdot 170^2)\big)$, $\sigma_{\mathrm{sp}} = 170$ px
Torch spot	$\Theta = 0.10 + 1.7\exp(-(\varrho/150)^6)$ plus back-scatter glow $0.04 + 0.10\exp\big(-\varrho^2/(2 \cdot 225^2)\big)$, $\sigma_{\mathrm{sp}} = 150$ px
Flat reflectance	$\Upsilon = 0.5$ plus fine texture (standard deviation about 0.017)
Textured reflectance	Smooth random field 0.30–0.60, seven objects with $\Upsilon \in [0.20{,}0.85]$, fine texture
Image	$I = \Pi\{\Theta\Upsilon + \mathrm{glow} + n\}$, sensor noise $n \sim \mathcal{N}(0, 0.004^2)$
Methods	Input; CLAHE ($\chi = 0.0035,\ 0.01,\ 0.03$; 8×8 tiles, CIELAB L^*); MSR $(15/80/250)$
Saturated input pixels	1.9 % (torch, flat), 1.4 % (torch, textured), none in the smooth scenes

5.3.3 Results and Analysis

E1. The findings of Table 26 are exact consequences of the equations. The most practically relevant ones are that ν can be set to any positive value without changing the result, that ω (not t_0) is the real limiter of the dehazing gain in the shipped configuration, and that the operator's scores are diluted by δ.

E2. Table 29 gives the spot contrast, Table 30 the texture gain, and Figure 9 shows the torch scene with flat reflectance together with the summary of all scenes.

Table *27*: *Spot contrast SC of the input and of each method; the factor in brackets is SC(output)/SC(input)*

Scene	Input	CLAHE 0.0035	CLAHE 0.01	CLAHE 0.03	MSR
smooth spot, flat	0.927	0.884 (×0.95)	0.797 (×0.86)	0.575 (×0.62)	0.731 (×0.79)
torch spot, flat	0.976	0.962 (×0.98)	0.934 (×0.96)	0.853 (×0.87)	0.661 (×0.68)
smooth spot, textured	0.578	0.550 (×0.95)	0.486 (×0.84)	0.326 (×0.57)	0.310 (×0.54)
torch spot, textured	0.924	0.880 (×0.95)	0.801 (×0.87)	0.657 (×0.71)	0.599 (×0.65)

Table *28*: *Texture gain TG (1.00 = unchanged) in the far field and in the spot core; the predicted gain ceiling of CLAHE is* $\mathcal{G}_{max} = 1 + 255\chi$

Scene	Region	CLAHE 0.0035	CLAHE 0.01	CLAHE 0.03	MSR
Predicted ceiling $\mathcal{G}_{\max}$	—	*1.89*	*3.55*	*8.65*	—
smooth spot, flat	far field	1.96	3.69	8.96	7.09
smooth spot, flat	core	1.75	2.80	4.74	1.35
torch spot, flat	far field	1.96	3.75	9.44	9.29
torch spot, flat	core	1.95	3.44	6.58	1.00
smooth spot, textured	far field	1.87	3.29	6.97	5.36
smooth spot, textured	core	1.56	2.27	2.91	1.03
torch spot, textured	far field	1.93	3.66	9.14	8.88
torch spot, textured	core	1.70	2.71	3.53	0.98

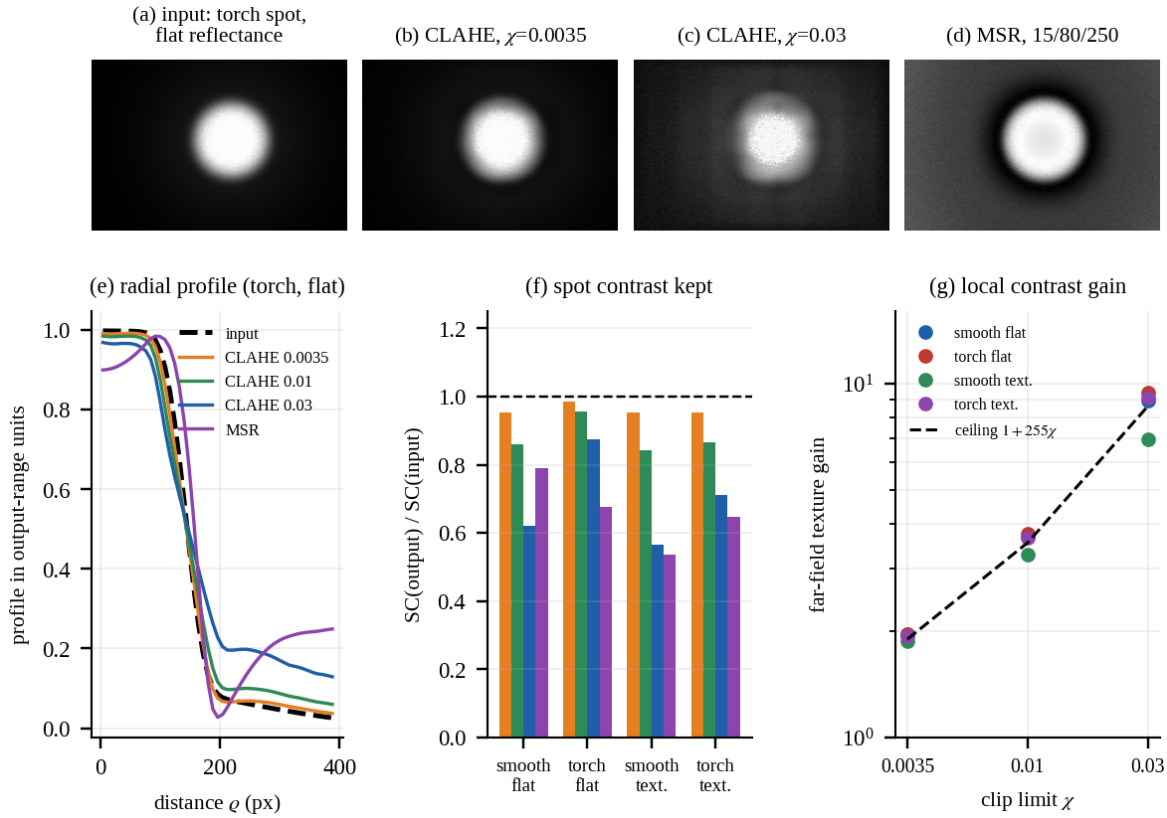


Figure 9: *Controlled synthetic experiment E2 (generated for this report, not photographs). (a)–(d) Torch-like spot on a nearly flat reflectance, shown with the same 1–99 % display stretch: input, CLAHE with* $\chi = 0.0035$ *and* $\chi = 0.03$*, and MSR. (e) Radial brightness profiles in units of the output range. (f) Fraction of the input's spot contrast that each method keeps, for the four scenes. (g) Far-field texture gain of CLAHE for the four scenes against the predicted ceiling* $1 + 255\chi$ *(dashed).*

The halo overshoot of the flat scenes was at most 0.011 of the output range for every CLAHE setting (input: ≤ 0.001), whereas MSR produced 0.221 for the torch spot (a dark ring around the spot, Figure 9d, e) and 0.004 for the smooth spot.

Per phase outputs: Here is a demonstration showing outputs obtained after passing a raw input image through each phase:

Phase	Input	Output
White Balance		

CLAHE		
MSRCR		
UDCP		

5.3.4 Interpretation and Conclusions on Data

1. **At the project's clip limit CLAHE leaves the illumination in place.** For $\chi = 0.0035$ the spot contrast kept 95–98 % of its input value in all four scenes (Table 29), so the large-scale spotlight is neither removed nor noticeably enlarged. A larger χ reduces the spot contrast (to 57–87 % at $\chi = 0.03$) because the dark periphery is brightened, but this is bought with the effects in items 2 and 3.
2. **CLAHE amplifies fine-scale contrast, and noise, by the predicted ceiling.** The far-field texture gain was 1.87–1.96 at $\chi = 0.0035$, 3.29–3.75 at $\chi = 0.01$ and 6.97–9.44 at $\chi = 0.03$, against the predicted ceilings 1.89, 3.55 and 8.65 (Table 30, Figure 9g). The measurement therefore confirms (3.13) almost quantitatively. The amplified quantity is whatever varies inside the tile, noise and fine texture as much as detail: in an illumination-dominated scene the dark periphery becomes grainy.
3. **The spot interior is also affected.** The texture gain in the spot core was 1.6–1.95 at $\chi = 0.0035$ and reached 2.9–6.6 at $\chi = 0.03$, which produces the mottled, noisy glare region visible in

Figure 9c. This local amplification around and inside the light source is the form of "localised halation" that the synthetic experiment reproduces.

4. **The hypothesis of a strong amplification of the global spot was not supported at this clip limit.** The expectation that tiled CLAHE would substantially enlarge the spot-to-background step did not appear in the synthetic data; the effect that did appear is the local one of items 2 and 3, together with the failure to remove the illumination. How strongly this is perceived as a "spotlight effect" on real photographs depends on the clip limit, on saturation, on noise and on the white-balance gains applied before CLAHE.
5. **MSR removed more of the illumination but behaves differently.** It cut the spot contrast to 54–79 % of the input, but it produced a dark halo ring around a bright torch spot (overshoot 0.22), amplified far-field texture 5–9 times, and in the smooth-spot scene, where the spot is as wide as its largest surround, it removed little.
6. **Consequence for the design.** CLAHE is kept as an optional candidate with a conservative limit, the operator can reject it in the preview, and illumination-dominated scenes are better served by the retinex candidate, or by a blend in which retinex dominates and CLAHE is given a small score.

The experiment has limits: the scenes are synthetic, one noise realization was drawn per scene, scikit-image CLAHE is not bit-identical to MATLAB's, the metrics are simple, and no perceptual test was made. The conclusions concern the direction and size of the mechanisms, not the exact appearance of real photographs.

5.4 Limitations of Tools (PO(e))

Each tool is limited by its model assumptions and by the numerical behavior of its MATLAB implementation. The critical observation on CLAHE is given first, because it motivated a design decision (CLAHE is optional); a summary table of the other limits follows.

Critical observation: Phase 2 illumination correction by CLAHE on non-uniformly lit scenes

Phase 2 is the only phase whose name suggests illumination correction, and it is the phase that fails most instructively when the ambient or artificial light is far from uniform, as it is near a torch or a strobe. The failure has six components.

(1) CLAHE has no model of illumination: In the model $I_c = \Theta_c Y_c$ the lightness Λ that CLAHE receives is a monotone function of the product of illumination and reflectance. CLAHE sees only the histogram of Λ in each tile and no spatial structure, so it cannot decide whether an intensity difference is due to light or to the scene. In a tile with little reflectance variation (open water, sand, hull plating, a wall) the width of the histogram is set by the illumination gradient across the tile and by noise, and CLAHE treats that gradient as contrast to be equalized. Global CLAHE (one tile) is worse in kind: a torch-lit frame has a bimodal histogram, a large dark periphery and a small bright spot, and a single mapping cannot serve both.

(2) The local gain equals the gain ceiling, whatever the origin of the contrast: If the values of a tile occupy a band of B bins and $B\,\mathcal{G}_{\max} < N_{\text{bin}}$, the clipped mapping has slope $\mathcal{G}_{\max}$ over the band ((3.13)), so that differences inside the band are enlarged by the same factor,

$$\Delta\Lambda^{\text{eq}} \approx \mathcal{G}_{\max}\,\Delta\Lambda,$$
$$\mathcal{G}_{\max} \approx 1 + (N_{\text{bin}} - 1)\chi = 1.89,\ 3.55,\ 8.65 \qquad \text{for } \chi = 0.0035,\ 0.01,\ 0.03 \tag{5.2}$$

The enlarged differences include illumination gradients, fine texture and sensor noise. The synthetic experiment confirms the magnitude: the fine-scale contrast of the dark periphery rose by 1.87–1.96, 3.29–3.75 and 6.97–9.44 times at the three limits (Table 30). In a torch-lit scene the periphery has the narrowest histograms and therefore receives the full gain: the periphery becomes grainy, and the visibility of the noise is the opposite of the intended *uniform* visibility.

(3) Tiles are mapped independently, and the mapped bands are redistributed: Consider two idealized narrow-band tiles at the ceiling $G_{\max} = 1.89$: a dark band $[0.08,0.33]$ of the lightness range and a bright band $[0.70,0.95]$. Conservation of the pixel count in the clipped CDF maps them to about $[0.055,0.53]$ and $[0.49,0.96]$. Each band is stretched by 1.89, the dark band is pulled up and the bright band is pulled down, so the contrast inside each tile grows while the tone relationship between neighboring tiles is distorted. Bilinear interpolation blends the mappings only over half a tile (about 43–64 px for a 683×1024 image with 8×8 tiles, Table 15), so the inconsistency appears as blotchy, tile-scale brightness variations around the light source. This is most visible at larger clip limits (Figure 9c). At the project's limit the large-scale spot contrast stayed at 95–98 % of its input value, i.e. the illumination was not corrected either.

(4) Saturation and the gains of Phase 1: The gains $\kappa_c \geq 1$ of Phase 1 brighten the image, and hot spots can reach the clipping value 1. A clipped plateau carries no recoverable detail; CLAHE can move it relative to its surroundings, but it cannot restore it. In the synthetic torch scenes 1.4–1.9 % of the input pixels were saturated.

(5) Colour side effects: Equalizing L^* alone preserves hue, but the unchanged a^*, b^* combined with a strongly changed L^* can leave the sRGB gamut; `lab2rgb` then clips the channels, which shifts saturation near very bright or very dark pixels, for example in a blue-green glow around the lamp.

(6) Consequences for use:

(a) CLAHE must not be used as an illumination corrector; it equalizes tonal distributions.

(b) If it is used, the clip limit should stay small (the ceiling at $\chi = 0.0035$ is 1.9), the tile should be larger than the spot, and the image should be denoised first.

(c) The large-scale illumination should be removed before equalization, for example by the retinex candidate or by dividing by a heavily smoothed luminance.

(d) In this framework CLAHE is one candidate among six, the preview shows it beside the others, and the operator can reject it or give it a small score in a blend.

The synthetic experiment did not show a strong amplification of the global spot-to-background step at the project's clip limit; it showed failure to correct the illumination, a local amplification of contrast and noise equal to the gain ceiling, and tile-scale mottling at larger limits. This is the evidence-based statement of the limitation.

Other limitations of the tools:

Table *29*: *Limitations of the tools, their size and the mitigation in the design*

Tool	Limitation	Bound or consequence	Mitigation
Phase 1: Minkowski white balance	Single global illuminant assumed; fails with mixed torch and ambient light. Gains $\kappa_c \geq 1$ brighten the image and can clip highlights	Clipped pixels carry no information	Other candidates; operator choice
Phase 1: red compensation	Assumes green is well preserved; weak in very turbid green-brown water	α_R is global	Candidate `udcp` bypass for green water; operator choice

Tool	Limitation	Bound or consequence	Mitigation
Phase 3: MSRCR	Halo ring near strong light sources; noise amplification in dark regions; CRF sign inversion if $r_c < 1/\mu$; scales in pixels tied to the resolution	Halo overshoot 0.22 and far-field texture gain 5–9 in the torch test	$D_{\max} = 1024$; operator choice; blending
Phase 3: display stretch	Data-dependent global 1 %/99 % clipping saturates 2 % of the values and makes the brightness of candidates image-dependent	2 % of values clipped	Visible in the preview
Phase 4: UDCP	Bright objects and white sand violate the prior and mislead A_c; one t for all channels; patch-minimum blocks	Gain up to 2 ($\omega = 0.5$), cap 6.7	Safeguards S1 and S2; t_0
Phase 4: safeguards	S1 and S2 are heuristics: a truly green scene triggers S1, and a legitimate large correction can trigger S2	$\iota_g = 1.25$; $\tau_{os} = 0.18$	Operator can blend UDCP back in
Phase 5: fusion	Both inputs come from the same white-balanced image, so its errors propagate; fusion creates no new information; gamma direction	—	Verify on the image set
Blending	δ dilutes the scores; scores are subjective	60 % request becomes 40–54 %	Documented in 3.2.8
`adapthisteq`	256 bins; absolute clip count derived from χ by (3.11); no illumination model	Gain ceiling $1 + 255\chi$	Conservative χ; optional candidate
`imresize` pyramids	Bilinear with anti-aliasing approximates but is not the 5-tap Gaussian pyramid	Slight difference in band content	Perfect reconstruction is preserved
`imgaussfilt`	Kernel truncated at $\pm 2\sigma$; replicated borders	1001 taps for $\sigma = 250$	—
`imguidedfilter`	Single-channel guide (luma), so iso-luminant colour edges are not followed	—	Guide is `rgb2gray`
`imsharpen`	Channel-wise sharpening can create colour fringes	Amount $\zeta = 1$	Weights decide where it is used
Output	8-bit PNG quantisation; `winopen` is Windows-only; `insertText` needs the Computer Vision Toolbox	Step $1/255 \approx 0.0039$	`try/catch` around both
Licence	MATLAB is proprietary	Cost outside universities	Port proposed (Chapter 9)

Numerical tolerances and operating limits. All arithmetic is in double precision (relative accuracy about 2.2×10^{-16}). The restoration gain of Phase 4 cannot exceed $1/t_0 = 6.67$ and with $\omega = 0.5$ stays below 2 for $Q^{\text{dark}} < 1.7$; the contrast gain of Phase 2 cannot exceed $\mathcal{G}_{\max} = 1.89$; the fusion weights are bounded below by $\delta/(\sum_{k'} U_{k'} + K\delta)$; the number of pyramid levels is at most 8 and a level is never smaller than 4×4. A 683×1024 double RGB array occupies $683 \times 1024 \times 3 \times 8$ B $= 16.8$ MB, so the six candidates and the pyramids of a blend need several tens to a few hundred MB. Accuracy cannot be stated in absolute terms because no ground-truth images are available; quality was judged visually by the operator.

5.5 Impact Assessment (PO(f))

5.5.1 Assessment of Societal and Cultural Issues

The tool can help divers, fisheries scientists, students and inspectors, particularly where only modest cameras and laptops are available, and it contributes to public awareness of reef and river ecosystems because clearer images communicate better. Cultural sensitivities arise when images show people, fishing communities or religious and heritage sites under water; consent and local norms must be respected. Enhanced images must not be presented as unprocessed documentation, because that would mislead the public.

5.5.2 Assessment of Health and Safety Issues

There is no direct health or safety hazard in the software. The indirect risk is a wrong decision based on an artefact, for example mistaking amplified noise for corrosion or a halo for a defect. The design keeps the original as candidate 1, records the processing path in the name of the result, and recommends that inspections are decided on the raw data. Divers must not rely on an enhanced still image for navigation or decompression decisions.

5.5.3 Assessment of Legal Issues

MATLAB and its toolboxes are used under an institutional licence; commercial use needs a separate licence. The five papers and any test photographs are protected by copyright: the papers are cited and not reproduced, and each test image must be used under its licence or with the photographer's permission. Photography in protected marine and forest areas, and the use of drones or diving equipment, may require permits from the competent authorities. Privacy and personal-data rules apply to identifiable persons in the images.

5.6 Sustainability Evaluation (PO(g))

Environmental: The framework needs no new hardware, runs on existing computers and is training-free, so it avoids the energy and embodied-carbon cost of GPU training and of dedicated devices.

Better images reduce repeated dives and re-surveys, which saves boat fuel and limits disturbance of the habitat; the tool supports monitoring of coral, seagrass and fish habitats, which relates to UN Sustainable Development Goal 14 (life below water).

Economic: The marginal cost of an additional image is practically the electricity of a few seconds of computing. The main recurring cost is the MATLAB licence outside the university, which a port to an open-source environment would remove.

Social: The tool lowers the skill and equipment barrier for underwater documentation and is suitable for teaching, in line with SDG 4 (quality education). The documentation (notation, constant register, user manual) allows other students to maintain it.

Technical (maintainability): The code is modular (one function per phase and a registry), has no random elements, and every constant is recorded, so changes can be made safely. The dependence on one commercial environment and on MATLAB toolbox behavior is the principal sustainability risk, and the documentation of the equations makes a re-implementation possible.

5.7 Ethical Issues (PO(h))

The central ethical issue is honesty about image processing. Enhancement changes the evidence, so-

(i) The original must be preserved and shown,

(ii) Enhanced images must be labelled as such in reports and publications, and

(iii) The tool must not be used to hide, create or exaggerate features, for example to disguise pollution or illegal fishing or to fabricate damage.

Further issues are the academic integrity of this report (all theory is derived from cited papers; the synthetic data are generated by the team and labelled as synthetic; the quotations are limited), the credit due to the authors of the algorithms, datasets and toolboxes, the responsible handling of images of people and communities, and the environmental ethics of collecting images without disturbing marine life. The team applied these principles by citing all sources, reporting the limitations of each method, and presenting the experiment that contradicted an initial expectation instead of omitting it.

6 Reflection on Individual and Team work (PO(i))

6.1 Individual Contribution of Each Member

The software has a modular structure (one core function and one wrapper per phase, a registry, a blending engine and a driver), which allows work to be divided by module. Table 32 lists the work packages; each team member marks the packages that he carried out.

Table *30*: *Work packages and responsibilities (tick the packages carried out)*

Work package	2206037	2206042	2206043	2206049
Literature study of the five papers and notation list	✓	✓	✓	✓
Driver, registry, preview and montage (`run_one`, `enhance_all`, `make_montage`)		✓		
Phase 1: colour compensation and white balance	✓			
Phase 2: CLAHE in CIELAB	✓	✓	✓	✓
Phase 3: MSRCR		✓	✓	
Phase 4: UDCP and safeguards	✓	✓		✓
Phase 5: fusion and score-weighted blending			✓	
Testing, tuning of the constants, image collection			✓	✓
Report, notation synchronisation, user manual and video	✓	✓		✓

6.2 Mode of Team Work

The project was organized around a modular software architecture and a strict functional interface, requiring systematic technical synchronization across all candidate restoration stages. Because every candidate stage took a double-precision RGB image bounded in $[0,1]$ and returned an output in the same domain, team members could independently develop, optimize, and unit-test their respective algorithms without creating blocking dependencies. To maintain structural consistency, the team established a single, unified mathematical notation and a master parameter register early in the development cycle. Every algorithm modification—such as setting the conservative clip limit in CLAHE ($\chi = 0.0035$), configuring the multi-scale retinex Gaussian standard deviations ($\sigma_n \in \{15,80,250\}$ px), or tuning the transmission floor ($t_0 = 0.15$) and haze coefficient ($\omega = 0.5$)—was cross-verified analytically and against invariant properties (such as partition of unity and perfect reconstruction in pyramid collapse) before being committed to the shared codebase.

Responsibilities were allocated by functional processing phase rather than by isolated sections of the report. The members who derived and implemented a core module also designed its corresponding defensive wrapper, benchmarked its run-time complexity, and defended its behavior during team evaluations. The team conducted weekly technical reviews where members projected candidate outputs side-by-side using the automated montage and preview pipeline across diverse water types. Several crucial algorithmic refinements—specifically the introduction of the green-dominance bypass (S_1) and the continuous channel-shift overshoot safeguard (S_2) to contain UDCP color artifacts, as well as the regularized denominator ($\delta = 0.1$) to prevent numerical instability and input exclusion during score-weighted multi-scale fusion—emerged directly from these collaborative critiques rather than the original paper formulations.

6.3 Diversity Statement of Team

The team consists of undergraduate students of the same academic level, yet bringing distinct technical proficiencies and analytical perspectives. Individual strengths within the group spanned mathematical signal analysis and optical modeling, algorithmic implementation and numerical vectorization in MATLAB, and technical synthesis, experimental design, and visual communication. Responsibilities were deliberately structured to cultivate technical growth across these skill sets rather than simply exploiting existing comfort zones: members stronger in theoretical optics and derivation collaborated closely with those focused on vectorized array programming during the formulation of the multi-scale Laplacian pyramids, while all members participated actively in running synthetic experiments, tuning wrapper constants, and evaluating the failure modes of spatial histogram equalization.

Every design proposal was evaluated with equal consideration, whether analyzing the analytical cancellation of the retinex gain ν or evaluating the limitations of CLAHE under non-uniform torch illumination. Technical disagreements—such as determining whether to enforce hard candidate selection or implement an operator-guided score-weighted pyramid blend, and whether CLAHE should remain a mandatory stage or an optional candidate—were settled objectively by testing against quantitative metrics, radial profile evaluations, and reproducible visual previews rather than subjective preference.

6.4 Log Book of Project Implementation

Table 33 is the project log. The milestones follow the structure of the work; the dates and the individual roles are to be completed from the team's own records.

Table *31*: *Log book of the project*

Week	Milestone achieved	Individual role	Team role	Comments
Week 1	Literature survey of five key papers; Optical degradation formulation & notation setup	Theoretical study & mathematical modeling	Scope definition	Theoretical basis established; unified symbol glossary created to prevent notation collisions
Week 2	Architecture design; Common candidate interface & pre-processing pipeline setup	Driver script skeleton (run_one.m) & registry build	System architecture	Enforced bounded $[0,1]$ double-precision interface, $D_{\max} = 1024$ px limit, and error-handling framework
Week 3	Phase 1 implementation: Red-channel compensation & Minkowski norm white balance	Core algorithm formulation (wb_underwater.m)	Design validation	Verified $p = 6$ illuminant estimation; guaranteed non-negative boost ($\kappa_c \geq 1$) without

Week	Milestone achieved	Individual role	Team role	Comments
				channel reduction
Week 4	Phase 2 implementation: CIELAB conversion & localized L^* CLAHE mapping	Lightness histogram equalization pipeline build	Feature review	Evaluated 8×8 grid tiles; set conservative clip limit $\chi = 0.0035$ to cap maximum noise gain to $G_{\max} \approx 1.89$
Week 5	Phase 3 implementation: Multi-Scale Retinex with logarithmic Color Restoration (MSRCR)	Center/surround Gaussian filtering & CRF scripting	Algorithmic review	Multi-scale Gaussian widths ($\sigma_n = 15,80,250$) verified; global $1\%/99\%$ percentile display stretch confirmed
Week 6	Phase 4 implementation: Underwater Dark Channel Prior (UDCP) & guided filter refinement	15×15 minimum filtering & background light estimation	Verification	Excluded red channel; guided filtering applied for edge-preserving transmission map; capped gain via $t_0 = 0.15$
Week 7	Defensive safeguards integration for UDCP: Green-water bypass (S_1) & overshoot blend (S_2)	Wrapper design (cand_udcp.m) & threshold validation	Fault containment	Green-dominance ratio $\iota_g = 1.25$ and channel shift tolerance $\tau_{\text{os}} = 0.18$ deployed to mitigate severe color casts
Week 8	Phase 5 implementation: Two-input Multi-Scale Laplacian-Pyramid Fusion	Weight map generation (Laplacian, Saturation, Saliency) & pyramid build	Integration check	Invariants verified: partition of unity checked ($\sum W_k = 1$) and perfect reconstruction confirmed on identical inputs
Week 9	Generalization to K-input score-weighted blending engine	Operator input parser & multi-scale blending build	Functional review	Implemented operator score normalization with weight

Week	Milestone achieved	Individual role	Team role	Comments
	& montage generation			regularization $\delta = 0.10$ and minimum weight floor $\pi_{\min} = 0.05$
Week 10	Synthetic spotlight investigation: CLAHE gain vs. illumination gradient experiments	Synthetic dataset generation & radial profile profiling	Experimental analysis	Confirmed CLAHE preserves global spotlight contrast (95--98%) while amplifying background noise by predictable ceiling $G_{\max}$
Week 11	Parameter fine-tuning across real underwater photographs & execution benchmarking	Dataset testing & execution profiling via tic/toc	Performance evaluation	Validated candidate quality across clear blue, green, and turbid coastal scenes; confirmed Retinex complexity dominates
Week 12	Final project report compilation, stakeholder user manual, and software documentation	Comprehensive report writing & verification checks	Final review	Final report, codebase package architecture, and executive stakeholder documentation finalized for submission

7 Communication to External Stakeholders (PO(j))

7.1 Executive Summary

Murky water hides colour and detail. Our team built a MATLAB tool that rescues underwater photographs without special hardware or training data. From a single photo it produces five enhanced versions (colour-corrected, contrast-boosted, shadow-lifted, haze-reduced and detail-fused) and lets the user preview and blend them. Every step is explained from published research and the code is documented, so others can reuse it. The tool can support reef monitoring, fisheries studies and hull inspection by making underwater scenes clearer, while the original image is always kept for honest comparison.

7.2 User Manual

Requirements: MATLAB R2017b or later (the code uses `isfolder` and implicit array expansion) with the Image Processing Toolbox; the Computer Vision Toolbox is optional (caption text on the preview).

Installation:

(1) Copy the folder `+uw` (wrapper and helper functions) and the folder `methods` (the four core algorithms) into one project folder.

(2) In MATLAB, make the project folder the current folder or add it to the path. `run_one.m` adds its parent folder and the `methods` folder to the path automatically.

First run:

1. Call the driver with an image file: `pick = uw.run_one('photo.jpg');`
2. Wait while the six candidates are computed (progress lines `running wb ...` are printed). The numbered preview `preview.png` opens automatically on Windows.
3. Inspect the preview. Candidate 1 is the unmodified input, 2 is `wb`, 3 is `clahe`, 4 is `msrcr`, 5 is `udcp` and 6 is `fusion`.
4. At the prompt `> your choice:` type one of the following: a single number (for example `6`) to select one candidate; a list with scores (for example `4:60,5:30,6:10`) to blend several candidates; a list of numbers (for example `4 6`) for an equal blend; or `quit` to stop.
5. Open the results in `results/photo/`: `best.png` is the chosen or blended image, `montage.png` compares all candidates with the result, and `candidates/` holds each candidate.

Options: Pass a structure as the second argument: `opts.max_dim` (default 1024, use `Inf` to disable resizing), `opts.exclude` (for example `{'clahe'}` to drop a candidate), `opts.preset` (for example `'4:60,5:30,6:10'` to skip the prompt, which allows batch processing), `opts.open_preview` (`false` to suppress opening), `opts.verbose` and `opts.out_dir`.

Choosing a candidate: Use `wb` or `fusion` for a plain colour cast; `msrcr` for uneven lighting (watch for dark rings around lamps); `udcp` for hazy scenes in blue water (it returns the input unchanged in green water); `clahe` only for flat, evenly lit scenes (it amplifies noise in dark regions). Typical blends combine one candidate with a large score and one or two with small scores; remember that the scores are preferences, not exact percentages.

Adjusting constants: The constants of the pipeline are in the wrappers `cand_*.m` and in the core functions; Table 23 gives the file and identifier of every value. The most useful are: `ClipLimit` in

`cand_clahe.m`, `omega` and `t0` in `cand_udcp.m`, `gamma` in `cand_fusion.m`, and the scores of the selection string.

7.3 GitHub Link

Here is the GitHub link of the project respiratory:

https://github.com/tasirulumor/underwater-image-processing

8 Project Management and Cost Analysis (PO(k))

8.1 Bill of Materials

The project is software-only; there is no hardware bill of materials. Table 35 lists the resources that the project consumed. The amounts in yellow must be replaced by the team's real figures.

Table *32*: *Bill of materials (resources used)*

#	Item	Qty	Unit cost (BDT)	Total (BDT)	Remark
1	Laptop or PC for development (existing, depreciation share)	3	0	0	Used existing, one per member
2	MATLAB with Image Processing Toolbox (institutional licence)	1	0	0	Used open-sourced file
3	Test photographs (own or public)	10	0	0	Used open-sourced file available online
4	Internet and literature access	-	0	0	Institutionally provided
6	Report printing and binding	1	300 (approx.)	300 (approx.)	
	Total			210 (approx.)	

8.2 Calculation of Per Unit Cost of Prototype

The "unit" of this software prototype is one installation on one computer. Its cost is the sum of the resources of Table 35 that must be repeated for every installation (hardware share, licence, electricity) plus a share of the development effort:

$$C_{\text{unit}} = C_{\text{hw}} + C_{\text{lic}} + C_{\text{dev}}/N_{\text{inst}} \tag{8.1}$$

where C_{hw} is the computer share, C_{lic} the licence, C_{dev} the development cost (member-hours multiplied by an hourly rate) and N_{inst} the number of installations. The marginal cost of one processed image is only the electricity of (5.3).

8.3 Calculation of Per Unit Cost of Mass-Produced Unit

For distribution, the marginal cost of a software copy is practically zero, so the cost per unit is dominated by $C_{\text{dev}}/N_{\text{inst}}$ and by support. Two routes exist: a compiled stand-alone application (the MATLAB Runtime is free to end users, but a compiler licence is needed for the developer), or a port to Python and OpenCV, which removes licence costs and allows an embedded unit (for example a single-board computer in an underwater camera housing), for which (8.1) is applied with the price of that board.

8.4 Timeline of Project Implementation

Figure 10 shows the schedule by week; it is indicative and must be replaced by the team's actual dates. The critical path runs through the Phase 4 safeguards and Phase 5, because they depend on the working Phase 1 and on the common interface; the tuning step cannot start before the blending engine exists.

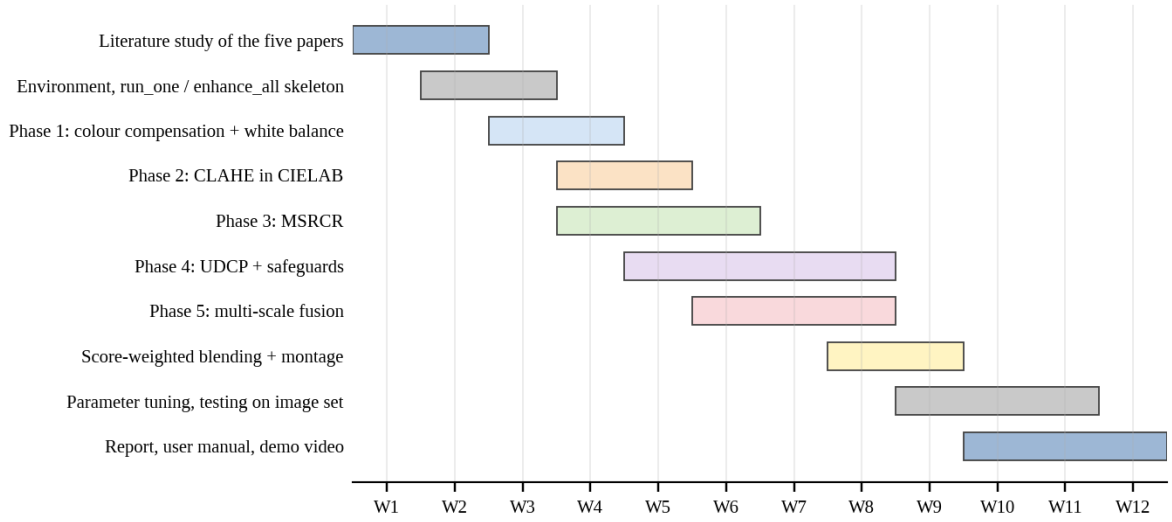


Figure *10*: *Indicative project schedule (weeks W1–W12), coloured by phase as in Figure 8. Replace with the actual dates of the team.*

Table *33*: *Risk register (team assessment, to be confirmed)*

Risk	Likelihood	Impact	Mitigation
Background light mis-estimated in UDCP	Medium	High	Safeguards S1 and S2; operator can reject the candidate
CLAHE amplifies noise and mottling	Medium	Medium	Conservative χ; optional candidate
Identifier collisions cause errors in the report	Medium	Medium	Unified notation and collision table (Section 3.2.1)
MATLAB licence or toolbox unavailable	Low	High	Port to an open-source environment (Chapter 9)
Limited team time near examinations	Medium	Medium	Modular work packages, indicative Gantt

9 Future Work (PO(l))

Life-long learning is part of the work: the project showed that the published constants of each paper are tied to image size, bit depth and water type, and that knowledge from several fields had to be combined. The following extensions are proposed.

1. **Quantitative evaluation.** Evaluate on a public benchmark such as UIEB [25] with the no-reference measures UCIQE [26] and UIQM [27], and compare each candidate and the blends with reference images where they exist.
2. **Automatic candidate scoring.** Replace the operator's scores by a no-reference quality score and use it in (3.48); this would make the system fully automatic.
3. **Illumination handling.** Replace the Gaussian surround of Phase 3 with an edge-preserving estimate (guided or bilateral) to suppress the halo, and correct the illumination before any equalization.
4. **Depth cues and per-channel transmission.** Implement the blurriness, maximum-intensity and red-channel cues of Peng and Cosman [5] to estimate the background light and a separate transmission for each channel.
5. **Check of the gamma direction** in Phase 5 and of the other E-class constants by a systematic ablation on the image set.
6. **Video.** Add temporal smoothing of the weights and of A_c for stable video enhancement.
7. **Portability.** Port the code to Python and OpenCV, publish it under an open-source licence, and test an embedded version; add a Bangla user manual.

10 References

[1] C. O. Ancuti, C. Ancuti, C. De Vleeschouwer, and P. Bekaert, "Color balance and fusion for underwater image enhancement," *IEEE Trans. Image Process.*, vol. 27, no. 1, pp. 379–393, Jan. 2018.

[2] K. Zuiderveld, "Contrast limited adaptive histogram equalization," in *Graphics Gems IV*, P. S. Heckbert, Ed. San Diego, CA, USA: Academic Press, 1994, pp. 474–485.

[3] D. J. Jobson, Z. Rahman, and G. A. Woodell, "A multiscale retinex for bridging the gap between color images and the human observation of scenes," *IEEE Trans. Image Process.*, vol. 6, no. 7, pp. 965–976, Jul. 1997.

[4] P. Drews Jr., E. do Nascimento, F. Moraes, S. Botelho, and M. Campos, "Transmission estimation in underwater single images," in *Proc. IEEE Int. Conf. Comput. Vis. Workshops (ICCVW)*, Sydney, NSW, Australia, 2013, pp. 825–830.

[5] Y.-T. Peng and P. C. Cosman, "Underwater image restoration based on image blurriness and light absorption," *IEEE Trans. Image Process.*, vol. 26, no. 4, pp. 1579–1594, Apr. 2017.

[6] K. He, J. Sun, and X. Tang, "Single image haze removal using dark channel prior," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 33, no. 12, pp. 2341–2353, Dec. 2011.

[7] K. He, J. Sun, and X. Tang, "Guided image filtering," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 35, no. 6, pp. 1397–1409, Jun. 2013.

[8] G. D. Finlayson and E. Trezzi, "Shades of gray and colour constancy," in *Proc. 12th Color Imaging Conf. (IS&T/SID)*, Scottsdale, AZ, USA, 2004, pp. 37–41.

[9] G. Buchsbaum, "A spatial processor model for object colour perception," *J. Franklin Inst.*, vol. 310, no. 1, pp. 1–26, 1980.

[10] P. J. Burt and E. H. Adelson, "The Laplacian pyramid as a compact image code," *IEEE Trans. Commun.*, vol. 31, no. 4, pp. 532–540, Apr. 1983.

[11] T. Mertens, J. Kautz, and F. Van Reeth, "Exposure fusion: A simple and practical alternative to high dynamic range photography," *Comput. Graph. Forum*, vol. 28, no. 1, pp. 161–171, 2009.

[12] R. Achanta, S. Hemami, F. Estrada, and S. Süsstrunk, "Frequency-tuned salient region detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Miami, FL, USA, 2009, pp. 1597–1604.

[13] S. M. Pizer *et al.*, "Adaptive histogram equalization and its variations," *Comput. Vis. Graph. Image Process.*, vol. 39, no. 3, pp. 355–368, 1987.

[14] D. J. Jobson, Z. Rahman, and G. A. Woodell, "Properties and performance of a center/surround retinex," *IEEE Trans. Image Process.*, vol. 6, no. 3, pp. 451–462, Mar. 1997.

[15] E. H. Land and J. J. McCann, "Lightness and retinex theory," *J. Opt. Soc. Am.*, vol. 61, no. 1, pp. 1–11, 1971.

[16] Z. Rahman, D. J. Jobson, and G. A. Woodell, "Retinex processing for automatic image enhancement," *J. Electron. Imaging*, vol. 13, no. 1, pp. 100–110, 2004.

[17] B. L. McGlamery, "A computer model for underwater camera systems," in *Proc. SPIE*, vol. 208, 1979, pp. 221–231.

[18] J. S. Jaffe, "Computer modeling and the design of optimal underwater imaging systems," *IEEE J. Ocean. Eng.*, vol. 15, no. 2, pp. 101–111, Apr. 1990.

[19] R. M. Pope and E. S. Fry, "Absorption spectrum (380–700 nm) of pure water. II. Integrating cavity measurements," *Appl. Opt.*, vol. 36, no. 33, pp. 8710–8723, 1997.

[20] Y.-T. Peng and P. C. Cosman, "Cameras and depth estimation of images acquired in a distorting medium," U.S. Patent 11 024 047 B2.

[21] The MathWorks, Inc., "Image Processing Toolbox documentation: adapthisteq, fspecial, imgaussfilt, imguidedfilter, imsharpen, rgb2lab," MathWorks Help Center.

[22] The MathWorks, Inc., "Contrast adaptive histogram equalization," Vision HDL Toolbox documentation, MathWorks Help Center.

[23] CIE, *Colorimetry*, 3rd ed., CIE 15:2004, Vienna, Austria, 2004; and IEC, *Multimedia systems and equipment: Colour measurement and management: Default RGB colour space (sRGB)*, IEC 61966-2-1, 1999.

[24] S. van der Walt *et al.*, "scikit-image: Image processing in Python," *PeerJ*, vol. 2, p. e453, 2014.

[25] C. Li *et al.*, "An underwater image enhancement benchmark dataset and beyond," *IEEE Trans. Image Process.*, vol. 29, pp. 4376–4389, 2020.

[26] M. Yang and A. Sowmya, "An underwater color image quality evaluation metric," *IEEE Trans. Image Process.*, vol. 24, no. 12, pp. 6062–6071, Dec. 2015.

[27] K. Panetta, C. Gao, and S. Agaian, "Human-visual-system-inspired underwater image quality measures," *IEEE J. Ocean. Eng.*, vol. 41, no. 3, pp. 541–551, Jul. 2016.

11 Appendix B: Full Source Code of MATLAB Software

The complete source code of the thirteen uploaded files follows (Consolas 7 pt, two columns). Comments were not present in the uploaded files; the explanation is given in Sections 3.2 and 4.1.

File: run_one.m

```
function pick = run_one(input_path, opts)
    if nargin < 2, opts = struct(); end
    here = fileparts(fileparts(mfilename('fullpath')));
    if ~isfield(opts,'out_dir'),      opts.out_dir      =
fullfile(here,'..','results'); end
    if ~isfield(opts,'max_dim'),      opts.max_dim      =
1024; end
    if ~isfield(opts,'exclude'),      opts.exclude      =
{};   end
    if ~isfield(opts,'open_preview'), opts.open_preview =
true; end
    if ~isfield(opts,'verbose'),      opts.verbose      =
true; end
    if ~isfield(opts,'preset'),       opts.preset       =
'';   end

    addpath(here); addpath(fullfile(here,'methods'));
    if ~isfolder(opts.out_dir), mkdir(opts.out_dir); end

    img = im2double(imread(input_path));
    if size(img,3) == 4, img = img(:,:,1:3); end
    [~, stem] = fileparts(input_path);
    if isfinite(opts.max_dim) &&
max(size(img,1),size(img,2)) > opts.max_dim
        img = imresize(img,
opts.max_dim/max(size(img,1),size(img,2)), 'bilinear');
    end
    odir = fullfile(opts.out_dir, stem);
    cdir = fullfile(odir, 'candidates');
    if ~isfolder(cdir), mkdir(cdir); end

    fprintf('[run_one] %s | generating candidates\n',
stem);
    cands = uw.enhance_all(img,
struct('exclude',{opts.exclude},'verbose',opts.verbose));
    for c = 1:numel(cands)
        imwrite(cands(c).image, fullfile(cdir,
[cands(c).name '.png']));
    end

    preview_path = fullfile(odir, 'preview.png');
    try, imwrite(numberedPreview_(cands), preview_path);
catch, end
    if opts.open_preview, try, winopen(preview_path);
catch, end; end

    n = numel(cands);
    fprintf('\nCandidates:\n');
    for i = 1:n
        fprintf('  [%d] %s\n', i, cands(i).name);
    end

    if isempty(opts.preset)
        fprintf('\nEnter candidates with weights, e.g.
4:60,5:30,6:10\n');
        fprintf('(a single number = that candidate; "quit"
to abort)\n');
        raw = strtrim(input('> your choice: ', 's'));
    else
        raw = strtrim(opts.preset);
    end

    if isempty(raw) || strcmpi(raw,'quit')
        fprintf('[run_one] %s | aborted\n', stem);
        pick = struct('name','', 'image',[], ...

'paths',struct('candidates',cdir,'preview',preview_path));
        return;
    end

    [idx, wt] = parseChoice_(raw, n);
    if isempty(idx)
        error('uw:run_one:badChoice', 'No valid candidate
numbers in "%s".', raw);
    end

    if numel(idx) == 1
        best = cands(idx).image;
        name = cands(idx).name;
    else
        wt = wt / sum(wt);
        imgs = cell(numel(idx),1);
        for k = 1:numel(idx), imgs{k} =
cands(idx(k)).image; end
        best = min(max(uw.blend_score_weighted(imgs, wt,
struct('weight_bias',1)), 0), 1);
        parts = arrayfun(@(k) sprintf('%s(%d%%)',
cands(idx(k)).name, round(100*wt(k))), ...
            1:numel(idx), 'UniformOutput', false);
        name = ['blend:' strjoin(parts, '+')];
    end

    best_path = fullfile(odir, 'best.png');
    mont_path = fullfile(odir, 'montage.png');
    imwrite(best, best_path);
    try, imwrite(uw.make_montage(cands,
struct('name',name,'image',best)), mont_path); catch, end

    fprintf('[run_one] %s | CHOSEN -> %s\n', stem, name);
    pick = struct('name',name, 'image',best, ...

'paths',struct('best',best_path,'montage',mont_path,'candid
ates',cdir,'preview',preview_path));
end

function [idx, wt] = parseChoice_(raw, n)
    tok = regexp(raw, '\(\s*(\d+)\s*,\s*(\d+)\s*\)',
'tokens');
    if isempty(tok)
        tok = regexp(raw, '(\d+)\s*:\s*(\d+)', 'tokens');
    end
    if ~isempty(tok)
        idx = cellfun(@(t) str2double(t{1}), tok);
        wt  = cellfun(@(t) str2double(t{2}), tok);
    else
        nums = sscanf(regexprep(raw, '[^0-9 ]', ' '),
'%d');
        idx = nums(:).';
        wt  = ones(1, numel(idx));
    end
    valid = idx >= 1 & idx <= n & isfinite(idx);
    idx = idx(valid); wt = wt(valid);
    [idx, ia] = unique(idx, 'stable'); wt = wt(ia);
    wt(wt <= 0) = 1;
end

function mont = numberedPreview_(cands)
    n = numel(cands); tileW = 480; barH = 28;
    [h0, w0, ~] = size(cands(1).image);
    tileH = max(64, round(h0 * tileW / max(w0,1)));
    tiles = cell(n,1);
    for i = 1:n
        t = min(max(im2double(cands(i).image),0),1);
        if size(t,3) == 4, t = t(:,:,1:3); end
        t = imresize(t, [tileH, tileW], 'bilinear');
        m = [t; zeros(barH, tileW, 3)];
        try
            m = insertText(m, [8, tileH+4], sprintf('%d)
%s', i, cands(i).name), ...

'FontSize',14,'BoxOpacity',0,'TextColor','white');
        catch
        end
        if size(m,1) ~= tileH+barH, m = imresize(m,
[tileH+barH, tileW]); end
        tiles{i} = m;
    end
    cols = 3; rows = ceil(n/cols);
    while numel(tiles) < rows*cols, tiles{end+1} =
ones(tileH+barH, tileW, 3); end
    strips = cell(rows,1);
    for r = 1:rows, strips{r} = cat(2, tiles{(r-
1)*cols+1:(r-1)*cols+cols}); end
    mont = min(max(cat(1, strips{:}), 0), 1);
end
```

File: enhance_all.m

```
function cands = enhance_all(img, opts)
    if nargin < 2, opts = struct(); end
    if ~isfield(opts,'exclude'), opts.exclude = {}; end
```

```
    if ~isfield(opts,'verbose'), opts.verbose = true; end

    img = im2double(img);
    if size(img,3) == 4, img = img(:,:,1:3); end
    img = min(max(img, 0), 1);

    registry = {
        'input',  @() img
        'wb',     @() uw.wb_underwater(img)
        'clahe',  @() uw.cand_clahe(img)
        'msrcr',  @() uw.cand_msrcr(img)
        'udcp',   @() uw.cand_udcp(img)
        'fusion', @() uw.cand_fusion(img)
    };
    keep = setdiff(registry(:,1), opts.exclude, 'stable');

    cands = struct('name', {}, 'image', {});
    for k = 1:numel(keep)
        name = keep{k};
        fn = registry{strcmp(registry(:,1), name), 2};
        if opts.verbose, fprintf('      running %s ...\n',
name); end
        try
            out = im2double(fn());
            if size(out,3) == 4, out = out(:,:,1:3); end
            cands(end+1).name = name;
            cands(end).image = min(max(out, 0), 1);
        catch ME
            warning('uw:enhance_all:cand_failed',
'Candidate "%s" failed (%s); skipping.', name, ME.message);
        end
    end
end
```

File: cand_clahe.m

```
function out = cand_clahe(img, cfg)
    if nargin < 2, cfg = struct(); end
    img = uw.wb_underwater(img);
    if ~isfield(cfg,'ClipLimit'), cfg.ClipLimit = 0.0035;
end
    if ~isfield(cfg,'NumTiles'),  cfg.NumTiles  = [8 8];
end
    if ~isfield(cfg,'Range'),     cfg.Range     =
'original'; end
    out = clahe_contrast(img, cfg);
    out = min(max(out, 0), 1);
end
```

File: cand_msrcr.m

```
function out = cand_msrcr(img, cfg)
    if nargin < 2, cfg = struct(); end
    if ~isfield(cfg,'sigmas'),  cfg.sigmas  = [15 80 250];
end
    if ~isfield(cfg,'alpha'),   cfg.alpha   = 100;
end
    if ~isfield(cfg,'beta'),    cfg.beta    = 40;
end
    if ~isfield(cfg,'clipPct'), cfg.clipPct = 1;
end
    img = im2double(img);
    if size(img,3) == 4, img = img(:,:,1:3); end
    out = msrcr_retinex(img, cfg);
    out = min(max(out, 0), 1);
end
```

File: cand_udcp.m

```
function out = cand_udcp(img, cfg)
    if nargin < 2, cfg = struct(); end
    if ~isfield(cfg,'patchSize'),     cfg.patchSize     =
15;    end
    if ~isfield(cfg,'omega'),         cfg.omega         =
0.5;   end
    if ~isfield(cfg,'t0'),            cfg.t0            =
0.15;  end
    if ~isfield(cfg,'topPercent'),    cfg.topPercent    =
0.001; end
    if ~isfield(cfg,'overshoot_tol'), cfg.overshoot_tol =
0.18;  end

    img = im2double(img);
    if size(img,3) == 4, img = img(:,:,1:3); end
    img = min(max(img, 0), 1);

    mR = mean(mean(img(:,:,1)));
    mG = mean(mean(img(:,:,2)));
    mB = mean(mean(img(:,:,3)));
    if mG > mB * 1.25 && mG > mR * 1.25
        out = img;
        return;
    end

    dehazed = udcp_dehaze(img, cfg);
    dehazed = min(max(dehazed, 0), 1);

    mIn  = squeeze(mean(mean(img,     1), 2));
    mOut = squeeze(mean(mean(dehazed, 1), 2));
    shift = max(abs(mOut - mIn));
    if shift > cfg.overshoot_tol
        w = min(1, (shift - cfg.overshoot_tol) /
cfg.overshoot_tol);
        out = (1 - w) * dehazed + w * img;
    else
        out = dehazed;
    end
    out = min(max(out, 0), 1);
end
```

File: cand_fusion.m

```
function [out, Wa, Wb] = cand_fusion(img, arg2, arg3)
    if nargin < 2, arg2 = struct(); end
    if nargin < 3, arg3 = struct(); end

    twoInput = isnumeric(arg2) && ndims(arg2) == 3 &&
size(arg2, 3) == 3;
    if twoInput
        A = im2double(img);
        B = im2double(arg2);
        cfg = arg3;
        if size(A, 3) == 4, A = A(:,:,1:3); end
        if size(B, 3) == 4, B = B(:,:,1:3); end
        [out, Wa, Wb] = ancuti_fusion(min(max(A,0),1),
min(max(B,0),1), cfg);
        out = min(max(out, 0), 1);
        return;
    end

    cfg = arg2;
    if ~isstruct(cfg), cfg = struct(); end
    if ~isfield(cfg,'gamma'),        cfg.gamma        =
1.5; end
    if ~isfield(cfg,'sharpAmount'),  cfg.sharpAmount  =
1.0; end
    if ~isfield(cfg,'sharpRadius'),  cfg.sharpRadius  = 2;
end

    base = uw.wb_underwater(img);

    input1 = base .^ (1/cfg.gamma);
    input1 = min(max(input1, 0), 1);

    input2 = zeros(size(base));
    for c = 1:3
        input2(:,:,c) = imsharpen(base(:,:,c), ...
            'Radius', cfg.sharpRadius, 'Amount',
cfg.sharpAmount);
    end
    input2 = min(max(input2, 0), 1);

    [out, Wa, Wb] = ancuti_fusion(input1, input2);
    out = min(max(out, 0), 1);
end
```

File: wb_underwater.m

```
function out = wb_underwater(img, cfg)
    if nargin < 2 || isempty(cfg), cfg = struct(); end
    if ~isfield(cfg,'alphaR'),      cfg.alphaR      = 1.0;
end
    if ~isfield(cfg,'alphaB'),      cfg.alphaB      = 0.0;
end
    if ~isfield(cfg,'minkowski_p'), cfg.minkowski_p = 6;
end
    if ~isfield(cfg,'clip'),        cfg.clip        = true;
end

    img = im2double(img);
    if size(img,3) == 4, img = img(:,:,1:3); end
    if size(img,3) ~= 3
        error('uw:wb_underwater:badInput', 'img must be
HxWx3 RGB.');
    end
    img = min(max(img, 0), 1);

    R = img(:,:,1); G = img(:,:,2); B = img(:,:,3);

    mR = mean(R(:)); mG = mean(G(:)); mB = mean(B(:));
    Rc = R + cfg.alphaR * (mG - mR) * (1 - R) .* G;
    Bc = B + cfg.alphaB * (mG - mB) * (1 - B) .* G;

    comp = cat(3, Rc, G, Bc);
    comp = min(max(comp, 0), 1);
```

```
    p = cfg.minkowski_p;
    illum = zeros(1,3);
    for c = 1:3
        ch = comp(:,:,c);
        illum(c) = mean(ch(:).^p)^(1/p);
    end
    illum = illum / (max(illum) + eps);
    gains = 1 ./ (illum + eps);

    out = zeros(size(comp));
    for c = 1:3, out(:,:,c) = comp(:,:,c) .* gains(c); end

    if cfg.clip
        out = min(max(out, 0), 1);
    end
end
```

File: clahe_contrast.m

```
function out = clahe_contrast(img, cfg)
    if nargin < 2
        cfg = struct();
    end
    if ~isfield(cfg, 'ClipLimit'), cfg.ClipLimit = 0.0025;
end
    if ~isfield(cfg, 'NumTiles'),  cfg.NumTiles  = [8 8];
end
    if ~isfield(cfg, 'Range'),     cfg.Range     =
'original'; end

    validateattributes(img, {'double'}, {'ndims', 3, '>=',
0, '<=', 1});
    if size(img,3) ~= 3
        error('clahe_contrast:badInput', ...
            'img must be an H x W x 3 RGB array (got
size(img,3) = %d).', size(img,3));
    end

    labImg = rgb2lab(img);
    L = labImg(:,:,1);
    a = labImg(:,:,2);
    b = labImg(:,:,3);

    L01 = L / 100;
    L01_eq = adapthisteq(L01, ...
        'ClipLimit', cfg.ClipLimit, ...
        'NumTiles',  cfg.NumTiles, ...
        'Range',     cfg.Range);
    L_eq = L01_eq * 100;

    labOut = cat(3, L_eq, a, b);
    out = lab2rgb(labOut);
    out = min(max(out, 0), 1);
end
```

File: msrcr_retinex.m

```
function out = msrcr_retinex(img, cfg)
    if nargin < 2 || isempty(cfg)
        cfg = struct();
    end
    cfg = fillDefaults(cfg);

    if ~isa(img, 'double')
        error('msrcr_retinex:badInput', 'Input must be a
double array in [0,1]. Use im2double first.');
    end
    img = min(max(img, 0), 1);
    epsVal = cfg.epsVal;

    imgSafe = img + epsVal;
    logI = log(imgSafe);

    numScales = numel(cfg.sigmas);
    R = zeros(size(img));
    for n = 1:numScales
        Gn = imgaussfilt(imgSafe, cfg.sigmas(n));
        R = R + (cfg.weights(n)) * (logI - log(Gn +
epsVal));
    end

    channelSum = sum(imgSafe, 3);
    CRF = cfg.beta * (log(cfg.alpha * imgSafe) -
log(channelSum));
    msrcr = CRF .* R;

    out = rescaleToDisplayGlobal(msrcr, cfg.clipPct);
    out = min(max(out, 0), 1);
end

function cfg = fillDefaults(cfg)
    defaults.sigmas  = [15 80 250];
    defaults.weights = [1 1 1]/3;
    defaults.alpha   = 125;
    defaults.beta    = 46;
    defaults.epsVal  = 1e-6;
    defaults.clipPct = 1;
    fn = fieldnames(defaults);
    for i = 1:numel(fn)
        if ~isfield(cfg, fn{i}) || isempty(cfg.(fn{i}))
            cfg.(fn{i}) = defaults.(fn{i});
        end
    end
end

function out = rescaleToDisplayGlobal(x, clipPct)
    lo = simplePercentile(x, clipPct);
    hi = simplePercentile(x, 100 - clipPct);
    if hi - lo < 1e-9
        out = 0.5 * ones(size(x));
    else
        out = (x - lo) / (hi - lo);
    end
end

function p = simplePercentile(v, pct)
    v = sort(v(:));
    n = numel(v);
    if n == 0
        p = 0;
        return;
    end
    idx = max(1, min(n, round(pct/100 * n)));
    p = v(idx);
end
```

File: udcp_dehaze.m

```
function out = udcp_dehaze(img, cfg)
    if nargin < 2
        cfg = struct();
    end
    if ~isfield(cfg, 'patchSize'),  cfg.patchSize  = 1;
end
    if ~isfield(cfg, 'omega'),      cfg.omega      = .1;
end
    if ~isfield(cfg, 't0'),         cfg.t0         = 0.10;
end
    if ~isfield(cfg, 'topPercent'), cfg.topPercent = 0.1;
end

    validateattributes(img, {'double'}, {'ndims', 3, '>=',
0, '<=', 1});
    if size(img,3) ~= 3
        error('udcp_dehaze:badInput', ...
            'img must be an H x W x 3 RGB array (got
size(img,3) = %d).', size(img,3));
    end

    [H, W, ~] = size(img);
    G = img(:,:,2);
    B = img(:,:,3);

    minGB = min(G, B);
    se = ones(cfg.patchSize, cfg.patchSize);
    darkChannel = ordfilt2(minGB, 1, se, 'symmetric');

    numPixels = numel(darkChannel);
    numTop = max(1, round(cfg.topPercent * numPixels));
    [~, sortedIdx] = sort(darkChannel(:), 'descend');
    topIdx = sortedIdx(1:numTop);

    A = zeros(1, 3);
    for c = 1:3
        channel = img(:,:,c);
        A(c) = mean(channel(topIdx));
    end

    safe_A_GB = max(A(2:3), 1e-6);
    normMinGB = min(cat(3, G, B) ./ reshape(safe_A_GB, 1,
1, 2), [], 3);
    darkNorm = ordfilt2(normMinGB, 1, se, 'symmetric');

    tRaw = 1 - cfg.omega * darkNorm;
    tRaw = min(max(tRaw, 0), 1);

    grayGuide = rgb2gray(img);
    tRefined = imguidedfilter(tRaw, grayGuide, ...
        'NeighborhoodSize', [2*cfg.patchSize+1,
2*cfg.patchSize+1], ...
        'DegreeOfSmoothing', 1e-3);
    tRefined = min(max(tRefined, cfg.t0), 1);
```

```
    out = zeros(H, W, 3);
    for c = 1:3
        Ic = img(:,:,c);
        out(:,:,c) = (Ic - A(c)) ./ max(tRefined, cfg.t0) +
A(c);
    end

    out = min(max(out, 0), 1);
end
```

File: ancuti_fusion.m

```
function [out, Wa, Wb] = ancuti_fusion(candidateA,
candidateB, cfg)
    if nargin < 3 || isempty(cfg)
        cfg = struct();
    end

    if ~isa(candidateA, 'double') || ~isa(candidateB,
'double')
        error('ancuti_fusion:badInput',
'candidateA/candidateB must both be double arrays in
[0,1].');
    end
    if ~isequal(size(candidateA), size(candidateB))
        error('ancuti_fusion:badInput', ...
            'candidateA and candidateB must be the same
size (got %s vs %s).', ...
           mat2str(size(candidateA)),
mat2str(size(candidateB)));
    end
    if ndims(candidateA) ~= 3 || size(candidateA,3) ~= 3
        error('ancuti_fusion:badInput', ...
            'candidateA/candidateB must be H x W x 3 RGB
arrays (got size %s).', mat2str(size(candidateA)));
    end
    candidateA = min(max(candidateA, 0), 1);
    candidateB = min(max(candidateB, 0), 1);

    [H, W, ~] = size(candidateA);
    cfg = fillDefaults(cfg, H, W);

    WA_agg = aggregatedWeight(candidateA, cfg);
    WB_agg = aggregatedWeight(candidateB, cfg);

    delta = cfg.weightRegularization;
    denom = WA_agg + WB_agg + 2*delta;
    Wa = (WA_agg + delta) ./ denom;
    Wb = (WB_agg + delta) ./ denom;

    sizes = computeLevelSizes(H, W, cfg.numLevels);
    numLevels = numel(sizes);

    La = genLaplacianPyramid(candidateA, sizes);
    Lb = genLaplacianPyramid(candidateB, sizes);
    Ga = genGaussianPyramid(Wa, sizes);
    Gb = genGaussianPyramid(Wb, sizes);

    Fused = cell(numLevels, 1);
    for l = 1:numLevels
        nCh = size(La{l}, 3);
        Ga3 = repmat(Ga{l}, [1 1 nCh]);
        Gb3 = repmat(Gb{l}, [1 1 nCh]);
        Fused{l} = Ga3 .* La{l} + Gb3 .* Lb{l};
    end

    out = collapsePyramid(Fused, sizes);
    out = min(max(out, 0), 1);
end

function cfg = fillDefaults(cfg, H, W)
    defaults.laplacianAlpha       = 0.2;
    defaults.saliencySigma        = max(4,
round(min(H,W)/16));
    defaults.weightRegularization = 0.1;
    defaults.numLevels            = [];
    fn = fieldnames(defaults);
    for i = 1:numel(fn)
        if ~isfield(cfg, fn{i}) || isempty(cfg.(fn{i}))
            cfg.(fn{i}) = defaults.(fn{i});
        end
    end
end

function W = aggregatedWeight(candidate, cfg)
    gray = rgb2gray(candidate);
    lapKernel = fspecial('laplacian', cfg.laplacianAlpha);
    WL = abs(imfilter(gray, lapKernel, 'replicate',
'same'));
    WSat = std(candidate, 0, 3);
    WS = abs(gray - imgaussfilt(gray, cfg.saliencySigma));
    W = WL + WSat + WS;
end

function sizes = computeLevelSizes(H, W,
numLevelsRequested)
    minDim = min(H, W);
    if isempty(numLevelsRequested)
        numLevelsRequested = max(1, floor(log2(minDim /
16)) + 1);
        numLevelsRequested = min(numLevelsRequested, 8);
    end
    maxLevels = max(1, floor(log2(minDim)) - 1);
    numLevels = max(1, min(numLevelsRequested, maxLevels));
    sizes = cell(numLevels, 1);
    sizes{1} = [H W];
    for l = 2:numLevels
        prev = sizes{l-1};
        sizes{l} = max(floor(prev/2), [4 4]);
    end
end

function out = pyrResize(im, targetSize)
    out = imresize(im, targetSize, 'bilinear');
end

function G = genGaussianPyramid(im, sizes)
    numLevels = numel(sizes);
    G = cell(numLevels, 1);
    G{1} = im;
    for l = 2:numLevels
        G{l} = pyrResize(G{l-1}, sizes{l});
    end
end

function L = genLaplacianPyramid(im, sizes)
    G = genGaussianPyramid(im, sizes);
    numLevels = numel(sizes);
    L = cell(numLevels, 1);
    for l = 1:numLevels-1
        expanded = pyrResize(G{l+1}, sizes{l});
        L{l} = G{l} - expanded;
    end
    L{numLevels} = G{numLevels};
end

function out = collapsePyramid(pyr, sizes)
    numLevels = numel(pyr);
    out = pyr{numLevels};
    for l = numLevels-1:-1:1
        out = pyrResize(out, sizes{l}) + pyr{l};
    end
end
```

File: blend_score_weighted.m

```
function out = blend_score_weighted(varargin)
    if iscell(varargin{1})
        imgs   = varargin{1};
        scores = double(varargin{2}(:));
        if nargin >= 3, cfg = varargin{3}; else, cfg =
struct(); end
    else
        if nargin < 4
            error('uw:blend_score_weighted:badArgs',
'Legacy call needs (A, B, sA, sB).');
        end
        imgs   = {varargin{1}, varargin{2}};
        scores = [double(varargin{3});
double(varargin{4})];
        if nargin >= 5, cfg = varargin{5}; else, cfg =
struct(); end
    end

    if ~isfield(cfg,'weight_bias'), cfg.weight_bias = 1.5;
end
    if ~isfield(cfg,'weight_min'),  cfg.weight_min  = 0.05;
end

    N = numel(imgs);
    if N < 1
        error('uw:blend_score_weighted:empty', 'No images
supplied.');
    end
    if numel(scores) ~= N
        error('uw:blend_score_weighted:sizeMismatch', 'Got
%d images but %d scores.', N, numel(scores));
    end

    for i = 1:N
        img = im2double(imgs{i});
        if size(img,3) == 4, img = img(:,:,1:3); end
        imgs{i} = min(max(img, 0), 1);
    end
```

```
    ref_size = size(imgs{1});
    for i = 2:N
        if ~isequal(size(imgs{i}), ref_size)
            imgs{i} = imresize(imgs{i}, [ref_size(1),
ref_size(2)], 'bilinear');
        end
    end

    if N == 1, out = imgs{1}; return; end

    s = max(0, scores);
    if sum(s) < eps
        g = ones(N,1) / N;
    else
        g = s / sum(s);
    end
    g = g .^ cfg.weight_bias;
    g = max(g, cfg.weight_min);
    g = g / sum(g);

    W = cell(N, 1);
    for i = 1:N
        W{i} = aggregatedWeight_(imgs{i}) * g(i);
    end

    delta = 0.10;
    denom = zeros(ref_size(1), ref_size(2));
    for i = 1:N, denom = denom + W{i} + delta; end
    w = cell(N, 1);
    for i = 1:N, w{i} = (W{i} + delta) ./ denom; end

    [H, Wd, ~] = size(imgs{1});
    sizes = computeLevelSizes_(H, Wd);
    numLevels = numel(sizes);

    L = cell(N, 1);
    G = cell(N, 1);
    for i = 1:N
        L{i} = laplacianPyramid_(imgs{i}, sizes);
        G{i} = gaussianPyramid_(w{i}, sizes);
    end

    fused = cell(numLevels, 1);
    for l = 1:numLevels
        acc = zeros(size(L{1}{l}));
        for i = 1:N
            wg3 = repmat(G{i}{l}, [1 1 3]);
            acc = acc + wg3 .* L{i}{l};
        end
        fused{l} = acc;
    end

    out = fused{numLevels};
    for l = numLevels-1:-1:1
        out = imresize(out, sizes{l}, 'bilinear') +
fused{l};
    end
    out = min(max(out, 0), 1);
end

function W = aggregatedWeight_(rgb)
    gray = rgb2gray(rgb);
    lap  = fspecial('laplacian', 0.2);
    WL   = abs(imfilter(gray, lap, 'replicate', 'same'));
    WSat = std(rgb, 0, 3);
    sig  = max(4, round(min(size(gray))/16));
    WS   = abs(gray - imgaussfilt(gray, sig));
    W    = WL + WSat + WS;
end

function sizes = computeLevelSizes_(H, W)
    minDim = min(H, W);
    numLevelsRequested = max(1, floor(log2(minDim / 16)) +
1);
    numLevelsRequested = min(numLevelsRequested, 8);
    maxLevels = max(1, floor(log2(minDim)) - 1);
    numLevels = max(1, min(numLevelsRequested, maxLevels));
    sizes = cell(numLevels, 1);
    sizes{1} = [H W];
    for l = 2:numLevels
        prev = sizes{l-1};
        sizes{l} = max(floor(prev/2), [4 4]);
    end
end

function G = gaussianPyramid_(im, sizes)
    numLevels = numel(sizes);
    G = cell(numLevels, 1);
    G{1} = im;
    for l = 2:numLevels
        G{l} = imresize(G{l-1}, sizes{l}, 'bilinear');
    end
end

function L = laplacianPyramid_(im, sizes)
    G = gaussianPyramid_(im, sizes);
    numLevels = numel(sizes);
    L = cell(numLevels, 1);
    for l = 1:numLevels-1
        expanded = imresize(G{l+1}, sizes{l}, 'bilinear');
        L{l} = G{l} - expanded;
    end
    L{numLevels} = G{numLevels};
end
```

File: make_montage.m

```
function montage_img = make_montage(cands, pick, cfg)
    if nargin < 3, cfg = struct(); end
    if ~isfield(cfg,'tileWidth'), cfg.tileWidth = 480; end
    if ~isfield(cfg,'cols'),      cfg.cols      = 3;   end
    if ~isfield(cfg,'barH'),      cfg.barH      = 28;  end

    n = numel(cands);
    labels = cell(n+1, 1);
    tiles  = cell(n+1, 1);
    for i = 1:n
        tiles{i}  = cands(i).image;
        labels{i} = sprintf('%d) %s', i, cands(i).name);
    end
    tiles{n+1}  = pick.image;
    labels{n+1} = sprintf('CHOSEN: %s', pick.name);

    [h0, w0, ~] = size(tiles{1});
    tileHeight = max(64, round(h0 * cfg.tileWidth / max(w0,
1)));
    for i = 1:numel(tiles)
        t = tiles{i};
        if size(t, 3) == 4, t = t(:,:,1:3); end
        t = min(max(im2double(t), 0), 1);
        t = imresize(t, [tileHeight, cfg.tileWidth],
'bilinear');
        tiles{i} = addCaptionBar_(t, labels{i}, cfg.barH);
    end

    cols = cfg.cols;
    rows = ceil(numel(tiles) / cols);
    fillH = tileHeight + cfg.barH;
    while numel(tiles) < rows * cols
        tiles{end+1} = ones(fillH, cfg.tileWidth, 3);
    end
    strip_rows = cell(rows, 1);
    for r = 1:rows
        rstart = (r-1)*cols + 1;
        rend   = r * cols;
        strip_rows{r} = cat(2, tiles{rstart:rend});
    end
    montage_img = cat(1, strip_rows{:});
    montage_img = min(max(montage_img, 0), 1);
end

function out = addCaptionBar_(tile, label, barH)
    W = size(tile, 2);
    bar = zeros(barH, W, 3);
    out = [tile; bar];
    try
        out = insertText(out, [8, size(tile, 1) + 4],
label, ...
            'FontSize', 14, 'BoxOpacity', 0, 'TextColor',
'white');
        if size(out, 1) ~= size(tile, 1) + barH
            out = imresize(out, [size(tile, 1) + barH, W]);
        end
    catch
    end
end
```